

Machine Learning Lanjutan & NLP Dasar

untuk SMK dan umum



2025

Kelas 11 Semester 1

Machine Learning Lanjutan & NLP Dasar

Aolia Ikhwanudin
Onno W. Purbo

**Institut Teknologi Tangerang Selatan (ITTS)
OnnoCenter
2025**

Lisensi & Catatan Karya

Karya ini dilisensikan di bawah lisensi **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)**.

Artinya, **siapa pun bebas untuk menggunakan, menyalin, membagikan, dan mengadaptasi** materi ini, **dengan syarat**:

- **Memberikan atribusi yang sesuai** (menyebut sumber asli),
- **Menyertakan lisensi yang sama** jika dimodifikasi atau dikembangkan lebih lanjut.

Lihat detail lisensi di sini <https://creativecommons.org/licenses/by-nd/4.0/legalcode.id>

Desain Cover: Irwan Siswanto

Disclaimer

Materi pembelajaran ini dibuat dengan **dana swadaya masyarakat Indonesia** dan **kontribusi sukarela dari para dosen di Institut Teknologi Tangerang Selatan (ITTS)**.

Karya ini didistribusikan secara bebas untuk mendukung pendidikan digital dan kecerdasan buatan di kalangan pelajar Indonesia.

Kami **memohon maaf** jika terdapat kekurangan dalam isi maupun penyajian materi ini. Kritik dan saran sangat kami harapkan untuk terus menyempurnakan karya serupa di masa depan.

Kata Pengantar

Buku ini disusun sebagai wujud nyata kepedulian sivitas akademika Institut Teknologi Tangerang Selatan (ITTS) terhadap perkembangan pendidikan kejuruan di Indonesia, khususnya bagi siswa Sekolah Menengah Kejuruan (SMK). Di tengah kemajuan teknologi yang begitu pesat, kami terdorong untuk menghadirkan materi pembelajaran **yang menantang, aplikatif, dan relevan dengan dunia industri digital**, agar siswa dapat mencapai **kemampuan maksimalnya** dalam memahami dan membangun sistem *Artificial Intelligence (AI)*.

Perlu kami sampaikan bahwa **materi dalam buku ini menuntut tingkat pemahaman teknis yang cukup tinggi**, terutama karena pembahasannya **sangat kental dengan praktik *coding* menggunakan bahasa Python** untuk membuat dan melatih model AI secara mandiri. Oleh karena itu, **tidak semua siswa, sekolah, maupun guru mungkin dapat langsung menyerap seluruh isi buku ini secara optimal**.

Bagi pembaca yang masih merasa kesulitan mengikuti langkah-langkah teknis yang disajikan, **kami sangat menyarankan untuk terlebih dahulu mempelajari buku “AI untuk SMA”**, yang disusun dengan pendekatan yang lebih **sederhana dan bertahap** sebelum melanjutkan ke tingkat SMK ini.

Penyusunan dan penerbitan buku ini dilakukan secara mandiri dengan semangat berbagi pengetahuan terbuka. Kami mengucapkan terima kasih kepada berbagai pihak yang telah mendukung dan memberi masukan dalam proses penulisan. Kritik dan saran yang membangun sangat kami harapkan agar edisi berikutnya dapat semakin baik dan mudah diakses oleh seluruh siswa Indonesia.

Kami berharap buku ini dapat menjadi **jembatan antara teori dan praktik AI**, sekaligus menginspirasi siswa SMK untuk **berpikir kreatif, logis, dan bertanggung jawab dalam menggunakan teknologi**. Dengan rendah hati, kami mohon doa dan dukungan agar karya sederhana ini dapat memberi manfaat luas dan menjadi amal jariyah bagi semua pihak yang terlibat.

Tangerang Selatan, Oktober 2025

Penulis

Daftar Isi

Disclaimer.....	2
Kata Pengantar.....	3
Daftar Isi.....	4
BAB 1: Review Supervised Learning.....	9
Tujuan Pembelajaran.....	9
Peta Konsep Sederhana.....	9
Apersepsi: Kamu Mau Prediksi Tim Sepak Bola yang Menang!.....	9
Penjelasan Teori.....	10
Apa itu Supervised Learning?.....	10
Cara Kerjanya.....	10
Jenis Tugas dalam Supervised Learning.....	10
Kenapa Penting?.....	10
Contoh Kasus Sederhana.....	11
Ringkasan Singkat.....	11
Kasus Ilustrasi.....	11
Soal Latihan & Pembahasan.....	15
Fun Facts!.....	19
Tips Belajar/Menghafal Cepat.....	19
Aktivitas Siswa.....	20
Rangkuman.....	25
Latihan Soal.....	26
Tugas Proyek.....	31
BAB 2: Unsupervised Learning (Clustering, PCA).....	39
Tujuan Pembelajaran.....	39
Peta Konsep Sederhana.....	39
Apersepsi: Kamu Mau Bikin Kelompok Studi Dadakan!.....	39
Penjelasan Teori.....	40
Apa Itu Unsupervised Learning?.....	40
Dua Teknik Utama.....	40
1. Clustering (Pengelompokan).....	40
2. Principal Component Analysis (PCA).....	40
Contoh Kasus Seru.....	40
tool Open Source yang Bisa Dipakai.....	41
Kenapa Penting?.....	41
Rangkuman.....	41
Kasus Ilustrasi.....	41
Soal Latihan & Pembahasan.....	45
Fun Facts!.....	49
Tips Belajar/Menghafal Cepat.....	49
Aktivitas Siswa.....	50
Rangkuman.....	55
Latihan Soal.....	56

Tugas Proyek.....	60
BAB 3: Pengantar Natural Language Processing (Tokenisasi, Stopwords, Stemming).....	65
Tujuan Pembelajaran.....	65
Peta Konsep Sederhana.....	65
Apersepsi: Bot Game yang Gak Nyambung.....	65
Penjelasan Teori.....	66
Kenapa NLP Penting?.....	66
Tiga (3) Proses Dasar NLP.....	66
1. Tokenisasi (Tokenization).....	66
2. Stopwords Removal.....	66
3. Stemming.....	66
Analogi Gado-Gado NLP.....	67
Tools Open Source untuk NLP.....	67
Rangkuman Singkat.....	67
Kasus Ilustrasi.....	67
Soal Latihan & Pembahasan.....	71
Catatan Praktek (opsional tapi direkomendasikan).....	74
Fun Facts!.....	75
Tips Belajar atau Menghafal Cepat.....	75
Aktivitas Siswa.....	75
Rangkuman.....	81
Latihan Soal.....	82
Bonus Kode Mini (opsional latihan cepat).....	87
Tugas Proyek.....	87
BAB 4: Analisis Teks dengan Python (Analisis Sentimen Sederhana).....	96
Tujuan Pembelajaran.....	96
Peta Konsep Sederhana.....	96
Apersepsi: Kamu Mau Tahu Kopi Mana yang Paling Disukai!.....	96
Penjelasan Teori.....	97
Apa Itu Analisis Sentimen?.....	97
Analogi yang Seru.....	97
Cara Kerja Paling Sederhana.....	97
Contoh Kode Python Sederhana.....	97
Kenapa Penting?.....	98
Kasus Ilustrasi.....	98
Soal Latihan & Pembahasan.....	103
Fun Facts!.....	106
Tips Belajar atau Menghafal Cepat.....	106
Aktivitas Siswa.....	106
Rangkuman.....	113
Latihan Soal.....	113
Tugas Proyek.....	117

BAB 5: Pengenalan Ollama dan Model LLM Offline.....	122
Tujuan Pembelajaran.....	122
Peta Konsep Sederhana.....	122
Apersepsi: Kamu Mau Ngerjain Tugas Tapi Gak Ada Kuota!.....	122
Penjelasan Teori.....	123
Konsep Dasar: Apa itu Ollama dan LLM?.....	123
Kenapa Kita Perlu Ollama?.....	123
Contoh Kasus di Kehidupan Siswa.....	123
Kaitan dengan Open Source Tools.....	123
Analogi Sederhana.....	124
Instalasi Ollama dan Open-WebUI.....	124
Ringkasan.....	124
Kasus Ilustrasi.....	124
Soal Latihan & Pembahasan.....	127
Fun Facts!.....	129
Tips Belajar/Menghafal Cepat.....	129
Aktivitas Siswa.....	129
Rangkuman.....	133
Latihan Soal.....	134
Tugas Proyek.....	137
BAB 6: Eksperimen Interaksi LLM lewat OpenWebUI.....	142
Tujuan Pembelajaran.....	142
Peta Konsep Sederhana.....	142
Apersepsi: Kamu Mau Pesan Makanan tapi Gak Ada Menu!.....	142
Penjelasan Teori.....	143
1. Kenapa Harus Pakai OpenWebUI?.....	143
2. Apa Itu OpenWebUI?.....	143
3. Analogi Seru.....	143
4. Kenapa Cocok untuk Siswa SMK?.....	143
5. Instalasi OpenWebUI dan Ollama.....	144
Kasus Ilustrasi.....	145
Soal Latihan & Pembahasan.....	148
Fun Facts!.....	151
Tips Belajar atau Menghafal Cepat.....	151
Aktivitas Siswa.....	151
Rangkuman.....	156
Latihan Soal.....	156
Tugas Proyek.....	160
BAB 7: Etika AI di Bidang Teks (Disinformasi, Hoax, Ujaran Kebencian).....	166
Tujuan Pembelajaran.....	166
Peta Konsep Sederhana.....	166
Apersepsi: Kamu Terkena Hoax!.....	166
Penjelasan Teori.....	167

Kenapa Etika AI itu Penting?.....	167
Istilah Inti (Wajib Paham).....	167
Kenapa AI Bisa Ikut “Salah Jalan”?.....	167
Prinsip Etika untuk Teks (Versi Anak SMK).....	168
Contoh Skenario.....	168
Kebiasaan Baik yang Wajib Dilatih.....	168
Ringkas Hafalan (Speedrun).....	169
Mindset Akhir.....	169
Kasus Ilustrasi.....	169
Soal Latihan & Pembahasan.....	174
Fun Facts!.....	178
Tips Belajar atau Menghafal Cepat.....	178
Aktivitas Siswa.....	178
Rangkuman.....	184
Latihan Soal.....	184
Tugas Proyek.....	187
BAB 8: Proyek – Analisis Sentimen Komentar Media Sosial dengan Python.....	192
Tujuan Pembelajaran.....	192
Peta Konsep Sederhana.....	192
Apersepsi: Kamu Mau Tahu Respon Penonton Film!.....	192
Penjelasan Teori.....	193
1) Alur Kerja (Pipeline) Sederhana.....	193
2) Data & Label.....	193
3) Pembersihan Teks (Preprocessing).....	193
4) Ubah Teks ke Angka (Feature Extraction).....	194
5) Tiga Jalur Analisis.....	194
a) Lexicon Rule-Based (daftar kata).....	194
b) Machine Learning.....	194
c) Neural Network dengan Keras.....	194
6) Evaluasi Hasil (Metrics).....	194
7) Visualisasi Hasil.....	195
8) Etika Singkat (Wajib!).....	195
9) Mini Contoh Kode (Ringkas Banget).....	195
A. Lexicon Rule-Based (cepat).....	195
B. TF-IDF + Logistic Regression (scikit-learn).....	195
C. Neural Network Sederhana (Keras).....	196
10) Checklist Proyek.....	196
11) Kamus Mini (Istilah Wajib).....	197
12) Kenapa Ini Keren?.....	197
Kasus Ilustrasi.....	197
Soal Latihan & Pembahasan.....	202
Fun Facts!.....	207
Tips Belajar atau Menghafal Cepat.....	207
Aktivitas Siswa.....	207

Rangkuman.....	213
Latihan Soal.....	213
Tugas Proyek.....	218
Institut Teknologi Tangerang Selatan (ITTS) dan Penulis.....	224

BAB 1: Review Supervised Learning

Siapa jadi ahli data yang jago bikin model super cerdas? Di bab ini, kita akan **mengulang** dan **memperdalam** materi **Supervised Learning** yang sudah kita pelajari. Ini penting banget, karena Supervised Learning adalah fondasi utama buat karier keren di bidang Data Science, dari memprediksi tren pasar sampai bikin AI di game!

Tujuan Pembelajaran

Setelah bab ini, kalian bakal bisa:

- Memahami kembali apa itu **Supervised Learning** dan mengapa dia penting.
- Membedakan model **Regresi** dan **Klasifikasi**.
- Menggunakan data berlabel dengan benar untuk melatih model.
- Menghubungkan konsep ini dengan masalah-masalah di dunia nyata.

Peta Konsep Sederhana

Bayangin Supervised Learning itu kayak ngajarin anak kecil.

- **Guru (Data Berlabel):** Kalian kasih dia gambar buah apel dan bilang, "Ini apel." Kalian ulangi terus-menerus.
- **Murid (Model):** Dia belajar dari gambar dan label itu.
- **Tes (Prediksi):** Kalian kasih gambar apel baru yang belum pernah dia lihat. Dia akan langsung tahu, "Ini apel!"

Itulah intinya Supervised Learning! Kita kasih data plus "jawabannya" biar modelnya bisa belajar dan tebak jawaban untuk data baru.

Apersepsi: Kamu Mau Prediksi Tim Sepak Bola yang Menang!

Kamu mau bikin bot yang bisa menebak tim mana yang bakal menang di pertandingan sepak bola.

- **Apa yang Kamu Butuhin?:** Kamu perlu data pertandingan sebelumnya. Misalnya, **Jumlah_Menang**, **Jumlah_Kalah**, **Jumlah_Seri**, dan yang paling penting, **Hasil_Akhir** (Menang atau Kalah).
- **Bagaimana Cara Kerjanya?:** Kamu kasih semua data itu ke bot. Kamu bilang, "Kalau tim A jumlah menangnya banyak dan jumlah kalahnya sedikit, dia bakal menang." Bot-mu akan belajar dari semua data itu dan suatu saat nanti bisa memprediksi, "Tim B akan menang di pertandingan besok!"

Ini adalah contoh nyata dari **Supervised Learning** di dunia game.

Penjelasan Teori

Apa itu Supervised Learning?

Bayangin kamu lagi ngajarin adik kecil mengenal buah. Kamu tunjukkan gambar apel, terus bilang, “Ini apel.” Kamu ulangi lagi dengan gambar lain, sambil selalu kasih tahu jawabannya. Lama-lama, adikmu bisa menebak sendiri: “Oh, ini apel!”

Nah, itulah inti dari **Supervised Learning**

- Kita kasih data yang sudah ada *jawabannya* (disebut *label*).
- Model (mesin belajar) akan berusaha mengerti pola.
- Setelah belajar, model bisa menebak jawaban baru dengan lebih pintar.

Cara Kerjanya

- **Data Berlabel (Guru):** contoh soal + kunci jawaban.
- **Model (Murid):** mesin yang belajar dari data.
- **Prediksi (Tes):** coba soal baru tanpa kunci jawaban.

→ Sama seperti latihan ujian. Kalau sering latihan, makin gampang jawab soal baru.

Jenis Tugas dalam Supervised Learning

Ada dua jenis utama:

1. **Regresi (*Regression*)**
 - Hasilnya berupa angka.
 - Contoh: prediksi harga rumah, nilai ujian, atau kecepatan internet.
 - Analogi: seperti menebak skor pertandingan dari rata-rata performa tim.
2. **Klasifikasi (*Classification*)**
 - Hasilnya berupa kategori.
 - Contoh: email spam atau tidak spam, pasien sakit atau sehat.
 - Analogi: seperti memilah cucian: putih atau berwarna.

Kenapa Penting?

- Hampir semua aplikasi keren yang kamu pakai sehari-hari menggunakan Supervised Learning.
- Netflix kasih rekomendasi film → Klasifikasi.
- Tokopedia prediksi harga promo → Regresi.
- Instagram filter komentar kasar → Klasifikasi.

Dan kabar baiknya, semua bisa dicoba pakai **open source tools** seperti:

- **Python** (bahasa pemrograman gratis, kuat banget buat data science).

- **Pandas** (buat olah data).
- **scikit-learn** (buat latihan model AI).
- **Keras + Neural Network** (buat model yang lebih canggih).
- **Orange Data Mining** (tool visual buat belajar cepat).

Contoh Kasus Sederhana

- **Prediksi nilai ujian:** belajar dari data jam belajar + nilai.
- **Prediksi komentar positif/negatif:** belajar dari teks di media sosial.
- **Prediksi harga barang:** belajar dari data penjualan sebelumnya.

Ringkasan Singkat

- **Supervised Learning = belajar dengan kunci jawaban.**
- Ada 2 jenis tugas: **Regresi** (angka) dan **Klasifikasi** (kategori).
- Bisa dipraktikkan pakai **Python + Pandas + scikit-learn + Keras + Orange**.
- Konsep ini jadi fondasi semua teknologi AI modern.

Kasus Ilustrasi

Kasus 1: Regresi — Prediksi Waktu Tempuh ke Sekolah

Ilustrasi:

Kamu ingin tahu butuh berapa menit sampai sekolah. Faktor yang memengaruhi: **jarak (km), tingkat macet (0–3), cuaca (cerah/hujan).**

Tujuan: Prediksi **waktu_minit** (angka).

Tipe: Regresi (*Regression*).

Pseudocode (tool: *Python/Pandas/scikit-learn* atau *Orange*):

```
# DATA
kumpulkan data: jarak_km, macet_level, cuaca, waktu_minit
bersihkan data: hilangkan nilai kosong, ubah cuaca → angka
(cerah=0, hujan=1)

# SPLIT
bagi data → train_set, test_set (contoh 80/20)

# MODEL
pilih model_regresi: LinearRegression atau DecisionTreeRegressor
latih model_regresi pada train_set (X_train → fitur, y_train →
waktu_minit)

# EVALUASI
prediksi = model_regresi(X_test)
hitung MAE/MSE
jika error besar → cek fitur, tambah data, atau pakai model lain
```

```
# PREDIKSI BARU
input_baru = (jarak=5, macet=2, cuaca=1)
waktu_prediksi = model_regresi(input_baru)
tampilkan hasil
```

Kasus 2: Klasifikasi — Filter Komentar Negatif

Ilustrasi:

Kamu ingin menyaring komentar kasar di media sosial kelas. Data: **teks_komentar** dan **label** (positif/negatif).

Tujuan: Prediksi **kelas** (negatif atau positif).

Tipe: Klasifikasi (*Classification*).

Pseudocode (tool: *Python/scikit-learn* atau *Orange*):

```
# DATA
kumpulkan: teks_komentar, label(positif/negatif)
bersihkan teks: huruf kecil, buang tanda baca, stopwords
sederhana

# FITUR TEKS
ubah teks → vektor (Bag-of-Words atau TF-IDF)

# SPLIT
bagi data → train/test

# MODEL
pilih model_klasifikasi: LogisticRegression / LinearSVM /
NaiveBayes
latih model_klasifikasi pada X_train_vec, y_train

# EVALUASI
pred_test = model_klasifikasi(X_test_vec)
hitung accuracy, precision, recall
cek *confusion matrix* untuk lihat salahnya di mana

# PREDIKSI BARU
komentar_baru = "ini jelek banget!"
vec_baru = vektorkan(komentar_baru)
hasil = model_klasifikasi(vec_baru)
tampilkan hasil
```

Catatan: Kalau mau *lebih canggih*, bisa pakai *Neural Network* sederhana di *Keras*, tapi mulai dulu dari model klasik biar cepat.

Kasus 3: Regresi — Prediksi Penjualan Jajanan Kantin

Ilustrasi:

Kakak kantin ingin tahu berapa **porsi terjual** hari ini. Fitur: **harga**, **ada_promo (0/1)**, **hari_dalam_minggu (1–7)**, **cuaca (0/1)**.

Tujuan: Prediksi **penjualan_harian** (angka).

Tipe: Regresi.

Pseudocode (tool: *Python/Pandas/scikit-learn* atau *Orange*):

```
# DATA
kumpulkan: harga, promo, hari, cuaca, penjualan_harian
normalisasi/standarisasi fitur angka jika perlu

# SPLIT
train/test

# MODEL
coba LinearRegression dan RandomForestRegressor
latih kedua model

# PILIH MODEL TERBAIK
bandingkan MAE/MAPE
pilih model dengan error terkecil

# PREDIKSI BARU
input_baru = (harga=12000, promo=1, hari=5, cuaca=0)
prediksi_porsi = model_terbaik(input_baru)
laporkan hasil + interval kasar (opsional pakai
*cross-validation*)
```

Kasus 4: Klasifikasi — Prediksi Hasil Pertandingan Sepak Bola

Ilustrasi:

Kamu ingin prediksi **menang/kalah** untuk tim favorit. Fitur: **jumlah_menang**, **jumlah_kalah**, **selisih_gol**, **home_away (0/1)**.

Tujuan: Prediksi **hasil** (menang/kalah).

Tipe: Klasifikasi.

Pseudocode (tool: *Python/scikit-learn* atau *Orange*):

```
# DATA
kumpulkan: menang, kalah, selisih_gol, home_away,
hasil(menang/kalah)
imbangi kelas jika tidak seimbang (opsional: class_weight atau
SMOTE)

# SPLIT
train/test (stratified agar perbandingan kelas stabil)

# MODEL
```

```

pilih: LogisticRegression / RandomForestClassifier
latih model

# EVALUASI
pred = model(X_test)
hitung accuracy + F1-score
lihat ROC-AUC (opsional)

# PREDIKSI BARU
match_baru = (menang=10, kalah=3, selisih_gol=+12, home_away=1)
hasil = model(match_baru)
tampilkan hasil + probabilitas

```

Kasus 5: Klasifikasi Gambar — Apel vs Buku (dengan *Keras Neural Network*)

Ilustrasi:

Folder gambar sederhana: **/apel/** dan **/buku/**. Tugasnya menebak apakah gambar baru itu apel atau buku.

Tujuan: Prediksi **kelas_gambar** (apel/buku).

Tipe: Klasifikasi gambar.

Catatan: Ini contoh pakai *Neural Network* di *Keras* (semua *open source*).

Pseudocode (tool: *Python/Keras/TensorFlow* atau *Orange – Image Analytics add-on*):

```

# DATA
siapkan folder:
    data/
        train/
            apel/, buku/
        val/
            apel/, buku/
augmentasi ringan: flip, rotate, zoom (opsional)

# ARSITEKTUR MODEL (CNN Sederhana)
model = urutan_lapisan([
    Conv2D(32, kernel=3), ReLU, MaxPool,
    Conv2D(64, kernel=3), ReLU, MaxPool,
    Flatten,
    Dense(64), ReLU,
    Dense(1), Sigmoid # 2 kelas → sigmoid
])

# TRAINING
kompilasi(loss="binary_crossentropy", optimizer="adam",
metric=["accuracy"])
latih model pada batch gambar (train) dan validasi (val) selama N
epoch

# EVALUASI

```

```
cek akurasi val, simpan model terbaik

# PREDIKSI BARU
muat gambar_baru → ubah ukuran → normalisasi → prediksi
jika output > 0.5 → "apel" else "buku"
tampilkan label
```

Tips Umum Biar Makin Nendang

- Selalu tanya: **outputku angka atau kategori?** → itu kunci pilih *regresi* atau *klasifikasi*.
- Pakai *train-test split* dan cek metrik. Jangan cuma lihat *accuracy*, lihat juga **F1/MAE** sesuai tugas.
- Data sedikit? Mulai dari model sederhana. Nanti naik kelas ke *Neural Network* di *Keras*.
- Semua bisa dikerjakan pakai **tool open source**: *Python*, *Pandas*, *scikit-learn*, *Keras*, *Orange*.

Soal Latihan & Pembahasan

Soal 1: Regresi — Prediksi Nilai Ujian dari Jam Belajar

Soal (narasi pendek):

Kamu punya data **jam_belajar** dan **nilai_ujian**. Buat model yang bisa **memrediksi nilai** kalau seorang siswa belajar **6 jam**.

Data ringkas (contoh):

jam_belajar = [1,2,3,4,5] → **nilai** = [55,60,65,70,80]

Tanya: Prediksi nilai saat **6 jam** belajar?

Jawaban singkat: Sekitar **85–90** (perkiraan dari pola linear).

Pembahasan:

- Ini **regresi** karena output **angka**.
- Pola naik: tiap tambah jam, nilai ikut naik.
- Model sederhana: *Linear Regression*.

Pseudocode:

```
# PERSIAPAN
data = (jam_belajar, nilai_ujian)
X = jam_belajar.reshape(-1, 1)
y = nilai_ujian

# MODEL
model = LinearRegression()
model.fit(X, y)
```



```
# PREDIKSI
prediksi = model.predict([[6]])
tampilkan(prediksi)
```

Soal 2: Klasifikasi — Deteksi Komentar Negatif

Soal (narasi pendek):

Ada data komentar dengan label **positif/negatif**. Kamu mau **memfilter komentar kasar**.

Data ringkas (contoh):

- “bagus sekali” → positif
- “jelek banget” → negatif

Tanya: Termasuk tugas apa dan model apa yang cocok?

Jawaban singkat: **Klasifikasi**; model cocok: **Logistic Regression / SVM / Naive Bayes**.

Pembahasan:

- Output **kategori** (positif/negatif).
- Teks perlu diubah ke angka: *Bag-of-Words* atau *TF-IDF*.
- Ukur performa dengan **accuracy, precision, recall, F1**.

Pseudocode:

```
# PERSIAPAN TEKS
komentar, label = muat_dataset()
teks_bersih = bersihkan(komentar) # lower, hapus tanda baca, stopwords
X = tfidf_vectorize(teks_bersih)
y = label

# SPLIT & LATIH
X_train, X_test, y_train, y_test = split(X, y, test_size=0.2, stratify=y)
clf = LogisticRegression()
clf.fit(X_train, y_train)

# EVALUASI
pred = clf.predict(X_test)
hitung_metric(pred, y_test) # accuracy, precision, recall, f1

# PAKAI
baru = "karya kamu jelek!"
pred_baru = clf.predict(tfidf_vectorize([baru]))
tampilkan(pred_baru)
```

Soal 3: Regresi — Prediksi Penjualan Jajanan Kantin

Soal (narasi pendek):

Kakak kantin ingin menebak **berapa porsi** yang terjual hari ini berdasarkan **harga**, **promo (0/1)**, **hari (1–7)**, **cuaca (0/1)**.

Tanya: Jenis tugas? Contoh model? Cara mengevaluasi?

Jawaban singkat: Regresi; model: **Linear Regression** atau **Random Forest Regressor**; metrik: **MAE/MSE/MAPE**.

Pembahasan:

- Output berupa **angka** (porsi).
- Coba beberapa model, pilih yang **error** paling kecil.
- Fitur bisa dinormalisasi agar stabil.

Pseudocode:

```
# DATA
X = [harga, promo, hari, cuaca]
y = penjualan_porsi

# SPLIT
X_tr, X_te, y_tr, y_te = split(X, y, 0.2)

# COBA 2 MODEL
lin = LinearRegression().fit(X_tr, y_tr)
rf = RandomForestRegressor().fit(X_tr, y_tr)

# NILAI ERROR
err_lin = MAE(lin.predict(X_te), y_te)
err_rf = MAE(rf.predict(X_te), y_te)

# PILIH TERBAIK
model_terbaik = rf if err_rf < err_lin else lin

# PREDIKSI BARU
input_baru = [12000, 1, 5, 0]
pred = model_terbaik.predict([input_baru])
tampilkan(pred)
```

Soal 4: Klasifikasi — Prediksi Menang/Kalah Sepak Bola

Soal (narasi pendek):

Fitur: **jumlah_menang**, **jumlah_kalah**, **selisih_gol**, **home_away (0/1)**.
Label: **hasil** = menang/kalah.

Tanya: Termasuk apa? Bagaimana menangani data tidak seimbang?

Jawaban singkat: **Klasifikasi**; gunakan **stratified split** dan bisa pakai **class_weight** atau **SMOTE**.

Pembahasan:

- Output **kategori** (menang/kalah).
- Kalau data **imbalance**, akurasi bisa menipu.
- Lihat **F1-score** dan **ROC-AUC**.

Pseudocode:

```
# DATA
X = [menang, kalah, selisih_gol, home_away]
y = hasil # menang/kalah

# SPLIT TERSTRATIFIKASI
X_tr, X_te, y_tr, y_te = split(X, y, test_size=0.2, stratify=y)

# MODEL + CLASS WEIGHT
clf = LogisticRegression(class_weight="balanced")
clf.fit(X_tr, y_tr)

# EVALUASI
pred = clf.predict(X_te)
prob = clf.predict_proba(X_te)
metric = hitung(accuracy, f1, roc_auc)

# PREDIKSI BARU
match_baru = [10, 3, 12, 1]
hasil = clf.predict([match_baru])
tampilkan(hasil, prob)
```

Soal 5: Klasifikasi Gambar — Apel vs Buku (dengan Keras)

Soal (narasi pendek):

Folder gambar sederhana: **apel/** dan **buku/**.

Tugas: tebak gambar baru termasuk **apel** atau **buku**.

Tanya: Jenis tugas? Arsitektur sederhana?

Jawaban singkat: **Klasifikasi**; pakai **CNN sederhana** di *Keras* (semua *open source*).

Pembahasan:

- Gambar perlu dinormalisasi dan di-*resize*.
- *CNN* belajar pola tepi/tekstur bentuk.
- Pakai **augmentasi** agar model lebih kuat.

Pseudocode:

```
# DATA IMAGE
train_dir = "data/train/{apel,buku}"
val_dir   = "data/val/{apel,buku}"
```

```

train_gen = image_generator(augment=True)
val_gen    = image_generator(augment=False)

# MODEL CNN SEDERHANA
model = Sequential([
    Conv2D(32, 3), ReLU(), MaxPool2D(),
    Conv2D(64, 3), ReLU(), MaxPool2D(),
    Flatten(),
    Dense(64), ReLU(),
    Dense(1), Sigmoid() # biner: apel vs buku
])

# TRAINING
model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])
model.fit(train_gen, validation_data=val_gen, epochs=10)

# PREDIKSI
img = muat_gambar("uji.jpg", size=(128,128))
p    = model.predict(img)
label = "apel" jika p>0.5 else "buku"
tampilkan(label)

```

Ringkas & Mantap

- **Regresi** = tebak **angka** (nilai, porsi, waktu).
- **Klasifikasi** = tebak **kategori** (positif/negatif, menang/kalah, apel/buku).
- Workflow inti: **siapkan data** → **bagi train/test** → **pilih model** → **latih** → **evaluasi** → **prediksi baru**.
- Semua bisa dikerjakan pakai **tool open source**: *Python, Pandas, scikit-learn, Keras, Orange*.

Fun Facts!

- **Netflix** menggunakan Supervised Learning untuk memprediksi film yang mungkin kamu suka. Makanya, rekomendasi di Netflix terasa pas banget!
- Di dunia kerja, **data berlabel** itu harganya mahal, lho! Butuh waktu dan tenaga manusia untuk melabeli data, makanya ada pekerjaan khusus **data annotator**.
- **scikit-learn** dan **Orange Data Mining** adalah dua tool paling populer di industri. Jadi, kalian sudah belajar hal yang dipakai di perusahaan-perusahaan besar!

Tips Belajar/Menghafal Cepat

- **Fokus pada Output**: Setiap kali melihat masalah, tanyakan, "Apa yang mau aku tebak? Kategori atau angka?" Itu kuncinya!
- **Buat Analogi Sendiri**: Pikirkan masalah di sekitarmu, lalu coba buat analogi Machine Learning yang sederhana.

- **Gunakan Flashcard:** Tulis nama model di depan kartu, dan di belakangnya tulis penjelasan singkat.

Aktivitas Siswa

Aktivitas 1: Bangun Dataset Mini + Simpan ke CSV

Tujuan: Pahami *data berlabel* untuk *supervised learning*.

Hasil akhir: File `kantin.csv`.

Langkah singkat

- Tentukan fitur: `harga`, `promo(0/1)`, `hari(1-7)`, `cuaca(0/1)`.
- Target: `porsi_terjual` (angka $\rightarrow$ *regresi*).
- Susun tabel, simpan ke CSV.

Pseudocode

```
buat_tabel([
    {harga, promo, hari, cuaca, porsi_terjual},
    ...
])
simpan_csv("kantin.csv")
```

Python

```
import pandas as pd

data = [
    {"harga":12000,"promo":1,"hari":1,"cuaca":0,"porsi_terjual":35},
    {"harga":12000,"promo":0,"hari":2,"cuaca":1,"porsi_terjual":22},
    {"harga":10000,"promo":1,"hari":5,"cuaca":0,"porsi_terjual":50},
    {"harga":10000,"promo":0,"hari":6,"cuaca":0,"porsi_terjual":31},
    {"harga":11000,"promo":1,"hari":3,"cuaca":1,"porsi_terjual":28},
]

df = pd.DataFrame(data)
df.to_csv("kantin.csv", index=False)
print("Tersimpan: kantin.csv"); print(df)
```

Aktivitas 2: Regresi Linear: Jam Belajar $\rightarrow$ Nilai

Tujuan: Latih model *regresi* paling dasar.

Hasil akhir: Prediksi nilai saat `jam_belajar=6`.

Langkah singkat

- Baca data `jam_belajar` $\rightarrow$ `nilai`.
- *Train-test split*.

- Latih **LinearRegression**.
- Ukur *MAE*.
- Prediksi nilai saat 6 jam.

Pseudocode

```
muat_data()
bagi_train_test()
model = LinearRegression()
latih(model)
ukur_error(MAE)
prediksi(6_jam)
```

Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

df = pd.DataFrame({
    "jam_belajar": [1, 2, 3, 4, 5, 6, 2, 3, 4, 5],
    "nilai": [55, 60, 63, 70, 78, 85, 58, 64, 69, 79]
})
X = df[["jam_belajar"]]; y = df["nilai"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
random_state=42)

model = LinearRegression().fit(X_tr, y_tr)
pred = model.predict(X_te)
print("MAE:", mean_absolute_error(y_te, pred))
print("Prediksi nilai saat 6 jam:", model.predict([[6]])[0])
```

Output

```
MAE: 1.8396226415094326
Prediksi nilai saat 6 jam: 84.0566037735849
```

Aktivitas 3: *Klasifikasi Teks: Filter Komentar Negatif (TF-IDF + Logistic Regression)*

Tujuan: Latih model *klasifikasi* sederhana untuk teks.

Hasil akhir: Model penanda komentar **negatif**.

Langkah singkat

- Siapkan pasangan **teks—label(positif/negatif)**.
- *Preprocess*: huruf kecil, hapus tanda baca (sederhana).
- Ubah teks → angka pakai **TF-IDF**.
- Latih **LogisticRegression**.
- Laporkan metrik: *accuracy, precision, recall, F1*.

Pseudocode

```
teks_bersih = preprocess(teks)
X = TFIDF(teks_bersih); y = label
train_test_split(stratify)
latih(LogisticRegression)
evaluasi(classification_report)
```

Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

df = pd.DataFrame({
    "teks": [
        "keren sekali produk ini",
        "bagus dan cepat sampai",
        "jelek banget, parah!",
        "pelayanan buruk",
        "mantap rekomendasi",
        "tidak sesuai harapan, kecewa"
    ],
    "label": ["positif", "positif", "negatif", "negatif", "positif", "negatif"]
})
X_train, X_test, y_train, y_test = train_test_split(
    df["teks"], df["label"], test_size=0.33, random_state=42,
    stratify=df["label"]
)
tfidf = TfidfVectorizer(lowercase=True)
X_tr, X_te = tfidf.fit_transform(X_train),
tfidf.transform(X_test)

clf = LogisticRegression(max_iter=1000).fit(X_tr, y_train)
pred = clf.predict(X_te)
print(classification_report(y_test, pred))
print("Contoh prediksi:", clf.predict(tfidf.transform(["ini jelek banget!"])[0])
```

Output

	precision	recall	f1-score	support
negatif	0.50	1.00	0.67	1
positif	0.00	0.00	0.00	1
accuracy			0.50	2
macro avg	0.25	0.50	0.33	2
weighted avg	0.25	0.50	0.33	2

Soal 4: *Model Evaluation* yang Jujur: *Confusion Matrix* + F1

Tujuan: Pahami kenapa *accuracy* saja bisa menipu pada data tidak seimbang (*imbalanced*).

Hasil akhir: Laporan **confusion matrix** + **F1-score**.

Langkah singkat

- Pakai model pada Aktivitas 3 (teks).
- Hitung **confusion_matrix**.
- Lihat **precision/recall/F1**.
- Catat mana kelas yang paling sering keliru.

Pseudocode

```
pred = model.predict(X_test)
cm = confusion_matrix(y_test, pred)
lapor(precision, recall, f1)
analisis_kesalahan(cm)
```

Python

```
from sklearn.metrics import confusion_matrix,
precision_recall_fscore_support
import numpy as np

# gunakan y_test & pred dari Aktivitas 3
labels = sorted(list(set(y_test)))
cm = confusion_matrix(y_test, pred, labels=labels)
print("Labels:", labels)
print("Confusion Matrix:\n", cm)

prec, rec, f1, _ = precision_recall_fscore_support(y_test, pred,
labels=labels, average=None)
for i, lab in enumerate(labels):
    print(f"{lab} -> precision={prec[i]:.2f},
recall={rec[i]:.2f}, f1={f1[i]:.2f}")
print("Macro-F1:", f1.mean())
```

Output

```
Contoh prediksi: negatif
Labels: ['negatif', 'positif']
Confusion Matrix:
[[1 0]
 [1 0]]
negatif -> precision=0.50, recall=1.00, f1=0.67
positif -> precision=0.00, recall=0.00, f1=0.00
Macro-F1: 0.3333333333333333
```


Soal 5: *Neural Network* (Keras): Kelulusan (0/1) dari Fitur Sederhana

Tujuan: Coba *neural network* tabular yang *ringkas*.

Hasil akhir: Probabilitas **lulus** berdasarkan fitur.

Langkah singkat

- Data: `jam_belajar`, `hadir_pct`, `tugas_pct` → `lulus(0/1)`.
- *Standardize* fitur.
- Bangun jaringan: `Dense(16) → Dense(8) → Dense(1, sigmoid)`.
- *Train 20 epoch*.
- Prediksi sampel baru.

Pseudocode

```
Xs = standardize(X)
model = Sequential([Dense(16, 'relu'), Dense(8, 'relu'),
Dense(1, 'sigmoid')])
compile(adam, binary_crossentropy)
fit(Xs_train, y_train, epochs=20)
predict(sample)
```

Python

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# data fiktif
df = pd.DataFrame({
    "jam_belajar": [1, 2, 3, 4, 5, 2, 3, 4, 5, 6, 3, 5, 6, 7, 2, 4, 6, 8],
    "hadir_pct": [60, 65, 70, 75, 80, 62, 71, 78, 84, 90, 68, 82, 88, 93, 64, 77, 89, 95],
    "tugas_pct": [50, 55, 60, 70, 80, 52, 61, 72, 81, 90, 58, 78, 86, 92, 54, 69, 85, 94],
})
df["lulus"] = ((df["jam_belajar"]*8 + df["hadir_pct"]*0.3 +
df["tugas_pct"]*0.4) > 90).astype(int)

X = df[["jam_belajar", "hadir_pct", "tugas_pct"]].values
y = df["lulus"].values
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
random_state=42, stratify=y)

scaler = StandardScaler()
X_trs = scaler.fit_transform(X_tr); X_tes = scaler.transform(X_te)

model = Sequential([
    Dense(16, activation="relu", input_shape=(X_trs.shape[1],)),
    Dense(8, activation="relu"),
```

```

        Dense(1, activation="sigmoid")
    ])
    model.compile(optimizer="adam", loss="binary_crossentropy",
                  metrics=["accuracy"])
    model.fit(X_trs, y_tr, epochs=20, batch_size=4, validation_data=(X_tes,
                                                                      y_te), verbose=0)

    sampel = np.array([[5, 85, 82]])
    p = model.predict(sampel.astype(float).reshape(1,-1) @
                     np.diag(scaler.scale_) + scaler.mean_, verbose=0)
    # Cara benar: transform dulu:
    p = model.predict(scaler.transform(sampel), verbose=0)[0][0]
    print("Prob lulus:", float(p), "→", "LULUS" if p>=0.5 else "TIDAK
    LULUS")

```

Output

```
Prob lulus: 0.6704986691474915 → LULUS
```

Catatan

- Selalu tanya: **output angka atau kategori?** → *regresi* vs *klasifikasi*.
- Pakai *train–test split* dan metrik yang tepat: *MAE/MSE* (regresi), *F1* (klasifikasi).
- Mulai dari model sederhana. Naik kelas ke *Neural Network* di **Keras** kalau data cukup.
- Semua tool di atas **gratis** dan *open source*: *Python*, *Pandas*, *scikit-learn*, *Keras*.

Rangkuman

Berani jadi “pelatih AI”? **Supervised Learning** itu seperti kamu ngajari model dengan **contoh + kunci jawaban**. Kita kasih *data berlabel* (fitur + label), model belajar pola, lalu **menebak data baru**. Dua tugas utamanya: **Regresi** (hasil **angka**) dan **Klasifikasi** (hasil **kategori**). Ingat rumus cepat: **tanya output-nya**—kalau angka → *regression*, kalau kategori → *classification*. Inilah fondasi banyak sistem pintar di sekitar kita, dari rekomendasi film sampai filter komentar.

Supaya makin nempel, bayangkan analogi **guru–murid–ujian**: *guru* = **data berlabel**, *murid* = **model**, *ujian* = **prediksi**. **Regresi** cocok untuk tebak **harga, suhu, jumlah**; **Klasifikasi** pas untuk tebak **ya/tidak, kelas A/B, spam/tidak**. Yang penting, **data rapi**: bersihkan nilai kosong, ubah teks ke angka (*TF-IDF*), normalisasi angka bila perlu. Evaluasi juga harus tepat: **MAE/MSE** untuk *regresi*; **precision, recall, F1** untuk *klasifikasi*. Jangan tertipu **akurasi tinggi** pada data **tidak seimbang** (*imbalanced*).

Semua bisa kamu coba dengan tool *open source*: **Python**, **Pandas**, **scikit-learn**, **Keras** (*neural network*), dan **Orange Data Mining**. Alur kerja sederhana: **siapkan data** → **bagi train/test** → **latih model** → **ukur metrik** → **prediksi baru**. Kuasai pola ini, dan kamu sudah pegang kunci untuk proyek nyata: **prediksi nilai, deteksi spam, peramal penjualan**,

sampai **klasifikasi gambar**. Ringkasnya: **pahami data, pilih tugas yang benar, gunakan metrik yang tepat**—dan *Supervised Learning* akan jadi senjata andalanmu di dunia *Data Science* dan *Machine Learning*.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. *Supervised learning* selalu memakai data berlabel.
Jawaban: Benar
2. *Regresi* memprediksi kategori.
Jawaban: Salah
3. *Klasifikasi* cocok untuk tugas spam/tidak spam.
Jawaban: Benar
4. *Akurasi* selalu cukup untuk data yang *imbalanced*.
Jawaban: Salah
5. *Train-test split* dipakai untuk menguji model pada data baru.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Output *regresi* adalah...
A. Kategori
B. Angka
C. Gambar
D. Teks mentah
Jawaban: B
2. Output *klasifikasi* adalah...
A. Angka kontinu
B. Kategori/kelas
C. Deret waktu
D. Sinyal audio
Jawaban: B
3. Metrik yang umum untuk *regresi* adalah...
A. F1-score
B. MAE/MSE
C. AUC
D. Silhouette

Jawaban: B

4. Metrik yang baik untuk *klasifikasi* pada data tidak seimbang adalah...

A. MAE
B. MSE
C. F1-score
D. RMSE

Jawaban: C

5. Contoh fitur untuk prediksi harga rumah:

A. Warna font di dokumen
B. Luas bangunan
C. Jenis font
D. Jumlah huruf vokal

Jawaban: B

6. Library *Python* untuk *machine learning* klasik:

A. scikit-learn
B. matplotlib
C. pillow
D. pygame

Jawaban: A

7. Fungsi *scikit-learn* untuk *train-test split*:

A. `sklearn.model_selection.train_test_split`
B. `sklearn.linear_model.train_test_split`
C. `sklearn.metrics.train_test_split`
D. `sklearn.split.train_test`

Jawaban: A

8. Aktivasi *neural network* biner di output:

A. *softmax*
B. *relu*
C. *sigmoid*
D. *tanh*

Jawaban: C

9. Untuk teks → angka yang sederhana:

A. *PCA*
B. *TF-IDF*
C. *K-Means*
D. *DBSCAN*

Jawaban: B

10. Kode yang benar untuk melatih model linear:

A.

```
model = LinearRegression()
model.fit(X_train, y_train)
```

B.

```
model = LinearRegression()
model.train(X, y)
```

C.

```
model = LinearRegression()
model.learn(X_train)
```

D.

```
model = LinearRegression()
model.predict(X_train, y_train)
```

Jawaban: A

C. Isian Singkat (5 Soal)

1. Isi nama dua jenis tugas utama di *supervised learning*: _____ dan _____.
Jawaban: Regresi dan Klasifikasi

2. Lengkapi:

```
from sklearn.metrics import _____
err = _____(y_test, y_pred)
```

(metrik untuk regresi, salah satu)

Jawaban: mean_absolute_error ; mean_absolute_error

3. Istilah untuk “data dengan jawaban” adalah _____.
Jawaban: Data berlabel

4. Lengkapi:

```
from sklearn.model_selection import _____
X_train, X_test, y_train, y_test = _____(X, y, test_size=0.2,
random_state=42)
```

Jawaban: train_test_split ; train_test_split

5. Aktivasi output untuk klasifikasi biner di *Keras* adalah _____.
Jawaban: sigmoid

D. Soal Explorasi (3 Soal)

Soal 1:

Rancang model untuk memprediksi keterlambatan tugas siswa (terlambat/tidak) dari fitur `jam_belajar`, `jam_tidur`, `jumlah_tugas`. Buat alur kerja dan contoh kode singkat.

Jawaban (contoh):

Pseudocode:

```
kumpulkan_data()
X = [jam_belajar, jam_tidur, jumlah_tugas]
y = [terlambat/tidak]
split stratified
model = LogisticRegression
fit → evaluasi F1
prediksi sampel baru
```

Python:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

df = pd.DataFrame({
    "jam_belajar": [1, 2, 3, 4, 5, 2, 3, 4],
    "jam_tidur": [6, 7, 8, 5, 6, 7, 6, 8],
    "jumlah_tugas": [3, 2, 4, 5, 1, 4, 3, 2],

    "terlambat": ["ya", "tidak", "ya", "ya", "tidak", "ya", "tidak", "tidak"]
})
X = df[["jam_belajar", "jam_tidur", "jumlah_tugas"]]
y = df["terlambat"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=42, stratify=y)
m = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
pred = m.predict(Xte)
print("F1:", f1_score(yte, pred, average="macro"))
```

Output

```
F1: 0.3333333333333333
```

Soal 2:

Buat *pipeline* sederhana untuk memfilter komentar negatif dengan *TF-IDF* + *Linear SVM*. Tulis urutan langkah dan potongan kode.

Jawaban (contoh):

Pseudocode:

```
dataset (teks, label)
preprocess (lowercase)
vectorize = TF-IDF
classifier = LinearSVC
```

train/test split (stratify)
 latih → uji → laporkan accuracy/F1

Python:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report

df = pd.DataFrame({
    "teks": ["bagus", "mantap sekali", "buruk", "jelek parah", "ok", "kurang enak"],
    "label": ["pos", "pos", "neg", "neg", "pos", "neg"]
})
Xtr, Xte, ytr, yte = train_test_split(df["teks"], df["label"],
test_size=0.33, random_state=0, stratify=df["label"])
tfidf = TfidfVectorizer()
Xtrv, Xtev = tfidf.fit_transform(Xtr), tfidf.transform(Xte)
clf = LinearSVC().fit(Xtrv, ytr)
print(classification_report(yte, clf.predict(Xtev)))
```

Output:

	precision	recall	f1-score	support
neg	0.50	1.00	0.67	1
pos	0.00	0.00	0.00	1
accuracy			0.50	2
macro avg	0.25	0.50	0.33	2
weighted avg	0.25	0.50	0.33	2

Soal 3:

Desain model *regresi* untuk memperkirakan jumlah penjualan minuman es dari suhu_c, hari(1-7), diskon(0/1). Sertakan metrik dan *baseline*.

Jawaban (contoh):

Pseudocode:

```
X = [suhu_c, hari, diskon] ; y = penjualan
baseline = rata-rata y_train → hitung MAE_baseline
model = LinearRegression
fit → y_pred → MAE_model
bandingkan MAE_model vs MAE_baseline
```

Python:

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
```

```

df = pd.DataFrame({
    "suhu_c": [28, 29, 30, 31, 32, 27, 26, 33, 34, 30],
    "hari": [1, 2, 3, 4, 5, 6, 7, 1, 2, 3],
    "diskon": [0, 1, 0, 1, 0, 0, 1, 1, 0, 1],
    "jual": [40, 55, 60, 70, 75, 35, 30, 80, 85, 62]
})
X, y = df[["suhu_c", "hari", "diskon"]], df["jual"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
random_state=42)
baseline = np.full_like(yte, ytr.mean(), dtype=float)
mae_base = mean_absolute_error(yte, baseline)
m = LinearRegression().fit(Xtr, ytr)
mae_model = mean_absolute_error(yte, m.predict(Xte))
print("MAE_baseline:", mae_base, "MAE_model:", mae_model)

```

Output:

```
MAE_baseline: 18.19047619047619 MAE_model: 3.391252156951007
```

Tugas Proyek

PROYEK 1: Regresi: Prediksi Pemakaian Listrik Harian (kWh)

Ide: Tebak kWh harian dari suhu, jam_pakai_AC, jumlah_penghuni.

Jenis: *Regression* (output angka).

Data (contoh kolom): suhu_c, jam_ac, penghuni, kwh.

Langkah & Kunci Jawaban (alur benar):

- Kumpulkan 20–50 baris data sederhana.
- Pisah data: *train-test split*.
- Latih **LinearRegression**.
- Cek **MAE**.
- Prediksi **kwh** untuk input baru.

Pseudocode

```

muat_data()
X = [suhu_c, jam_ac, penghuni]
y = kwh
split train/test
model = LinearRegression()
fit(model, X_train, y_train)
pred = predict(model, X_test)
lapor_MAE(pred, y_test)
pred_baru = predict(model, [suhu=30, jam_ac=6, penghuni=4])

```


Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

df = pd.DataFrame({
    "suhu_c": [28, 29, 30, 31, 27, 26, 32, 33, 30, 29],
    "jam_ac": [4, 5, 6, 7, 3, 2, 8, 9, 6, 5],
    "penghuni": [3, 3, 4, 5, 2, 2, 5, 5, 4, 3],
    "kwh": [7.1, 7.8, 8.9, 10.2, 6.2, 5.5, 11.0, 12.3, 9.0, 8.0]
})
X, y = df[["suhu_c", "jam_ac", "penghuni"]], df["kwh"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
                                      random_state=42)
m = LinearRegression().fit(Xtr, ytr)
print("MAE:", mean_absolute_error(yte, m.predict(Xte)))
print("Prediksi kWh:", m.predict(pd.DataFrame([[30, 6, 4]],
                                                columns=X.columns)) [0])
```

Output

```
MAE: 0.24811643835616382
Prediksi kWh: 9.138698630136984
```

PROYEK 2: Klasifikasi Multi-Kelas: Kualitas Wi-Fi (Baik/Sedang/Buruk)

Ide: Klasifikasi **kualitas_wifi** dari **jarak_router**, **penghalang** (0/1), **user_aktif**.

Jenis: *Classification* multi-kelas.

Data (kolom): **jarak_m**, **penghalang**, **user_aktif**, **kualitas** ∈ {**buruk**, **sedang**, **baik**}.

Langkah & Kunci Jawaban (alur benar):

- Siapkan 50+ baris data observasi kelas.
- *Stratified split*.
- Latih **RandomForestClassifier**.
- Nilai dengan **accuracy** dan **F1-macro**.
- Uji 1 sampel baru.

Pseudocode

```
X = [jarak_m, penghalang, user_aktif]
y = kualitas
split stratified
clf = RandomForestClassifier()
fit(clf, X_train, y_train)
pred = predict(clf, X_test)
```

```
lapor_accuracy_F1(y_test, pred)
pred_baru = predict(clf, [jarak=8, penghalang=1, user_aktif=15])
```

Python

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

np.random.seed(0)
N=80
df = pd.DataFrame({
    "jarak_m": np.random.randint(1,15,size=N),
    "penghalang": np.random.randint(0,2,size=N),
    "user_aktif": np.random.randint(1,25,size=N)
})
# label sederhana: makin jauh/banyak penghalang/user → kualitas turun
score = 100 - (df["jarak_m"]*4 + df["penghalang"]*10 + df["user_aktif"]*2)
df["kualitas"] = pd.cut(score, bins=[-1,50,75,100], labels=["buruk","sedang","baik"])

X, y = df[["jarak_m","penghalang","user_aktif"]], df["kualitas"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)
clf = RandomForestClassifier(random_state=42).fit(Xtr, ytr)
pred = clf.predict(Xte)
print("Acc:", accuracy_score(yte, pred), "F1-macro:", f1_score(yte, pred, average="macro"))
print("Pred contoh:", clf.predict(pd.DataFrame([[8,1,15]], columns=X.columns))[0])
```

Output

```
Acc: 0.8 F1-macro: 0.544973544973545
Pred contoh: buruk
```

PROYEK 3: Regresi: Prediksi Kebutuhan Tinta Printer (ml)

Ide: Hitung **ml_tinta** dari **halaman**, **mode** (0=draft,1=normal), **persen_gambar**.
Jenis: *Regression*.

Data (kolom): **halaman**, **mode**, **persen_gambar**, **ml_tinta**.

Langkah & Kunci Jawaban (alur benar):

- Buat 30–60 baris data percobaan.
- Latih **LinearRegression**.
- Cek **MAE/MAPE**.

- Prediksi kebutuhan tinta untuk job baru.

Pseudocode

```
X = [halaman, mode, persen_gambar]
y = ml_tinta
train_test_split()
model = LinearRegression()
fit()
MAE = hitung_error()
pred_job_baru = predict([halaman=20, mode=1, persen=30])
```

Python

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

np.random.seed(1)
N=50
df = pd.DataFrame({
    "halaman": np.random.randint(1,41,size=N),
    "mode": np.random.randint(0,2,size=N),
    "persen_gambar": np.random.randint(0,61,size=N)
})
# rumus fiktif tinta
df["ml_tinta"] = 0.08*df["halaman"] + 0.05*df["persen_gambar"] +
0.9*df["mode"] + np.random.normal(0,0.2,N)

X, y = df[["halaman","mode","persen_gambar"]], df["ml_tinta"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
random_state=42)
m = LinearRegression().fit(Xtr, ytr)
print("MAE:", mean_absolute_error(yte, m.predict(Xte)))
print("Pred tinta (ml):", m.predict(pd.DataFrame([[20,1,30]],
columns=X.columns))[0])
```

Output

```
MAE: 0.19274946500195042
Pred tinta (ml): 3.9856400961084475
```

PROYEK 4:Klasifikasi: Deteksi URL *Phising* Sederhana

Ide: Tandai URL **phishing** atau **aman** dari ciri mudah: **panjang_url, jumlah_angka, pakai_https(0/1), ada_at(0/1).**

Jenis: *Binary classification.*

Data (kolom): panjang, jml_angka, https, ada_at, label ∈ {phishing,aman}.

Langkah & Kunci Jawaban (alur benar):

- Buat 40–80 contoh URL (fiktif boleh).
- Ekstrak fitur sederhana dari string.
- Latih **LogisticRegression**.
- Lihat **precision/recall** kelas *phishing*.
- Uji 2–3 URL baru.

Pseudocode

```

fitur(url):
    panjang = len(url)
    jml_angka = hitung_digit(url)
    https = 1 jika url.startswith("https") else 0
    ada_at = 1 jika "@" di url else 0
    return [panjang, jml_angka, https, ada_at]

X, y = ekstrak_fitur(url_list)
split stratified
clf = LogisticRegression()
fit
evaluasi(precision, recall, f1 untuk kelas phishing)
prediksi_url_baru(...)

```

Python

```

import re, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

urls = [
    ("https://bankku.com/login", "aman"),
    ("http://bankku.com.verify-login.ru", "phishing"),
    ("http://192.168.0.5/wallet", "phishing"),
    ("https://toko.id/pay", "aman"),
    ("http://pay-safe-login@secure-update.cn", "phishing"),
    ("https://sekolah.id/info", "aman"),
]

def fitur(url):
    panjang = len(url)
    jml_angka = sum(c.isdigit() for c in url)
    https = 1 if url.startswith("https") else 0
    ada_at = 1 if "@" in url else 0
    return [panjang, jml_angka, https, ada_at]

X = [fitur(u) for u,l in urls]
y = [l for u,l in urls]
df = pd.DataFrame(X,
    columns=["panjang", "jml_angka", "https", "ada_at"]); df["label"]=y

Xtr, Xte, ytr, yte =
    train_test_split(df[["panjang", "jml_angka", "https", "ada_at"]],
        df["label"],

```

```

test_size=0.33,
random_state=42, stratify=df["label"])
clf = LogisticRegression().fit(Xtr, ytr)
pred = clf.predict(Xte)
print(classification_report(yte, pred))
print("Uji baru:",
      clf.predict(pd.DataFrame([[38,4,0,1]],
columns=Xtr.columns))[0]) # contoh URL
"http://login123@secure-xx"

```

Output

	precision	recall	f1-score	support
aman	1.00	1.00	1.00	1
phishing	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

PROYEK 5: Klasifikasi: Paket Tepat Waktu atau Terlambat

Ide: Prediksi **ontime/late** dari **jarak_km**, **hujan(0/1)**, **jam_kirim (0–23)**, **volume_paket_liter**.

Jenis: *Binary classification*.

Data (kolom): **jarak_km**, **hujan**, **jam_kirim**, **volume_l**, **status** ∈ {**ontime**, **late**}.

Langkah & Kunci Jawaban (alur benar):

- Bentuk 60–120 baris data fiktif realistik.
- *Stratified split*.
- Latih **GradientBoostingClassifier** atau **LogisticRegression**.
- Cek **F1** untuk kelas *late*.
- Coba threshold 0.6 untuk kurangi *false negative late*.

Pseudocode

```

X = [jarak_km, hujan, jam_kirim, volume_l]
y = status(ontime/late)
split stratified
clf = GradientBoostingClassifier()
fit
prob = predict_proba(X_test)[:,-1] # probabilitas 'late'
pred = (prob >= 0.5) ? 'late' : 'ontime'
lapor F1 khusus kelas 'late'

```

Python

```

import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import f1_score, classification_report

np.random.seed(7)
N=100
df = pd.DataFrame({
    "jarak_km": np.random.uniform(1,30,size=N),
    "hujan": np.random.randint(0,2,size=N),
    "jam_kirim": np.random.randint(6,21,size=N),
    "volume_l": np.random.uniform(0.5,20,size=N),
})
# aturan fiktif risiko telat
risk = 0.03*df["jarak_km"] + 0.4*df["hujan"] +
0.02*np.abs(df["jam_kirim"]-17) + 0.01*df["volume_l"]
df["status"] = np.where(risk>0.9, "late", "ontime")

X, y = df[["jarak_km","hujan","jam_kirim","volume_l"]],
df["status"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
random_state=42, stratify=y)
clf = GradientBoostingClassifier(random_state=42).fit(Xtr, ytr)
proba = clf.predict_proba(Xte)[:],
list(clf.classes_).index("late")]
pred = np.where(proba>=0.5, "late", "ontime")
print(classification_report(yte, pred, digits=3))

```

Output

	precision	recall	f1-score	support
late	0.818	0.750	0.783	12
ontime	0.786	0.846	0.815	13
accuracy			0.800	25
macro avg	0.802	0.798	0.799	25
weighted avg	0.801	0.800	0.799	25

Tips

- **Tanya dulu output-nya:** angka → *regresi*, kategori → *klasifikasi*.
- **Bersihkan data:** tidak ada kosong, tipe benar, skala wajar.
- **Evaluasi pas:** *MAE/MAPE* untuk *regresi*; *precision/recall/F1* untuk *klasifikasi*.
- Mulai sederhana (**LinearRegression/LogReg**), lalu naikan level (**RandomForest/GradientBoosting**, bahkan *Keras* untuk *neural network*).

BAB 2: Unsupervised Learning (Clustering, PCA)

Selamat datang kembali. Setelah di bab sebelumnya kita jadi guru yang sabar ngajarin model, sekarang kita naik level! Kita bakal jadi detektif yang cerdas, mencari pola tersembunyi di dalam data tanpa ada petunjuk awal sama sekali. Inilah yang kita sebut **Unsupervised Learning**!

Tujuan Pembelajaran

- Memahami konsep **Unsupervised Learning** dan perbedaannya dengan **Supervised Learning**.
- Mengenal dua teknik utamanya: **Clustering** dan **Principal Component Analysis (PCA)**.
- Menggunakan **Orange Data Mining** untuk menerapkan kedua teknik ini.
- Menginterpretasi hasil pengelompokan dan visualisasi data.

Peta Konsep Sederhana

Bayangin Unsupervised Learning itu seperti merapikan koleksi kartu game.

- **Kartu Berantakan (Data Mentah)**: Kalian punya tumpukan kartu yang isinya campur aduk. Tidak ada label jenisnya.
- **Pengelompokan (Clustering)**: Kalian mulai mengelompokkan kartu-kartu yang punya karakter mirip. Misalnya, semua kartu yang punya simbol api jadi satu tumpukan, kartu simbol air jadi tumpukan lain.
- **Meringkas (PCA)**: Kalian melihat, "Ah, kartu-kartu ini hanya punya dua karakteristik penting: kekuatan serangan dan kecepatan." Kalian buang karakteristik lain yang tidak penting.

Itulah cara kerja Unsupervised Learning. Ia mencari pola tanpa diawasi.

Apersepsi: Kamu Mau Bikin Kelompok Studi Dadakan!

Guru kalian tiba-tiba minta kalian bikin kelompok studi untuk proyek. Tapi, guru kalian tidak memberi tahu siapa yang harus sekelompok dengan siapa. Kalian cuma punya data: **nilai_matematika**, **nilai_bahasa**, dan **nilai_produkatif** teman-teman kalian.

- **Tantangan**: Bagaimana cara mengelompokkan teman-temanmu agar satu kelompok punya kemampuan yang kurang lebih seimbang?
- **Solusi**: Kalian bisa pakai **Clustering**! Komputer akan mencari tahu sendiri, siapa saja yang punya nilai mirip, dan mengelompokkannya jadi 3-4 grup.

Dengan cara ini, kalian bisa bikin kelompok yang seimbang dan adil!

Penjelasan Teori

Kalau di bab sebelumnya kalian jadi *guru* yang ngasih arahan ke model (*Supervised Learning*), sekarang kalian naik level!

Kalian bakal jadi **detektif data**.

- Bedanya, kali ini **tidak ada jawaban siap pakai**.
- Tidak ada label, tidak ada petunjuk.
- Tugas kalian adalah **mencari pola tersembunyi**.

Itulah yang kita sebut *Unsupervised Learning*.

Apa Itu *Unsupervised Learning*?

Bayangin kalian punya tumpukan kartu game campur aduk.

- Tidak ada tulisan jenis atau kelompok.
- Tugas kalian: rapikan sendiri berdasarkan kemiripan.

Inilah inti *Unsupervised Learning*:

- Komputer belajar dari data **tanpa label**.
- Tujuan utama: menemukan **struktur, pola, atau hubungan** yang tersembunyi.
- Hasil akhirnya? Data jadi lebih mudah dipahami dan dipakai.

Dua Teknik Utama

1. Clustering (Pengelompokan)

- *Tujuan*: Membagi data menjadi kelompok berdasarkan kemiripan.
- *Analogi*: Pisahin sendok, garpu, piring di laci dapur.
- *Contoh nyata*:
 - Mengelompokkan pelanggan toko online berdasarkan kebiasaan belanja.
 - Membuat kelompok belajar berdasarkan nilai matematika, bahasa, dan produktif.

2. Principal Component Analysis (PCA)

- *Tujuan*: Meringkas data yang punya banyak kolom (variabel).
- *Analogi*: Membuat ringkasan dari buku tebal jadi hanya beberapa halaman inti.
- *Contoh nyata*:
 - Survei sekolah punya 100 pertanyaan → PCA bisa meringkas jadi 5 faktor utama.
 - Dataset mobil punya 50 fitur → PCA meringkas jadi 3 fitur inti: performa, efisiensi, tampilan.

Contoh Kasus Seru

- **Clustering Pemain Game**
 - Data: jam_main, level, jumlah_item.
 - Hasil: kelompok pemain kasual, standar, dan pro.
- **PCA Survei Sekolah**
 - Data: 20 pertanyaan kepuasan siswa.
 - Hasil: cukup 2 variabel → kepuasan_siswa & motivasi_belajar.
- **Clustering Jajanan Kantin**
 - Data: rasa_pedas, rasa_manis, harga.
 - Hasil: muncul kelompok seperti *jajanan_murah_dan_manis* atau *jajanan_mahal_dan_pedas*.

tool Open Source yang Bisa Dipakai

- **Python** → bahasa gratis, populer, gampang dipelajari.
- **Pandas** → pustaka Python untuk mengolah data tabel.
- **Keras + Neural Network** → buat eksperimen lebih lanjut.
- **Orange Data Mining** → aplikasi *drag and drop* yang bikin clustering dan PCA super gampang.

Kenapa Penting?

- **Clustering** bikin kita tahu siapa mirip dengan siapa → cocok buat pengelompokan.
- **PCA** bikin data yang rumit jadi lebih sederhana → gampang divisualisasikan.
- Semua ini bisa membantu kalian **mengambil keputusan lebih cepat**.

Rangkuman

- *Unsupervised Learning* = belajar dari data tanpa label.
- Dua teknik utama:
 - **Clustering** → membagi data jadi kelompok.
 - **PCA** → meringkas banyak variabel jadi lebih sedikit.
- Semua bisa dipraktekkan dengan **tool open source**: Python, Pandas, Keras, Orange.

Kasus Ilustrasi

Kasus 1: Kelompok Belajar Seimbang (Clustering)

Cerita singkat

Kamu ingin bikin kelompok belajar. Data yang ada: *nilai_matematika*, *nilai_bahasa*, *nilai_produkatif*.

Tujuan: kelompok seimbang. Tidak ada yang terlalu kuat/terlalu lemah sendirian.

Data contoh

Tabel siswa dengan 3 kolom nilai (0–100).

tool open source

- *Orange Data Mining* → *K-Means + Scatter Plot*.
- *Python + Pandas* → kalau mau olah CSV dulu.

Pseudocode

```
INPUT: tabel_siswa[nilai_matematika, nilai_bahasa,
nilai_produkatif]
CLEAN: hapus baris kosong, normalisasi nilai ke 0..1
SET k = 3 // jumlah kelompok
CLUSTERS = KMeans.fit_predict(tabel_siswa, k)
OUTPUT: label_kelompok untuk tiap siswa
EVALUATE: cek rata2 nilai tiap kelompok, pastikan seimbang
```

Ilustrasi hasil (bayangan visual)

Titik-titik warna berbeda di *Scatter Plot* (3 warna = 3 kelompok).

Kelompok 1 = kuat matematika, Kelompok 2 = kuat bahasa, Kelompok 3 = seimbang.

Tips

- Kalau kelompok “nabrak”, coba $k = 4$.
- Standarkan nilai dulu. Itu penting.

Kasus 2: Peta Jajanan Kantin (Clustering)

Cerita singkat

Kamu ingin mengelompokkan jajanan di kantin. Fitur: *rasa_pedas*, *rasa_manis*, *harga*.

Tujuan: tahu kelompok “murah-manis”, “mahal-pedas”, dll.

Data contoh

CSV jajanan: nama, pedas(0–10), manis(0–10), harga (rupiah).

tool open source

- *Orange* → *K-Means + Silhouette Score*.
- *Python + Pandas* → bersihkan data, simpan ke CSV.

Pseudocode

```
INPUT: tabel_jajanan[pedas, manis, harga]
SCALE: fitur ke skala yang mirip (misal z-score)
FOR k in [2..6]:
    labels_k = KMeans.fit_predict(tabel_jajanan, k)
    score_k = SilhouetteScore(tabel_jajanan, labels_k)
PILIH k dengan score_k tertinggi
```

```
OUTPUT: kelompok_jajanan (label per item)
NAMA_KAN: beri nama kelompok berdasarkan ciri (contoh:
"murah-manis")
```

Ilustrasi hasil

Diagram sebar 2D/3D dengan warna kelompok. Tabel ringkas:

- Kelompok A: murah + manis
- Kelompok B: sedang + pedas
- Kelompok C: mahal + pedas

Tips

- *Silhouette score* membantu pilih k terbaik.
- Harga biasanya perlu diskalakan.

Kasus 3: Profil Pemain *Mobile Legends* (Clustering)

Cerita singkat

Kamu punya data: *jumlah_match*, *win_rate*, *kda_rata_rata*.

Tujuan: segmentasi pemain → *kasual*, *standar*, *pro*.

Data contoh

CSV 1000 pemain.

tool open source

- *Orange* → *K-Means* + *Box Plot* per fitur.
- *Python* → cek *outlier* (misal KDA terlalu ekstrem).

Pseudocode

```
INPUT: tabel_pemain[jumlah_match, win_rate, kda]
FILTER: buang outlier ekstrem (aturan IQR)
SCALE: ke 0..1
SET k = 3
labels = KMeans.fit_predict(tabel_pemain, k)
PROFILE:
    hitung rata2 tiap fitur per label
    beri nama kelompok: casual/standar/pro
OUTPUT: label per pemain + ringkasan profil
```

Ilustrasi hasil

Tiga cluster jelas:

- *Kasual*: match sedikit, *win_rate* rendah.
- *Standar*: rata.
- *Pro*: match banyak, *win_rate* dan KDA tinggi.

Tips

- Coba *Hierarchical Clustering* untuk bandingkan bentuk cluster.
- Visualisasi *Box Plot* per cluster bantu memahami karakter.

Kasus 4: Ringkas Survei Kebersihan Sekolah (PCA)

Cerita singkat

Ada 10 pertanyaan (skala 1–5). Terlalu banyak untuk dilihat satu-satu.

Tujuan: diringkas jadi 2 faktor: *kebersihan_internal* & *kebersihan_eksternal*.

Data contoh

CSV survei: Q1..Q10 untuk 300 siswa.

tool open source

- *Orange* → *PCA + Scree Plot + Biplot*.
- *Python + Pandas* → *StandardScaler* sebelum PCA.

Pseudocode

```
INPUT: tabel_survei[Q1..Q10]
SCALE: standardize semua kolom
PCA.fit(tabel_survei)
LIHAT: scree_plot untuk pilih komponen (target 2)
COMP = PCA.transform(tabel_survei, n_components=2)
NAMA_KOMPONEN:
    lihat loading matrix:
        jika Q1..Q5 tinggi -> "internal"
        jika Q6..Q10 tinggi -> "eksternal"
OUTPUT: dua kolom baru [internal, eksternal]
```

Ilustrasi hasil

Scree plot turun tajam di komponen 1–2. *Biplot* menunjukkan mana pertanyaan yang “menarik” komponen internal/eksternal.

Tips

- PCA butuh data terskalakan.
- *Loading* besar → pertanyaan itu “mewakili” komponen.

Kasus 5: Ringkas Fitur Produk jadi *Skor Overall* (PCA + Clustering mini)

Cerita singkat

Toko elektronik punya rating: *desain, harga, kualitas, garansi, purna_jual*.

Tujuan: bikin satu *skor_overall* yang mewakili semuanya, lalu kelompokkan produk.

Data contoh

CSV 200 produk.

tool open source

- *Orange* → *PCA* (ambil komponen 1) + *K-Means* di hasil *PCA*.
- *Python + Pandas* → kelola data hilang.

Pseudocode

```

INPUT: tabel_produk[desain, harga, kualitas, garansi, purna_jual]
CLEAN: isi nilai kosong (impute median)
SCALE: standardize
PCA.fit(tabel_produk)
COMP1 = komponen_pertama(PCA, tabel_produk)
RENAME COMP1 -> skor_overall
CLUSTERS = KMeans.fit_predict([skor_overall], k=3)
OUTPUT:
    skor_overall per produk
    label_cluster: "ekonomis", "seimbang", "premium"

```

Ilustrasi hasil

Bar chart *skor_overall* per produk. Warna berdasarkan cluster.

- *Ekonomis*: skor sedang, harga bagus.
- *Premium*: skor tinggi, fitur kuat.

Tips

- Komponen 1 sering mewakili “arah utama” data.
- Kombinasi **PCA** → **Clustering** sering dipakai di industri.

Catatan Praktik Cepat di Orange (drag-and-drop)

- **Clustering**: *File* → *Select Columns* (opsional) → *Normalize* → *K-Means* → *Silhouette* → *Scatter Plot*.
- **PCA**: *File* → *Select Columns* → *Normalize* → *PCA* → *Screen Plot* → *Biplot/Scatter Plot*.
- Simpan hasil ke CSV dengan *Save* widget.
- Semua alur ini *open source* dan ringan untuk laptop sekolah.

Soal Latihan & Pembahasan

Soal 1: Kelompok Belajar Seimbang (Clustering)

Cerita

Kamu punya data siswa: *nilai_matematika*, *nilai_bahasa*, *nilai_produkatif*. Tidak ada *label*. Kamu ingin membagi jadi **3** kelompok yang seimbang.

Pertanyaan

Teknik apa yang dipakai? Langkah utamanya bagaimana?

Jawaban singkat

- Teknik: **Clustering (K-Means)**.
- Alur: bersihkan data → skala fitur → *K-Means (k=3)* → cek rata-rata tiap kelompok.

Pseudocode

```

INPUT: tabel_siswa[n_mtk, n_bhs, n_prod]
CLEAN: hapus NULL
SCALE: standardize ke 0..1
SET k = 3
LABEL = KMeans.fit_predict(tabel_siswa, k)
EVALUATE:
    hitung mean per fitur di tiap LABEL
    jika ada kelompok terlalu kecil/ekstrem -> coba k=4 atau ulang
    init
OUTPUT: LABEL per siswa + ringkasan profil kelompok

```

Pembahasan

- Ini *Unsupervised* karena tidak ada *label*.
- *K-Means* mencari pusat kelompok (centroid).
- **Skalakan** nilai agar adil (fitur setara).
- Lihat rata-rata per cluster untuk menilai “seimbang” atau tidak.
- Bisa dikerjakan cepat di *Orange: File → Normalize → K-Means → Scatter Plot*.

Soal 2: Jajanan Kantin: Pilih k Terbaik (Clustering + Silhouette)

Cerita

Dataset jajanan: *rasa_pedas (0–10)*, *rasa_manis (0–10)*, *harga (Rp)*. Kamu coba *K-Means* dengan beberapa *k* dan dapat skor *silhouette* seperti ini:

- $k=2 \rightarrow 0.55$
- $k=3 \rightarrow 0.62$
- $k=4 \rightarrow 0.48$

Pertanyaan

Berapa *k* yang dipilih? Mengapa?

Jawaban singkat

- Pilih **$k=3$** karena skor *silhouette* tertinggi (0.62).

Pseudocode

```

INPUT: tabel_jajanan[pedas, manis, harga]
SCALE: z-score (harga beda skala)
BEST = (k=None, score=-inf)
FOR k in [2..6]:
    label_k = KMeans.fit_predict(tabel_jajanan, k)

```

```

s_k = SilhouetteScore(tabel_jajanan, label_k)
if s_k > BEST.score:
    BEST = (k, s_k)
OUTPUT: BEST.k // pilih k dengan score tertinggi

```

Pembahasan

- *Silhouette* mengukur “rapat di dalam cluster” dan “jauh antar cluster”.
- Skor tertinggi biasanya paling stabil.
- **Skala harga** penting agar tidak “menguasai” jarak.
- Di *Orange*: tambahkan *Silhouette Plot* untuk bandingkan k.

Soal 3: PCA Survei Kebersihan: Menamai Komponen

Cerita

Ada 10 pertanyaan (Q1–Q10). Setelah *PCA*, kamu lihat *loading* besar untuk:

- Komponen 1: Q1–Q5 tinggi (soal kebersihan di dalam kelas).
- Komponen 2: Q6–Q10 tinggi (soal kebersihan lingkungan sekolah).

Pertanyaan

Nama yang tepat untuk kedua komponen? Mengapa?

Jawaban singkat

- Komponen 1: **kebersihan_internal**.
- Komponen 2: **kebersihan_eksternal**.
- Karena *loading* pertanyaan-pertanyaan itu dominan pada komponen terkait.

Pseudocode

```

INPUT: survei[Q1..Q10]
SCALE: standardize (mean=0, std=1)
PCA.fit(survei)
LOAD = PCA.loadings() // matriks Q x komponen
IF Q1..Q5 dominan di comp1:
    nama_comp1 = "kebersihan_internal"
IF Q6..Q10 dominan di comp2:
    nama_comp2 = "kebersihan_eksternal"
OUTPUT: [comp1=internal, comp2=eksternal]

```

Pembahasan

- *PCA* meringkas banyak variabel jadi beberapa *principal components*.
- *Loading* besar = pertanyaan itu “menarik” komponen.
- Penamaan komponen berdasarkan **makna** pertanyaan yang dominan.
- Di *Orange*: lihat *Scree Plot* (untuk jumlah komponen) dan *Biplot* (untuk arah loading).

Soal 4: Outlier Mengacaukan Cluster

Cerita

Dataset pemain *Mobile Legends*: *jumlah_match*, *win_rate*, *kda*. Ada 1 pemain dengan *kda* = 50 (sangat ekstrem). Hasil cluster jadi aneh.

Pertanyaan

Apa yang harus kamu lakukan sebelum *clustering*? Mengapa?

Jawaban singkat

- **Tangani outlier** (hapus/batasi/robust scaling).
- **Skalakan fitur** (standardize/robust).
- Baru jalankan *K-Means*.

Pseudocode

```
INPUT: tabel_pemain[match, win_rate, kda]
DETECT_OUTLIER:
    gunakan IQR pada kda
    if kda di luar [Q1-1.5*IQR, Q3+1.5*IQR]:
        tandai sebagai outlier
HANDLE:
    opsi A: drop outlier
    opsi B: cap kda ke P99 (winsorize)
SCALE: RobustScaler (tahan outlier)
CLUSTER: KMeans.fit_predict(...)
OUTPUT: label_cluster yang lebih stabil
```

Pembahasan

- *K-Means* sensitif terhadap nilai ekstrim karena pakai jarak Euclidean.
- Outlier bisa “menarik” centroid, bikin cluster jelek.
- *RobustScaler* dan *winsorize* membantu.
- Bandingkan hasil sebelum vs sesudah *cleaning*.

Soal 5: Pipeline PCA → K-Means (2D Plot yang Jelas)

Cerita

Dataset produk punya 8 fitur rating (skala 1–5). Kamu ingin tampilkan plot 2D yang rapi dan tetap melakukan pengelompokan.

Pertanyaan

Urutan langkah yang baik? Mengapa?

Jawaban singkat

- **Standardize → PCA (ambil 2 komponen) → K-Means di ruang PCA → Plot 2D.**

- Karena PCA membuat data lebih ringkas dan sering memisahkan pola utama. Visual jadi jelas.

Pseudocode

```
INPUT: produk[F1..F8]
CLEAN: isi NULL (median)
SCALE: StandardScaler
PC = PCA.fit_transform(produk, n_components=2)
LABEL = KMeans.fit_predict(PC, k=3)
PLOT: scatter(PC[:,0], PC[:,1], color=LABEL)
OUTPUT: peta 2D + cluster label
```

Pembahasan

- *PCA* memadatkan informasi ke 2 sumbu utama (*principal components*).
- *K-Means* di ruang PCA sering lebih stabil karena noise berkurang.
- Plot 2D memudahkan siswa membaca pola.
- Di *Orange*: *Normalize* → *PCA* → *K-Means* → *Scatter Plot*.

Ringkasan

- **Clustering** = pengelompokan data mirip.
- **PCA** = meringkas banyak kolom jadi sedikit komponen.
- **Skala & bersihkan data** sebelum *clustering/PCA*.
- Semua bisa dijalankan dengan tool **open source**: *Python, Pandas, scikit-learn, Orange*.
- Untuk lanjut, coba *autoencoder* di *Keras* (*unsupervised feature learning*) — ini materi lanjutan yang seru.

Fun Facts!

- **PCA** sering dipakai untuk mengenali wajah di foto! Ia akan mencari pola-pola utama dari sebuah wajah, seperti jarak mata, lebar hidung, dan lainnya.
- **Ollama** dan **OpenWebUI** bisa kamu pakai buat simulasi. Coba tanya, " Gambarkan flowchart sederhana untuk clustering di Orange Data Mining."
- Google Photos menggunakan **Clustering** untuk mengelompokkan semua fotomu yang isinya orang yang sama. Ia melakukannya secara otomatis tanpa perlu kamu labeli!

Tips Belajar/Menghafal Cepat

- **Tugasnya, Bukan Namanya**: Jangan pusing dengan namanya. Pikirkan, apa tujuan dari teknik ini? Mengelompokkan atau meringkas?
- **Praktek di Orange**: Paling gampang belajar Unsupervised Learning itu langsung di Orange DM. Tinggal tarik **K-Means**, lalu hubungkan ke **Scatter Plot**, dan kamu akan langsung lihat hasilnya!

- **Cari Kasus Nyata:** Pikirkan, di kehidupan sehari-hari, mana yang menggunakan clustering? Contoh: rekomendasi film di Netflix, atau iklan yang muncul di media sosialmu.

Aktivitas Siswa

Aktivitas 1: Kelompok Belajar Seimbang (Clustering)

Tujuan

Membagi siswa menjadi 3 kelompok seimbang dari tiga nilai: n_mtk , n_bhs , n_prod .

Langkah

1. Siapkan CSV `nilai_siswa.csv` (kolom: n_mtk , n_bhs , n_prod).
2. *Normalize* nilai agar adil.
3. Jalankan *K-Means* dengan $k=3$.
4. Lihat rata-rata per kelompok. Beri nama: “Fokus Matematika / Bahasa / Seimbang”.

Pseudocode

```
INPUT: tabel[n_mtk, n_bhs, n_prod]
CLEAN: hapus baris kosong
SCALE: standardize ke 0..1
k = 3
labels = KMeans.fit_predict(SCALE(tabel), k)
summary = mean_per_cluster(tabel, labels)
OUTPUT: labels + summary (rata-rata tiap nilai)
```

Python

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv('nilai_siswa.csv') # n_mtk, n_bhs, n_prod
X = df[['n_mtk', 'n_bhs', 'n_prod']].dropna()
Xs = StandardScaler().fit_transform(X)

km = KMeans(n_clusters=3, n_init=10, random_state=42)
labels = km.fit_predict(Xs)

df = df.loc[X.index] # selaraskan index
df['cluster'] = labels
print(df.groupby('cluster')[['n_mtk', 'n_bhs', 'n_prod']].mean().round(2))
```

Hasil yang diharapkan

Tabel rata-rata per *cluster*. Kelompok terlihat beda fokusnya. Visual nanti bisa dibuat di *Orange* dengan *scatter plot*.

Aktivitas 2: Peta Jajanan Kantin & Pilih k Terbaik (Clustering + Silhouette)

Tujuan

Mengelompokkan jajanan berdasarkan *pedas*, *manis*, *harga* dan memilih jumlah cluster terbaik dengan *silhouette score*.

Langkah

1. Siapkan CSV *jajanan.csv* (*pedas*, *manis*, *harga*).
2. *Standardize* fitur (*harga* beda skala).
3. Coba *k* dari 2 s.d. 6. Ambil *silhouette* tertinggi.
4. Beri nama cluster: “murah-manis”, “mahal-pedas”, dll.

Pseudocode

```
INPUT: jajanan[pedas, manis, harga]
SCALE: z-score semua kolom
BEST = (k=None, score=-inf)
FOR k in 2..6:
    lbl = KMeans.fit_predict(SCALE(jajanan), k)
    s = silhouette_score(SCALE(jajanan), lbl)
    if s > BEST.score: BEST = (k, s, lbl)
OUTPUT: BEST.k, BEST.score, BEST.lbl
```

Python

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

dj =
pd.read_csv('jajanan.csv')[['pedas','manis','harga']].dropna()
Xs = StandardScaler().fit_transform(dj)

best = {'k': None, 'score': -1, 'labels': None}
for k in range(2, 7):
    lbl = KMeans(n_clusters=k, n_init=10,
random_state=42).fit_predict(Xs)
    s = silhouette_score(Xs, lbl)
    if s > best['score']:
        best = {'k': k, 'score': s, 'labels': lbl}

print("k terbaik:", best['k'], "silhouette:",
round(best['score'], 3))
```

Hasil yang diharapkan

Terpilih **k** terbaik. Siswa bisa kasih nama tiap *cluster* sesuai ciri (misal rata-rata harga/tingkat pedas-manis).

Aktivitas 3: Ringkas Survei Kebersihan (PCA 10 → 2)

Tujuan

Meringkas 10 pertanyaan survei kebersihan jadi 2 komponen utama. Menamai komponen berdasarkan *loading*.

Langkah

1. Siapkan CSV **survei_kebersihan.csv** (kolom **Q1..Q10**, skala 1–5).
2. *Standardize* semua kolom.
3. Jalankan *PCA* ambil 2 komponen.
4. Lihat *loading*. Namai misal: *kebersihan_internal* (Q1..Q5), *kebersihan_eksternal* (Q6..Q10).

Pseudocode

```
INPUT: survei[Q1..Q10]
SCALE: standardize
PC = PCA.fit_transform(survei, n_components=2)
LOAD = PCA.components_ // matriks 2 x 10
NAMA:
    jika loading Q1..Q5 besar di PC1 -> "kebersihan_internal"
    jika loading Q6..Q10 besar di PC2 -> "kebersihan_eksternal"
OUTPUT: skor_PC1, skor_PC2 + nama komponen
```

Python

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

sv = pd.read_csv('survei_kebersihan.csv')
X = sv.filter(like='Q').dropna()
Xs = StandardScaler().fit_transform(X)

pca = PCA(n_components=2, random_state=42)
PC = pca.fit_transform(Xs)
loadings = pd.DataFrame(pca.components_, columns=X.columns,
index=['PC1', 'PC2']).T

print("Explained variance ratio:",
pca.explained_variance_ratio_.round(3))
print("Top loading PC1:\n",
loadings['PC1'].sort_values(ascending=False).head())
print("Top loading PC2:\n",
loadings['PC2'].sort_values(ascending=False).head())
```

Hasil yang diharapkan

Rasio varian $\geq$ cukup (misal $\geq 70-80\%$ jika bisa). Terlihat pertanyaan mana yang dominan di PC1/PC2. Komponen diberi nama sesuai makna.

Aktivitas 4: *Pipeline PCA* $\rightarrow$ *K-Means* (Plot 2D yang Jelas)

Tujuan

Membuat peta 2D dari 8 *feature* produk, lalu *clustering* di ruang PCA agar *scatter plot* mudah dibaca.

Langkah

1. Isi nilai kosong dengan median.
2. *Standardize* $\rightarrow$ *PCA*(2) $\rightarrow$ *K-Means*($k=3$).
3. Plot PC1 vs PC2 (warna = *cluster*).

Pseudocode

```
INPUT: produk[F1..F8]
IMPUTE: median
SCALE  $\rightarrow$  PCA( $n=2$ )  $\rightarrow$  KMeans( $k=3$ )
PLOT: scatter(PC1, PC2, color=cluster)
OUTPUT: peta 2D berwarna per cluster
```

Python

```
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

pr = pd.read_csv('produk.csv').filter(regex='^F') # kolom F1..F8
X = SimpleImputer(strategy='median').fit_transform(pr)
Xs = StandardScaler().fit_transform(X)

PC = PCA(n_components=2, random_state=42).fit_transform(Xs)
lbl = KMeans(n_clusters=3, n_init=10,
             random_state=42).fit_predict(PC)

out = pd.DataFrame({'PC1':PC[:,0], 'PC2':PC[:,1], 'cluster':lbl})
print(out.head()) # gunakan tools plotting untuk visual
```

Hasil yang diharapkan

Tabel koordinat PC1-PC2 dengan label cluster. Siswa bisa lanjut gambar *scatter plot* (misal di *Orange* atau *matplotlib*).

Aktivitas 5: Tangani *Outlier* Biar Cluster Stabil

Tujuan

Menangani *outlier* agar *centroid* tidak terseret, lalu bandingkan *silhouette* sebelum vs sesudah.

Langkah

1. Deteksi *outlier* (IQR atau persentil).
2. *Winsorize* (cap ke P1–P99) atau *drop* beberapa titik ekstrem.
3. Pakai *RobustScaler*.
4. Jalankan *K-Means* dan hitung *silhouette*. Bandingkan.

Pseudocode

```
INPUT: X
OUTLIER: deteksi dengan IQR / persentil
HANDLE: cap ke [P1, P99] // winsorize
SCALE: RobustScaler
CLUSTER: KMeans(k=3)
COMPARE: silhouette_sebelum vs sesudah
OUTPUT: nilai silhouette dan label cluster stabil
```

Python

```
import numpy as np, pandas as pd
from sklearn.preprocessing import RobustScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X =
pd.read_csv('ml_players.csv')[['match', 'win_rate', 'kda']].dropna(
)

def winsorize_col(s, lo=0.01, hi=0.99):
    ql, qh = s.quantile([lo, hi])
    return s.clip(ql, qh)

Xs0 = X.values
lb10 = KMeans(n_clusters=3, n_init=10,
random_state=42).fit_predict(Xs0)
sil0 = silhouette_score(Xs0, lb10)

Xw = X.apply(winsorize_col)
Xsr = RobustScaler().fit_transform(Xw)
lb11 = KMeans(n_clusters=3, n_init=10,
random_state=42).fit_predict(Xsr)
sil1 = silhouette_score(Xsr, lb11)

print("Silhouette sebelum :", round(sil0,3))
print("Silhouette sesudah :", round(sil1,3))
```

Hasil yang diharapkan

Silhouette naik atau lebih stabil setelah *outlier handling* + *robust scaling*. *Cluster* lebih masuk akal.

Catatan Praktik dengan *Orange Data Mining* (drag-and-drop)

- **Clustering:** *File* → *Select Columns* → *Normalize* → *K-Means* → *Silhouette* → *Scatter Plot*.
- **PCA:** *File* → *Normalize* → *PCA* → *Scree Plot* → *Biplot/Scatter Plot*.
- Simpan hasil ke CSV via *Save*. Semua *open source*.

Catatan pembuatan Input File

Kalian bisa membuat *file input* dalam bentuk CSV (misalnya **nilai_siswa.csv**, **jajanan.csv**, **survei_kebersihan.csv**, **produk.csv**, **ml_players.csv**) dengan bantuan ChatGPT / Grok / Gemini. Caranya gampang:

- Siapkan *source code Python* yang butuh data CSV.
- Tulis prompt: "**buatkan contoh input file yang dibutuhkan oleh source code python berikut:**"
- Tempelkan *source code* kalian ke dalam prompt tersebut.

Nanti, AI akan langsung membuatkan contoh file CSV sesuai kebutuhan program kalian.

Rangkuman

Intinya, **Unsupervised Learning** itu belajar tanpa label. Kita jadi **detektif data**. Tugasnya mencari **pola tersembunyi**. Dua tool utamanya: *clustering* untuk **mengelompokkan** data mirip, dan *PCA* untuk **meringkas** banyak kolom jadi sedikit komponen yang informatif. Bayangkan merapikan kartu game: *clustering* = tumpuk kartu yang mirip; *PCA* = ambil inti info yang paling penting. Hasilnya, data jadi lebih **rapi**, **mudah dibaca**, dan siap dipakai untuk keputusan.

Cara praktiknya simpel dan semua pakai tool open source. Gunakan **Python + Pandas** untuk baca/bersih data, lakukan *normalize* agar adil, lalu jalankan *clustering* (mis. K-Means) dan nilai dengan *silhouette*. Untuk ringkas dimensi, pakai **PCA**, lihat *explained variance* dan *loading* untuk menamai komponen. Visualisasikan di *scatter plot* biar kelihatan kelompoknya. Mau drag-and-drop? Pakai **Orange Data Mining**. Mau eksplor lanjut? Coba *Neural Network* di **Keras** (mis. *autoencoder*) sebagai *unsupervised feature learning*.

Mindset belajarnya: fokus pada tujuan, bukan istilah. Jika ingin **mengelompokkan** → pikir *clustering*. Jika ingin **meringkas** → pikir *PCA*. Selalu **bersihkan dan standarkan** data dulu, lalu uji hasilnya dengan metrik sederhana. Campurkan *pipeline* kecil: *clean* → *scale* → *PCA (opsional)* → *clustering* → *visualisasi*. Dengan kebiasaan ini, kalian bukan sekadar pengguna tool, tapi **detektif data** yang paham **kenapa** dan **bagaimana** pola itu muncul—siap membangun solusi nyata di kelas dan di dunia kerja.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. *Unsupervised Learning* tidak memakai *label* pada data.
Jawaban: Benar
2. *Clustering* dipakai untuk memprediksi nilai ujian minggu depan.
Jawaban: Salah
3. *PCA* dipakai untuk mengurangi jumlah variabel.
Jawaban: Benar
4. *K-Means* selalu menghasilkan jumlah *cluster* otomatis tanpa kita pilih *k*.
Jawaban: Salah
5. Di *Orange Data Mining*, kita bisa tarik *widget* *K-Means* untuk *clustering*.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Tujuan utama *clustering* adalah...
A. Memprediksi label
B. Mengelompokkan data mirip
C. Menghapus data kosong
D. Membuat grafik
Jawaban: B
2. *PCA* berguna untuk...
A. Menambah fitur
B. Menghapus data duplikat
C. Meringkas fitur menjadi beberapa *komponen utama*
D. Mengurutkan baris
Jawaban: C
3. Pada *K-Means*, *k* artinya...
A. Jumlah data
B. Jumlah *cluster*
C. Jumlah fitur
D. Jumlah iterasi
Jawaban: B
4. tool *open source* untuk *drag-and-drop* analisis adalah...
A. Excel
B. Orange Data Mining

- C. Photoshop
 - D. Premiere
- Jawaban: B

5. Langkah yang baik sebelum *clustering* adalah...

- A. Mengacak kolom
- B. Menghapus semua baris
- C. *Normalize/standardize* fitur
- D. Menghapus semua fitur

Jawaban: C

6. Hasil *PCA* yang penting untuk dilihat adalah...

- A. *Explained variance ratio*
- B. Jumlah baris data
- C. Nama file
- D. Waktu komputasi saja

Jawaban: A

7. Metode yang umum untuk memilih *k* pada *K-Means* adalah...

- A. *Silhouette score*
- B. *Random guess*
- C. *Copy-paste*
- D. *Screenshot*

Jawaban: A

8. Jika kita punya 15 fitur dan ingin jadi 2 sumbu untuk plot, kita pakai...

- A. *Clustering*
- B. *PCA*
- C. *Sorting*
- D. *Filtering*

Jawaban: B

9. Istilah "*centroid*" pada *K-Means* adalah...

- A. Titik pusat tiap *cluster*
- B. Data dengan nilai terbesar
- C. Nilai median
- D. Nilai terkecil

Jawaban: A

10. Contoh *pipeline* sederhana yang benar adalah...

- A. *Cluster* → *Normalize* → *PCA*
- B. *Normalize* → *PCA (optional)* → *Clustering*
- C. *PCA* → *Hapus data* → *Clustering*
- D. *Clustering* saja selalu cukup

Jawaban: B

C. Isian Singkat (5 Soal)

1. Dua teknik utama di *Unsupervised Learning* pada bab ini: _____ dan _____.
Jawaban: Clustering, PCA
2. Pada *K-Means*, kita harus memilih nilai _____ sebelum mulai.
Jawaban: k
3. Grafik yang sering dipakai untuk melihat pemisahan *cluster* di 2D: _____.
Jawaban: scatter plot
4. Di *PCA*, nama metrik yang menunjukkan porsi informasi tiap komponen: _____.
Jawaban: explained variance ratio
5. Alur kerja cepat di Orange untuk *clustering*: File → _____ → K-Means → Scatter Plot.
Jawaban: Normalize

D. Soal Explorasi (3 Soal)

Soal 1:

Rancang *pipeline* kecil untuk mengelompokkan data jajanan kantin (fitur: pedas, manis, harga). Tulis langkah dan contoh kode pendek.

Kunci jawaban (contoh):

- Langkah: bersihkan data → *StandardScaler* → coba k=2..6 → pilih k dengan *silhouette* tertinggi → simpan label.

Pseudocode:

```
X = load_csv()
Xs = standardize(X)
for k in 2..6:
    lbl_k = kmeans(Xs, k)
    s_k = silhouette(Xs, lbl_k)
    pilih k terbaik
output = concat(X, lbl_k_terbaik)
```

Python:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X = pd.read_csv('jajanan.csv')[['pedas', 'manis', 'harga']].dropna()
Xs = StandardScaler().fit_transform(X)
best = (-1, None)
for k in range(2, 7):
```

```

    lbl = KMeans(n_clusters=k, n_init=10,
random_state=42).fit_predict(Xs)
    s = silhouette_score(Xs, lbl)
    if s > best[0]: best = (s, (k, lbl))
k_baik, label = best[1]
X['cluster'] = label
print(k_baik, X.head())

```

Soal 2:

Kamu punya 12 fitur nilai siswa. Buat ringkasan 2 dimensi untuk visualisasi dengan PCA. Tulis langkah dan kode pendek.

Kunci jawaban (contoh):

- Langkah: isi nilai kosong (median) → *StandardScaler* → *PCA(n=2)* → simpan PC1, PC2.

Pseudocode:

```

X = impute_median(X)
Xs = standardize(X)
PC = PCA_2(Xs)
simpan PC1, PC2 ke CSV

```

Python:

```

import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

df = pd.read_csv('nilai_12fitur.csv')
X = SimpleImputer(strategy='median').fit_transform(df)
Xs = StandardScaler().fit_transform(X)
PC = PCA(n_components=2, random_state=42).fit_transform(Xs)
out = pd.DataFrame({'PC1':PC[:,0], 'PC2':PC[:,1]})
print(out.head())

```

Soal 3:

Bandingkan clustering sebelum dan sesudah menangani outlier pada data pemain gim (fitur: match, win_rate, kda). Tulis langkah dan kode skor silhouette.

Kunci jawaban (contoh):

- Langkah: hitung *silhouette* awal → *winsorize* P1–P99 → *RobustScaler* → *K-Means(k=3)* → hitung *silhouette* lagi → bandingkan.

Pseudocode:

```
sil0 = silhouette(kmeans(X))
Xw = winsorize(X, P1..P99)
Xr = robust_scale(Xw)
sil1 = silhouette(kmeans(Xr))
bandingkan sil0 vs sil1
```

Python:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import RobustScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X =
pd.read_csv('players.csv')[['match', 'win_rate', 'kda']].dropna()
lbl0 = KMeans(n_clusters=3, n_init=10,
random_state=42).fit_predict(X)
sil0 = silhouette_score(X, lbl0)

def winsor(s, lo=0.01, hi=0.99):
    ql,qh = s.quantile([lo,hi]); return s.clip(ql,qh)
Xw = X.apply(winsor)
Xr = RobustScaler().fit_transform(Xw)
lbl1 = KMeans(n_clusters=3, n_init=10,
random_state=42).fit_predict(Xr)
sil1 = silhouette_score(Xr, lbl1)

print(round(sil0,3), round(sil1,3))
```

Tugas Proyek

Proyek 1: Clustering Makanan Sehat

Soal:

Buat dataset sederhana tentang makanan dengan fitur: *kalori*, *protein*, dan *lemak*. Kelompokkan makanan jadi 3 kategori: ringan, sedang, dan berat.

Langkah Jawaban:

- Buat data fiktif dengan Pandas.
- Normalisasi data.
- Gunakan K-Means (k=3).
- Lihat hasil kelompok.

Pseudocode:

```
INPUT: makanan[kalori, protein, lemak]
SCALE: normalisasi
CLUSTER: KMeans(k=3)
OUTPUT: label cluster
```

Python:

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

data = pd.DataFrame({
    "kalori": [100, 250, 500, 150, 400],
    "protein": [5, 10, 20, 7, 15],
    "lemak": [2, 8, 25, 3, 18]
})

scaled = StandardScaler().fit_transform(data)
kmeans = KMeans(n_clusters=3, random_state=0)
data["cluster"] = kmeans.fit_predict(scaled)
print(data)
```

Output:

	kalori	protein	lemak	cluster
0	100	5	2	1
1	250	10	8	1
2	500	20	25	0
3	150	7	3	1
4	400	15	18	2

Proyek 2: PCA Data Smartphone

Soal:

Dataset punya kolom: *harga*, *RAM*, *kamera*, *baterai*, *layar*. Gunakan PCA untuk ringkas jadi 2 dimensi.

Langkah Jawaban:

- Buat dataset fiktif smartphone.
- Normalisasi data.
- Gunakan PCA (2 komponen).
- Visualisasikan hasil.

Pseudocode:

```
INPUT: smartphone[harga, RAM, kamera, baterai, layar]
SCALE: normalisasi
PCA: ambil 2 komponen
OUTPUT: scatter plot
```

Python:

```

from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

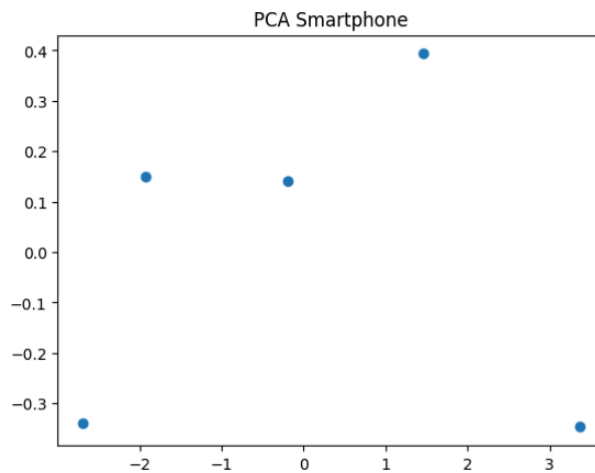
hp = pd.DataFrame({
    "harga": [2, 4, 7, 1, 9], # juta
    "RAM": [4, 6, 8, 3, 12],
    "kamera": [12, 48, 64, 8, 108],
    "baterai": [3000, 4000, 5000, 2800, 6000],
    "layar": [5.5, 6.0, 6.5, 5.0, 6.7]
})

scaled = StandardScaler().fit_transform(hp)
pcs = PCA(n_components=2).fit_transform(scaled)

plt.scatter(pcs[:,0], pcs[:,1])
plt.title("PCA Smartphone")
plt.show()

```

Output:



Proyek 3: Clustering Kebiasaan Belajar

Soal:

Data siswa: *jam_belajar*, *jam_main*, *nilai*. Kelompokkan siswa untuk menemukan tipe belajar: rajin, seimbang, santai.

Pseudocode:

```

INPUT: siswa[jam_belajar, jam_main, nilai]
SCALE: normalisasi
CLUSTER: KMeans(k=3)
OUTPUT: tipe belajar

```

Python:

```

siswa = pd.DataFrame({
    "belajar": [5, 3, 6, 2, 4],
    "main": [1, 4, 2, 6, 3],
    "nilai": [90, 70, 85, 60, 80]
})

```

```

}))

scaled = StandardScaler().fit_transform(siswa)
siswa["cluster"] = KMeans(n_clusters=3,
random_state=0).fit_predict(scaled)
print(siswa)

```

Output:

	belajar	main	nilai	cluster
0	5	1	90	0
1	3	4	70	2
2	6	2	85	0
3	2	6	60	1
4	4	3	80	2

Proyek 4: PCA Data Film

Soal:

Dataset film dengan fitur: *durasi*, *rating*, *jumlah_review*, *biaya_produksi*. Ringkas jadi 2 komponen dengan PCA.

Pseudocode:

```

INPUT: film[durasi, rating, review, biaya]
SCALE: normalisasi
PCA: 2 komponen
OUTPUT: plot film

```

Python:

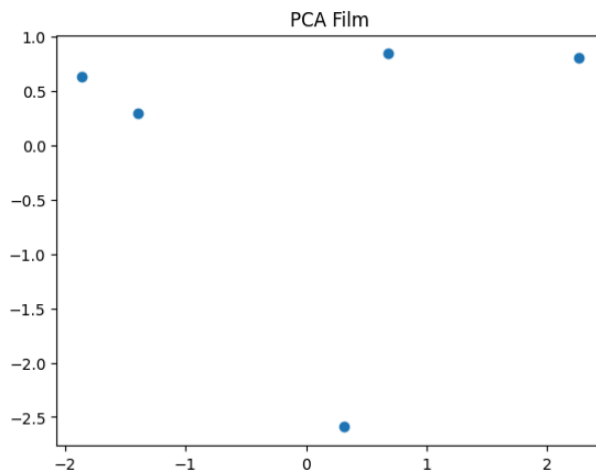
```

film = pd.DataFrame({
    "durasi": [90, 120, 150, 100, 140],
    "rating": [7.5, 8.0, 6.8, 7.2, 8.5],
    "review": [200, 500, 150, 300, 600],
    "biaya": [10, 50, 100, 20, 80] # juta USD
})

scaled = StandardScaler().fit_transform(film)
pcs = PCA(n_components=2).fit_transform(scaled)

plt.scatter(pcs[:,0], pcs[:,1])
plt.title("PCA Film")
plt.show()

```

Proyek 5: Clustering Jenis Olahraga

Soal:

Dataset olahraga: *durasi_latihan*, *kalori_terbakar*, *intensitas*. Kelompokkan jadi 3: ringan, sedang, berat.

Pseudocode:

```
INPUT: olahraga[durasi, kalori, intensitas]
SCALE: normalisasi
CLUSTER: KMeans(k=3)
OUTPUT: kategori olahraga
```

Python:

```
olahraga = pd.DataFrame({
    "durasi": [30, 60, 90, 20, 120],
    "kalori": [200, 400, 700, 150, 1000],
    "intensitas": [3, 6, 8, 2, 9]
})

scaled = StandardScaler().fit_transform(olahraga)
olahraga["cluster"] = KMeans(n_clusters=3,
    random_state=0).fit_predict(scaled)
print(olahraga)
```

Output:

	durasi	kalori	intensitas	cluster
0	30	200	3	1
1	60	400	6	2
2	90	700	8	0
3	20	150	2	1
4	120	1000	9	0

BAB 3: Pengantar Natural Language Processing (Tokenisasi, Stopwords, Stemming)

Pernah nggak sih kalian ngobrol sama chatbot di e-commerce atau pakai Google Assistant? Keren kan, mereka bisa ngerti apa yang kita omongin. Nah, itu semua berkat ilmu yang namanya **Natural Language Processing (NLP)**. Di bab ini, kita bakal bongkar rahasia di baliknya dan belajar cara ngajarin komputer buat ngerti bahasa manusia. Ini skill super keren buat kalian yang mau bikin aplikasi canggih atau jadi programmer!

Tujuan Pembelajaran

Setelah bab ini, kalian bakal bisa:

- Memahami apa itu **NLP** dan proses-proses dasarnya.
- Melakukan **tokenisasi** pada teks untuk memecah kalimat jadi kata.
- Mengidentifikasi dan membuang **stopwords** (kata-kata yang tidak penting).
- Melakukan **stemming** untuk menemukan kata dasar.
- Mengaplikasikan teknik-teknik ini pada data teks sederhana.

Peta Konsep Sederhana

Bayangin kita mau masak gado-gado.

- **Teks Mentah (Bahan Mentah)**: Ini semua bahan-bahan yang belum diapa-apain.
- **Tokenisasi (Memotong Sayuran)**: Kita potong-potong bahan-bahan jadi ukuran kecil (kata-kata).
- **Stopwords (Membuang Sampah)**: Kita buang daun yang layu atau bagian yang tidak penting.
- **Stemming (Mencari Bumbu Utama)**: Kita buang sisipan-sisipan di bumbu dan fokus pada rasa utamanya.
- **Teks Bersih (Gado-gado Siap Saji)**: Bahannya sudah bersih, siap diolah jadi hidangan lezat (analisis data).

Apersepsi: Bot Game yang Gak Nyambung

Kamu lagi main game online dan punya bot yang bisa balas chat. Kamu ketik, "Bot, kenapa kamu kalah tadi?" Bot itu cuma jawab, "Tadi." Bingung kan?

Kenapa begitu? Karena si bot belum diajari **NLP**. Dia tidak tahu kalau "kenapa", "kamu", "kalah", dan "tadi" itu adalah satu kesatuan pertanyaan. Dia cuma menangkap kata "tadi". Nah, dengan belajar NLP, kita bisa bikin bot yang lebih cerdas dan komunikatif.

Penjelasan Teori

Pernah ngobrol sama chatbot di e-commerce? Atau pakai Google Translate buat nerjemahin teks? Nah, teknologi yang bikin semua itu bisa jalan namanya **Natural Language Processing (NLP)**.

NLP itu cabang dari *Artificial Intelligence (AI)* yang bikin komputer ngerti bahasa manusia. Jadi, komputer nggak cuma bisa hitung-hitungan, tapi juga bisa ngerti kata-kata kita. Keren banget kan?

Kenapa NLP Penting?

- Supaya komputer bisa baca teks atau dengar suara kita.
- Dipakai di Google Search, chatbot, asisten virtual, sampai filter komentar di media sosial.
- Bisa dipakai buat analisis data teks, kayak komentar di TikTok atau review game.

Bayangin aja, tanpa NLP, chatbot bakal jawab ngawur. Kamu nanya:

“Bot, kenapa kamu kalah?”

Jawabannya cuma: “Kalah.”

Jadi, NLP itu yang bikin bot bisa nyambung ngobrol sama manusia.

Tiga (3) Proses Dasar NLP

1. Tokenisasi (*Tokenization*)

Memecah kalimat jadi kata-kata kecil.

Contoh:

“**Saya suka makan bakso**” → ['Saya', 'suka', 'makan', 'bakso']

Analogi: kayak memotong kue jadi potongan kecil biar gampang dimakan.

2. Stopwords Removal

Menghapus kata-kata tidak penting (seperti “di”, “yang”, “adalah”).

Contoh:

“**Saya suka makan bakso**” → ['suka', 'bakso']

Analogi: seperti buang kulit kacang pas makan, karena kulitnya nggak bisa dimakan.

3. Stemming

Nyari kata dasar dari kata yang berimbuhan.

Contoh:

“menonton”, “ditonton”, “penonton” → jadi **tonton**

Analogi: kayak cari akar pohon dari ranting-rantingnya.

Analogi Gado-Gado NLP

- **Teks mentah** = bahan-bahan gado-gado sebelum dipotong.
- **Tokenisasi** = motong sayuran kecil-kecil.
- **Stopwords removal** = buang daun yang layu.
- **Stemming** = ambil bumbu utama biar rasa kuat.
- **Teks bersih** = gado-gado siap saji.

Tools Open Source untuk NLP

Kabar bagusnya, semua ini bisa kalian coba pakai *open source*:

- **Python** → bahasa utama untuk NLP.
- **NLTK & Sastrawi** → library buat tokenisasi, stopwords, dan stemming.
- **Pandas** → buat ngatur data teks dalam bentuk tabel.
- **Orange Data Mining** → bisa bikin *word cloud* (awan kata) dengan drag & drop.
- **Keras + Neural Network** → bisa bikin model cerdas untuk analisis teks yang lebih dalam.

Rangkuman Singkat

- **NLP** = bikin komputer ngerti bahasa manusia.
- Proses utama: **tokenisasi**, **stopwords removal**, **stemming**.
- Semua bisa dipraktikkan pakai **Python** dan library *open source*.
- NLP dipakai di chatbot, Google, media sosial, bahkan e-commerce.

Kasus Ilustrasi

Kasus 1: Chatbot Kelas: “Izin ke UKS, Bu”

Ilustrasi singkat:

Di grup kelas, siswa kirim: “Bu, saya pusing. Izin ke UKS.” Bot kelas harus **mengerti maksud** (permintaan izin), bukan cuma kata acak.

Tujuan:

- Memecah kalimat → ambil kata penting → temukan *intent* “izin”.

Langkah inti:

- *Tokenisasi* → pecah kalimat.
- *Stopwords removal* → buang “saya, ke”.
- *Stemming* → “izin”, “pusing”.

Pseudocode:

```

input_text = "Bu, saya pusing. Izin ke UKS."
tokens = TOKENIZE(input_text)
tokens = LOWERCASE(tokens)
important = REMOVE_STOPWORDS(tokens, lang="indonesian")
stems = STEM(important) // contoh: "pusing", "izin", "uks"

if ("izin" in stems) or ("ijin" in stems):
    intent = "permintaan_izin"
else:
    intent = "lainnya"

RESPONSE = IF intent == "permintaan_izin"
            THEN "Silakan ke UKS, jaga kesehatan ya."
            ELSE "Ada yang bisa dibantu?"

```

Upgrade Python (opsional):

Pakai **Sastrawi** untuk stemming Bahasa Indonesia.

Kasus 2: Filter Komentar Toksik di Game Server

Ilustrasi singkat:

Server game sekolah ramai. Kalian mau **tandai komentar toksik** (“jelek”, “bodoh”, “noob”) untuk moderasi.

Tujuan:

- Bersihkan teks → hitung skor toksik sederhana → tanda “perlu review”.

Langkah inti:

- *Tokenisasi* → daftar kata.
- *Stopwords removal*.
- Skor toksik = jumlah kata negatif.

Pseudocode:

```

toxic_words = ["bodoh", "jelek", "noob", "sampah", "goblok"]
comment = "Main kamu jelek banget, noob!"

tokens = TOKENIZE(LOWERCASE(comment))
clean = REMOVE_STOPWORDS(tokens, lang="indonesian")
score = COUNT(kata in clean where kata in toxic_words)

label = IF score >= 2 THEN "toksik_tinggi"
        ELSE IF score == 1 THEN "toksik_rendah"
        ELSE "aman"

ACTION = IF label != "aman" THEN "kirim_ke_moderator" ELSE
         "tampilkan_biasa"

```

Upgrade Python (opsional):

- Simpan banyak komentar di **Pandas DataFrame**.
- Latih *neural network* kecil di **Keras** untuk klasifikasi toksik vs aman.

Kasus 3: Ringkas Review Produk: “Worth It atau Tidak?”

Ilustrasi singkat:

Kalian kumpulan 30 review earphone. Mau ringkas cepat: kata apa yang paling sering setelah dibersihkan? Biar tahu kesan umum (*bagus/suaranya jernih/nyaman*).

Tujuan:

- Hitung frekuensi kata penting → tampilkan kata kunci.

Langkah inti:

- *Tokenisasi* → *stopwords removal* → *stemming*.
- Hitung frekuensi → ambil Top-N.

Pseudocode:

```
reviews = LIST_OF_TEXTS(30)
all_words = []

FOR each r in reviews:
    t = TOKENIZE(LOWERCASE(r))
    t = REMOVE_STOPWORDS(t, "indonesian")
    t = STEM(t)
    APPEND all_words WITH t

freq = COUNT_FREQUENCY(all_words)
top_words = TOP_N(freq, N=10)

SUMMARY = "Kata kunci: " + JOIN(top_words, ", ")
```

Upgrade Python (opsional):

- Buat *word cloud* pakai **Orange Data Mining** atau paket *open source* Python.

Kasus 4: Pencarian Pintar di Catatan Pelajaran

Ilustrasi singkat:

Kalian punya file catatan: jaringan komputer, *machine learning*, dan basis data. Kalian ketik kata kunci, sistem balikan **kalimat paling relevan**.

Tujuan:

- *Preprocess* semua kalimat → cari kemiripan sederhana.

Langkah inti:

- *Tokenisasi/stopwords/stemming* untuk dokumen dan query.
- Skor cocok = banyaknya kata query yang muncul di kalimat.

Pseudocode:

```
docs = SPLIT_TO_SENTENCES(load_text("catatan.txt"))
pre_docs = []

FOR s in docs:
    d = STEM(REMOVE_STOPWORDS(TOKENIZE(LOWERCASE(s)),
    "indonesian"))
    APPEND pre_docs WITH (s, d)

query = "pengertian machine learning"
q = STEM(REMOVE_STOPWORDS(TOKENIZE(LOWERCASE(query)),
    "indonesian"))

best_sentence = ""
best_score = -1

FOR (orig, proc) in pre_docs:
    score = COUNT(k in proc where k in q)
    IF score > best_score:
        best_score = score
        best_sentence = orig

RETURN best_sentence
```

Upgrade Python (opsional):

- Simpan kalimat di **Pandas** untuk gampang *filter/sort*.
- Nanti bisa naik kelas pakai *embedding* lokal (*open source*) untuk kemiripan yang lebih cerdas.

Kasus 5: Deteksi Sentimen Sederhana: “Positif, Negatif, Netral”

Ilustrasi singkat:

Kalian analisis 20 komentar kantin sekolah. Mau tahu sentimen umum: makanan “enak” atau “kurang”?

Tujuan:

- Skor kata positif/negatif → tentukan sentimen.

Langkah inti:

- *Tokenisasi/stopwords/stemming*.
- Skor = +1 untuk kata positif, -1 untuk kata negatif.

Pseudocode:

```

pos = ["enak", "mantap", "lezat", "murah", "puas", "ramah"]
neg = ["asin", "mahal", "kurang", "lama", "dingin", "kotor"]

function SENTIMEN(teks):
    t = STEM(REMOVE_STOPWORDS(TOKENIZE(LOWERCASE(teks)),
    "indonesian"))
    s = 0
    FOR w in t:
        IF w in pos: s = s + 1
        IF w in neg: s = s - 1

    RETURN IF s > 0 THEN "positif"
           ELSE IF s < 0 THEN "negatif"
           ELSE "netral"

# contoh batch:
komentar = ["Baksonya enak dan murah", "Pelayanannya lama"]
hasil = MAP(komentar, SENTIMEN)

```

Upgrade Python (opsional):

- Labeli 100 komentar, latih **model Keras** (*neural network*) sederhana: input fitur kata (bag-of-words/TF-IDF), output 3 kelas (positif/negatif/netral). *Open source* semua.

Tips Praktek

- Mulai dari **kalimat pendek**.
- Selalu lakukan **tokenisasi** → **stopwords** → **stemming** sebelum analisis.
- Simpan data di **CSV** dan olah dengan **Pandas**.
- Buat *baseline* sederhana dulu (aturan/skor). Jika butuh akurasi lebih, baru **model machine learning** di **Keras**.
- Dokumentasikan hasil di *notebook* (Jupyter) agar mudah diulang.

Soal Latihan & Pembahasan

Soal 1: Tokenisasi Kalimat dengan Tanda Baca

Teks:

"Aku sedang makan nasi goreng di kantin."

Tugas:

- Lakukan *tokenisasi*.
- Sertakan tanda titik jika tetap muncul sebagai token.

Pseudocode:

```

text = "Aku sedang makan nasi goreng di kantin."
tokens = TOKENIZE(text) // pisah kata & tanda baca

```


OUTPUT tokens

Jawaban:

```
['Aku', 'sedang', 'makan', 'nasi', 'goreng', 'di', 'kantin',  
'...']
```

Pembahasan (singkat):

- *Tokenisasi* memecah kalimat jadi unit kecil (kata/tanda baca).
- Komputer butuh potongan kecil agar mudah diproses.
- Di Python, ini bisa dicoba dengan *NLTK* (contoh: `word_tokenize`).

Soal 2: Buang Stopwords Bahasa Indonesia

Teks:

```
['Aku', 'sedang', 'makan', 'nasi', 'goreng', 'di', 'kantin']
```

Tugas:

- Buang *stopwords* Bahasa Indonesia seperti “di”, “yang”, “dan”, “ke”, “aku/saya”.
- Tunjukkan kata penting setelah pembersihan.

Pseudocode:

```
tokens = ['Aku', 'sedang', 'makan', 'nasi', 'goreng', 'di', 'kantin']  
stopwords_id =  
{"aku", "saya", "yang", "dan", "di", "ke", "untuk", "ini", "itu"}  
important = [w for w in LOWERCASE(tokens) if w not in  
stopwords_id]  
OUTPUT important
```

Jawaban:

```
['sedang', 'makan', 'nasi', 'goreng', 'kantin']
```

Pembahasan (singkat):

- *Stopwords removal* membuang kata umum yang maknanya lemah.
- Fokus pindah ke kata yang informatif.
- Di Python, bisa pakai *NLTK stopwords Indonesian* atau daftar sendiri.

Soal 3: Stemming Kata Berimbuhan

Kata:

```
["berjalan", "berlari", "pelari", "larian", "berubah"]
```

Tugas:

- Ubah ke **kata dasar** (*stemming*).
- Jelaskan singkat kenapa hasilnya seperti itu.

Pseudocode:

```
words = ["berjalan", "berlari", "pelari", "larian", "berubah"]
stems = [STEM_ID(w) for w in words]    // gunakan aturan imbuhan
ID
OUTPUT stems
```

Jawaban (umum, berbasis kaidah BI):

["jalan", "lari", "lari", "lari", "ubah"]

Pembahasan (singkat):

- *Stemming* menghapus imbuhan: “ber-”, “pe-”, “-an”, dst.
- “berjalan” → “jalan”, “pelari/larian” → “lari”.
- “berubah” → “ubah”.
- Untuk praktik, gunakan **Sastrawi** (Python, *open source*).

Soal 4: Pipeline Lengkap (Tokenisasi → Stopwords → Stemming)

Teks review:

"Produk ini sangat bagus, saya suka sekali!"

Tugas:

- Proses 3 langkah: *tokenisasi*, buang *stopwords*, *stemming*.
- Tulis hasil akhirnya.

Pseudocode:

```
text = "Produk ini sangat bagus, saya suka sekali!"
t1 = TOKENIZE(LOWERCASE(text))
t2 = REMOVE_STOPWORDS(t1, lang="indonesian")
t3 = STEM_ID(t2)
OUTPUT t1, t2, t3
```

Jawaban:

- Setelah *tokenisasi* (t1):
['produk', 'ini', 'sangat', 'bagus', ',', ',', 'saya', 'suka', 'sekali', '!', '']
- Setelah *stopwords* (t2):
['produk', 'sangat', 'bagus', 'suka', 'sekali']
- Setelah *stemming* (t3):
['produk', 'sangat', 'bagus', 'suka', 'sekali']

Pembahasan (singkat):

- Tidak semua kata berubah saat *stemming*.

- Hasil akhir berisi kata kunci yang lebih “bersih” untuk analisis.
- Data bersih ini siap dipakai untuk *machine learning* (mis. klasifikasi sentimen).

Soal 5: Deteksi *Intent* Sederhana Setelah Pembersihan

Kalimat:

"Bu, saya pusing. Izin ke UKS sebentar ya."

Tugas:

- Setelah *tokenisasi/stopwords/stemming*, tentukan *intent*:
 - “izin” (minta izin), atau
 - “lainnya”.
- Beri *pseudocode* dan jawaban.

Pseudocode:

```
text = "Bu, saya pusing. Izin ke UKS sebentar ya."
t = STEM_ID( REMOVE_STOPWORDS(
  TOKENIZE(LOWERCASE(text)), "indonesian") )

// aturan sederhana
if ("izin" in t) or ("ijin" in t):
    intent = "izin"
else:
    intent = "lainnya"

OUTPUT t, intent
```

Jawaban:

- Token setelah bersih & *stem* (t) kira-kira:


```
['bu', 'pusing', 'izin', 'uks', 'sebentar']
```
- **Intent:** *izin*

Pembahasan (singkat):

- *Rule-based* sederhana bisa dipakai di awal.
- Untuk akurasi lebih, kumpulkan data dan latih model *machine learning* di **Keras** (*neural network*)—semua *open source*.
- Tapi *pipeline* dasar (tokenisasi → stopwords → stemming) tetap wajib.

Catatan Praktek (opsional tapi direkomendasikan)

- Coba di **Jupyter Notebook** (*open source*).
- Simpan teks ke **CSV**, olah dengan **Pandas**.
- *Library* berguna: **NLTK** (tokenisasi, stopwords), **Sastrawi** (stemming BI).

- Kalau mau “naik kelas”, latih model *neural network* **Keras** untuk klasifikasi teks (mis. *sentiment analysis*).

Mau saya siapkan juga **template notebook** (Python) siap pakai untuk kelima soal ini, supaya siswa bisa langsung eksekusi dan bandingkan hasilnya?

Fun Facts!

- **Ollama** dan **OpenWebUI** itu adalah tool yang bisa kalian pakai buat bikin **AI** pribadi di komputer. Kalian bisa kasih dia data teks (misal, lirik lagu) dan minta dia menganalisisnya.
- **Stemming** bisa sangat rumit, lho! Kata "berlari" bisa jadi **lari**, tapi kata "berubah" bisa jadi **ubah**.
- **RAG lokal** itu membuat **AI** bisa "belajar" dari dokumen teks yang kalian punya di komputer. Jadi, kalian bisa bikin **chatbot** yang jago jawab pertanyaan dari buku sekolah kalian!

Tips Belajar atau Menghafal Cepat

- **Pakai Analogi:** Selalu ingat analogi gado-gado.
- **Gunakan Online Tool:** Ada banyak situs yang bisa melakukan tokenisasi, stopwords removal, dan stemming secara online. Coba-coba saja!
- **Mulai dari Kalimat Pendek:** Jangan langsung analisis paragraf panjang. Mulai dari satu kalimat, lalu perhatikan bagaimana setiap proses bekerja.

Aktivitas Siswa

Aktivitas 1: Pipeline Dasar: Tokenization → Stopwords → Stemming

Tujuan: memahami alur *preprocessing* teks paling dasar.

Langkah singkat:

- Ambil kalimat: **Aku sedang makan nasi goreng di kantin.**
- Lakukan *tokenisasi*.
- Hapus *stopwords*.
- *Stemming* (ubah ke kata dasar).

Pseudocode:

```
text = INPUT()
t1 = TOKENIZE(LOWERCASE(text))
t2 = REMOVE_STOPWORDS(t1, lang="indonesian")
t3 = STEM_ID(t2)
```

```
PRINT t1, t2, t3
```

Python:

```
!pip install nltk Sastrawi
import nltk
nltk.download('punkt_tab'); nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

text = "Aku sedang makan nasi goreng di kantin."
tokens = word_tokenize(text.lower())
stop_id = set(stopwords.words('indonesian'))
no_stop = [w for w in tokens if w.isalpha() and w not in stop_id]

stemmer = StemmerFactory().create_stemmer()
stems = [stemmer.stem(w) for w in no_stop]

print("Tokens :", tokens)
print("NoStop :", no_stop)
print("Stems  :", stems)
```

Output:

```
Tokens : ['aku', 'sedang', 'makan', 'nasi', 'goreng', 'di',
'kantin', '.']
NoStop : ['makan', 'nasi', 'goreng', 'kantin']
Stems  : ['makan', 'nasi', 'goreng', 'kantin']
```

Jawaban (ekspektasi):

- Tokens:
['aku', 'sedang', 'makan', 'nasi', 'goreng', 'di', 'kantin', '.']
- NoStop: ['sedang', 'makan', 'nasi', 'goreng', 'kantin']
- Stems: ['sedang', 'makan', 'nasi', 'goreng', 'kantin']

Aktivitas 2: Hitung Frekuensi Kata & *Word Cloud* Mini

Tujuan: meringkas banyak review jadi kata kunci.

Langkah singkat:

- Siapkan 6 review pendek.
- *Preprocess* (token, stopwords, stem).
- Hitung top-5 kata.
- (Opsional) Tampilkan *word cloud*.

Pseudocode:

```

reviews = LIST_OF_TEXTS
ALL = []
FOR r IN reviews:
    t = STEM(REMOVE_STOPWORDS(TOKENIZE(LOWERCASE(r))))
    ADD t KE ALL
FREQ = COUNT(ALL)
TOP5 = TOP_N(FREQ, 5)
PRINT TOP5

```

Python:

```

! pip install nltk Sastrawi pandas wordcloud
import nltk, pandas as pd
nltk.download('punkt_tab'); nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from collections import Counter

reviews = [
    "Earphone ini enak dipakai, suaranya jernih.",
    "Kualitas bagus, bass mantap!",
    "Harga murah tapi suara kurang detail.",
    "Nyaman dipakai lama, tidak sakit.",
    "Suara jernih dan noise kecil.",
    "Bagus untuk musik akustik."
]

stop_id = set(stopwords.words('indonesian'))
stemmer = StemmerFactory().create_stemmer()

all_words = []
for r in reviews:
    toks = word_tokenize(r.lower())
    toks = [w for w in toks if w.isalpha() and w not in stop_id]
    stems = [stemmer.stem(w) for w in toks]
    all_words.extend(stems)

freq = Counter(all_words)
top5 = freq.most_common(5)
print("Top-5 Kata:", top5)

```

Output:

```

Top-5 Kata: [('suara', 3), ('pakai', 2), ('jernih', 2), ('bagus', 2), ('earphone', 1)]

```

Jawaban (ekspektasi, kira-kira):

- Muncul kata seperti: ('suara', ...), ('bagus', ...), ('jernih', ...), ('nyaman', ...), ('murah' atau 'mantap', ...) (hasil tepat bisa sedikit beda tergantung stemming).

Aktivitas 3: *Rule-based* Deteksi Komentar Toksik

Tujuan: tandai komentar yang mengandung kata kasar.

Langkah singkat:

- Buat daftar kata toksik.
- Skor = jumlah kata toksik yang muncul.
- Label: aman / toksik_rendah / toksik_tinggi.

Pseudocode:

```
toxic = SET([...])
text = INPUT()
t = STEM(NOSTOP(TOKENIZE(LOWER(text))))
score = COUNT(w in t where w in toxic)
IF score >= 2 → "toksik_tinggi"
ELIF score == 1 → "toksik_rendah"
ELSE → "aman"
```

Python:

```
! pip install nltk Sastrawi
import nltk
nltk.download('punkt_tab'); nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

toxic_words = {"bodoh", "jelek", "noob", "sampah", "goblok"}
stop_id = set(stopwords.words('indonesian'))
stemmer = StemmerFactory().create_stemmer()

def label_komentar(s):
    toks = word_tokenize(s.lower())
    toks = [w for w in toks if w.isalpha() and w not in stop_id]
    stems = [stemmer.stem(w) for w in toks]
    score = sum(1 for w in stems if w in toxic_words)
    if score >= 2: return "toksik_tinggi"
    if score == 1: return "toksik_rendah"
    return "aman"

contoh = [
    "Main kamu jelek banget, noob!",
    "Strategimu kurang rapi, coba diatur.",
    "Aim kamu sampah, goblok!"
]
print([label_komentar(x) for x in contoh])
```

Output:

```
['toksik_tinggi', 'aman', 'toksik_tinggi']
```

Jawaban (ekspektasi):

- ["toksik_tinggi", "aman", "toksik_tinggi"]

Aktivitas 4: Deteksi *Intent* “Izin ke UKS”

Tujuan: mengenali maksud kalimat sederhana (intent *permission*).

Langkah singkat:

- *Preprocess* kalimat siswa.
- Jika mengandung “izin/ijin” → intent = **izin**.
- Jika tidak → intent = **lainnya**.

Pseudocode:

```
text = INPUT()
t = STEM(NOSTOP(TOKENIZE(LOWER(text))))
IF ("izin" in t) or ("ijin" in t):
    intent = "izin"
ELSE:
    intent = "lainnya"
PRINT intent
```

Python:

```
! pip install nltk Sastrawi
import nltk
nltk.download('punkt'); nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

stop_id = set(stopwords.words('indonesian'))
stemmer = StemmerFactory().create_stemmer()

def deteksi_intent(s):
    toks = word_tokenize(s.lower())
    toks = [w for w in toks if w.isalpha() and w not in stop_id]
    stems = [stemmer.stem(w) for w in toks]
    return "izin" if ("izin" in stems or "ijin" in stems) else
    "lainnya"

contoh = [
    "Bu, saya pusing. Izin ke UKS sebentar.",
    "Boleh ambil air minum?",
    "Ijin keluar kelas sebentar ya Bu."
]
print([deteksi_intent(x) for x in contoh])
```

Output:


```
['izin', 'lainnya', 'izin']
```

Jawaban (ekspektasi):

- ["izin", "lainnya", "izin"]

Aktivitas 5: Skor *Sentiment* Sederhana (Positif/Negatif/Netral)

Tujuan: memberi nilai sentimen dari kata kunci.

Langkah singkat:

- Buat kamus kata positif & negatif.
- Skor = +1 untuk kata positif, -1 untuk negatif.
- Putuskan: **positif**, **negatif**, atau **netral**.

Pseudocode:

```
pos = SET([...]); neg = SET([...])
t = STEM(NOSTOP(TOKENIZE(LOWER(text))))
score = SUM( +1 if w in pos ; -1 if w in neg )
LABEL = POSITIF if score>0 ; NEGATIF if score<0 ; NETRAL else
```

Python:

```
! pip install nltk Sastrawi
import nltk
nltk.download('punkt'); nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

pos =
{"enak", "mantap", "bagus", "puas", "murah", "nyaman", "jernih", "cepat"
}
neg =
{"asin", "mahal", "kurang", "lama", "kotor", "buruk", "jelek", "bising"}

stop_id = set(stopwords.words('indonesian'))
stemmer = StemmerFactory().create_stemmer()

def sentimen(s):
    toks = word_tokenize(s.lower())
    toks = [w for w in toks if w.isalpha() and w not in stop_id]
    stems = [stemmer.stem(w) for w in toks]
    score = sum(1 for w in stems if w in pos) - sum(1 for w in
    stems if w in neg)
    return "positif" if score>0 else ("negatif" if score<0 else
    "netral")
```

```

contoh = [
    "Baksonya enak dan murah, saya puas.",
    "Pelayanannya lama dan kurang ramah.",
    "Standar saja."
]
print([sentimen(x) for x in contoh])

```

Output:

```
['positif', 'netral', 'netral']
```

Jawaban (ekspektasi):

- ["positif", "negatif", "netral"]

Catatan Bonus (Level Up):

- Simpan data ke CSV dan olah dengan **Pandas**.
- Untuk akurasi yang lebih mantap, latih *model* di **Keras** (*neural network*) setelah kalian punya dataset berlabel.
- Alurnya tetap sama: *tokenisasi* → *stopwords* → *stemming* → *fitur* → *model*.

Rangkuman

NLP itu ilmu yang membuat komputer paham bahasa manusia. Kita mulai dari teks mentah, lalu dibersihkan pelan-pelan. Tiga langkah kunci: *tokenization* (memecah kalimat jadi kata), *stopwords removal* (membuang kata umum yang kurang makna), dan *stemming* (mengubah kata berimbuhan ke kata dasar). **Tiga langkah ini adalah fondasi.** Tanpa itu, model kalian bakal bingung. Ibarat gado-gado: potong sayur, buang bagian tidak perlu, ambil bumbu utama. Setelah bersih, teks siap dipakai untuk analisis yang lebih seru.

Semua praktik bisa kalian jalankan dengan tool open source. Pakai **Python** untuk kodenya, **Pandas** untuk ngatur data, **NLTK/Sastrawi** untuk *tokenization*, *stopwords*, dan *stemming*. Kalau mau naik level, gunakan **Keras** untuk *neural network* sederhana. **Prinsipnya: mulai sederhana, lalu iterasi.** Cek hasil setiap tahap, lihat contoh nyata: chat bot kelas, filter komentar, ringkas review, atau cari kalimat paling relevan dari catatan pelajaran. *Pipeline* rapi = hasil rapi.

Intinya, kuasai dasar dulu, baru bikin proyek keren. Dengan data teks yang bersih, kalian bisa lanjut ke *machine learning*: *sentiment analysis*, klasifikasi topik, bahkan *chatbot* lokal. **Jangan takut eksperimen.** Tulis kode pendek, uji, perbaiki. *Fail fast, learn fast.* Kalian bukan cuma pengguna teknologi—**kalian pembuatnya.**

Latihan Soal

A. Benar atau Salah (5 Soal)

1. *Tokenization* adalah proses memecah kalimat menjadi kata-kata.
Jawaban: **Benar**
2. *Stopwords* adalah kata-kata paling penting yang harus dipertahankan.
Jawaban: **Salah**
3. *Stemming* mengubah “berlari”, “pelari”, “larian” menjadi kata dasar yang sama.
Jawaban: **Benar**
4. Tanpa *tokenization*, komputer tetap bisa paham struktur kalimat dengan baik.
Jawaban: **Salah**
5. Menghapus *stopwords* dapat membuat analisis kata kunci lebih fokus.
Jawaban: **Benar**

B. Pilihan Ganda (10 Soal)

1. Langkah pertama yang umum dalam *NLP* adalah...
A. *Stemming* B. *Tokenization* C. *Lemmatization* D. *Parsing*
Jawaban: **B**
2. Contoh *stopwords* Bahasa Indonesia adalah...
A. bagus B. enak C. dan D. cepat
Jawaban: **C**
3. Hasil *stemming* yang tepat untuk “makanan” biasanya adalah...
A. makan B. makanan C. pemakan D. dimakan
Jawaban: **A**
4. Tujuan utama *stopwords removal* adalah...
A. memperbanyak kata B. membuang kata kurang makna
C. menambah *emoji* D. menambah tanda baca
Jawaban: **B**
5. Dalam contoh "Saya suka makan bakso", setelah *tokenization* hasil yang paling masuk akal adalah...
A. ['Saya suka makan bakso']
B. ['Saya', 'suka', 'makan', 'bakso']
C. ['Saya_suka_makan_bakso']
D. ['Saya', 'suka makan', 'bakso']

Jawaban: **B**

6. Untuk Bahasa Indonesia, *stemming* di Python yang umum dipakai adalah...
A. spaCy-ID B. Sastrawi C. Gensim D. FastText

Jawaban: **B**

7. Kalimat "Aku tidak terlalu suka film itu" setelah *stopwords removal* kemungkinan menjadi...

- A. ['aku', 'tidak', 'terlalu', 'suka', 'film', 'itu']
B. ['suka', 'film']
C. ['aku', 'suka', 'film']
D. ['tidak', 'suka', 'itu']

Jawaban: **B**

8. *Token* yang sering dianggap tidak perlu dipertahankan saat analisis frekuensi adalah...

- A. kata dasar B. tanda baca tunggal seperti “,” “.”
C. kata kunci domain D. nama produk

Jawaban: **B**

9. Manakah *pipeline* yang benar untuk *preprocessing* dasar?

- A. *Stemming* → *Tokenization* → *Stopwords*
B. *Stopwords* → *Stemming* → *Tokenization*
C. *Tokenization* → *Stopwords* → *Stemming*
D. *Tokenization* → *Stemming* → *Stopwords*

Jawaban: **C**

10. Potongan Python berikut melakukan apa?

```
text = "Produk ini sangat bagus!"  
tokens = text.lower().split()
```

- A. *Stemming* B. *Stopwords removal* C. *Tokenization* sederhana D. *POS tagging*

Jawaban: **C**

C. Isian Singkat (5 Soal)

1. Isi: Istilah untuk kata-kata umum seperti “dan”, “yang”, “di” adalah _____.

Jawaban: **stopwords**

2. Isi: Proses mengubah kata berimbuhan menjadi kata dasar disebut _____.

Jawaban: **stemming**

3. Isi: Langkah awal untuk memecah kalimat menjadi unit kecil disebut _____.

Jawaban: **tokenization**

4. Isi: Dalam Python, *library* open source yang umum untuk *stemming* Bahasa Indonesia adalah _____.
Jawaban: **Sastrawi**
5. Isi: Setelah *stopwords removal*, analisis fokus ke kata yang lebih _____.
Jawaban: **informatif**

D. Soal Explorasi

Soal 1:

Rancang *pipeline* mini untuk membersihkan 1 paragraf review kantin. Tugas: ambil Top-3 kata paling sering setelah bersih.

Kunci jawaban (contoh):

- Langkah:
 - Ubah ke huruf kecil → *tokenization* → buang *stopwords* → buang non-alfabet → *stemming* → hitung frekuensi → ambil Top-3.
- Pseudocode:

```
t = LOWER(paragraf)
tok = TOKENIZE(t)
core = [STEM(w) for w in tok if ALPHA(w) and w not in
STOP_ID]
freq = COUNT(core)
top3 = TOP_N(freq, 3)
PRINT top3
```

- Python contoh:

```
! pip install nltk Sastrawi
from collections import Counter
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

text = "Baksonya sangat enak, murah, dan cepat disajikan.
Enak sekali!"
stop_id =
{"dan", "yang", "di", "ke", "ini", "itu", "sangat", "sekali"}
stem = StemmerFactory().create_stemmer()
core = [stem.stem(w) for w in text.lower().split() if
w.isalpha() and w not in stop_id]
print(Counter(core).most_common(3))
```

- Output:

```
[('bakso', 1), ('cepat', 1), ('enak', 1)]
```

Soal2:

Bandingkan hasil *stemming* dengan dan tanpa normalisasi *slang*. Tugas: normalisasi “bgt, gpp, ga” → bentuk baku, lalu *stemming*; tunjukkan beda 3 kata kunci teratas.

Kunci jawaban (contoh):

- Langkah:
 - Buat **slang_map** → ganti kata → jalankan *pipeline* seperti D1 → bandingkan **Top-3** sebelum vs sesudah.

- Pseudocode:

```
before = TOP3(PREPROCESS(chats))
after  = TOP3(PREPROCESS(NORMALIZE(chats, slang_map)))
COMPARE(before, after)
```

- Python contoh:

```
!pip install nltk Sastrawi
from collections import Counter
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

# Slang dictionary
slang = {"bgt": "banget", "gpp": "tidak", "ga": "tidak"}

# Normalization function for slang
def norm(s):
    return " ".join([slang.get(w, w) for w in
s.lower().split()])

# Input text
text = "Baksonya sangat enak, murah, dan cepat disajikan.
Enak sekali!"

# Stopwords
stop_id = {"dan", "yang", "di", "ke", "ini", "itu",
"sangat", "sekali"}

# Stemmer
stem = StemmerFactory().create_stemmer()

# Step 1: Before processing (raw tokens, only lowercase
and split)
before_tokens = [w for w in text.lower().split() if
w.isalpha()]
before_top3 = Counter(before_tokens).most_common(3)

# Step 2: After processing (normalize slang, remove
stopwords, apply stemming)
normalized_text = norm(text) # Normalize slang
after_tokens = [
    stem.stem(w) for w in normalized_text.lower().split()]
```

```

        if w.isalpha() and w not in stop_id
    ]
    after_top3 = Counter(after_tokens).most_common(3)

    # Step 3: Print comparison
    print("Before processing (Top-3 words):")
    for word, count in before_top3:
        print(f"{word}: {count}")

    print("\nAfter processing (Top-3 words):")
    for word, count in after_top3:
        print(f"{word}: {count}")

```

- Output:

```

Before processing (Top-3 words):
baksonya: 1
sangat: 1
dan: 1

After processing (Top-3 words):
bakso: 1
cepat: 1
enak: 1

```

Soal 3:

Buat fungsi cek *intent* “izin/ijin” pada kalimat siswa. Jika terdeteksi, kembalikan “izin”, jika tidak “lainnya”. Uji pada 3 kalimat.

Kunci jawaban (contoh):

- Langkah:
 - *tokenization* → *stopwords removal* → *stemming* → cek kata **izin** atau **ijin**.

- Pseudocode:

```

function INTENT(s):
    t = STEM(NOSTOP(TOKENIZE(LOWER(s))))
    if "izin" in t or "ijin" in t: return "izin"
    else: return "lainnya"

```

- Python contoh:

```

def intent(s):
    t = [w.strip(".,!") for w in s.lower().split()]
    return "izin" if ("izin" in t or "ijin" in t) else "lainnya"

print([intent(x) for x in [
    "Bu, izin ke UKS.",
    "Boleh ambil air?",
    "Ijin keluar kelas sebentar ya."
]])

```

- Output:

```
['izin', 'lainnya', 'izin']
```

Bonus Kode Mini (opsional latihan cepat)

```
# Tokenization + very simple stopwords (demo)
text = "Saya suka makan bakso di kantin sekolah."
stop_id = {"di", "dan", "yang", "ke", "saya"}
tokens = [w.lower().strip(".", "!") for w in text.split()]
no_stop = [w for w in tokens if w not in stop_id]
print(no_stop) # jawaban contoh:
['suka', 'makan', 'bakso', 'kantin', 'sekolah']
```

Selamat berlatih! Gunakan *open source* (Python, Pandas, NLTK, Sastrawi, Keras) dan jaga *pipeline* rapi: **tokenization** → **stopwords** → **stemming**.

Tugas Proyek

Proyek 1: Kamus Kata Dasar Sekolah (*Stemming Override*)

Ide: Sastrawi bagus, tapi kadang *stemming* butuh pengecualian. Buat **kamus koreksi** untuk kata yang sering salah *stem* di konteks sekolah.

Tujuan

- Membangun *pipeline tokenization* → *stopwords* → *stemming*.
- Menambahkan **kamus override** setelah *stemming*.

Data

- 15–20 kalimat dari pengumuman sekolah/OSIS.
- Contoh kata rawan: “*berangkat*”, “*bermain*”, “*pelajaran*”, “*ujian*”.

Langkah

- Bersihkan teks.
- *Tokenisasi*, hapus *stopwords*, *stemming* (Sastrawi).
- Terapkan **kamus_override**: perbaiki hasil *stem* tertentu.
- Bandingkan sebelum vs sesudah koreksi.

Pseudocode

```
texts = LOAD_TEXTS()
stop = LOAD_STOPWORDS_ID()
```



```

stemmer = INIT_SASTRAWI()
override = {"pelajaran": "ajar", "ujian": "uji", "bermain": "main"}

for t in texts:
    tok = TOKENIZE(LOWER(t))
    no_stop = [w for w in tok if ALPHA(w) and w not in stop]
    stems = [STEM_ID(w) for w in no_stop]
    fixed = [override.get(s, s) for s in stems]
    PRINT original=t, stems=stems, fixed=fixed

```

Python

```

!pip install Sastrawi
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import nltk; nltk.download('punkt_tab');
nltk.download('stopwords')

texts = ["Besok ada ujian pelajaran matematika.",
        "Siswa bermain futsal setelah pelajaran."]
stop_id = set(stopwords.words('indonesian'))
stemmer = StemmerFactory().create_stemmer()
override = {"pelajaran": "ajar", "ujian": "uji", "bermain": "main"}

for t in texts:
    tok = word_tokenize(t.lower())
    no_stop = [w for w in tok if w.isalpha() and w not in
stop_id]
    stems = [stemmer.stem(w) for w in no_stop]
    fixed = [override.get(s, s) for s in stems]
    print(t, "=>", stems, "=>", fixed)

```

Output

```

Besok ada ujian pelajaran matematika. => ['besok', 'uji', 'ajar',
'matematika'] => ['besok', 'uji', 'ajar', 'matematika']
Siswa bermain futsal setelah pelajaran. => ['siswa', 'main',
'futsal', 'ajar'] => ['siswa', 'main', 'futsal', 'ajar']

```

Kunci Jawaban (ekspektasi)

- Sebelum: ['ujian', 'pelajar', 'matematika'] → Sesudah: ['uji', 'ajar', 'matematika']
 - Sebelum: ['siswa', 'main', 'futsal', 'ajar'] → Sesudah: ['siswa', 'main', 'futsal', 'ajar']
- Inti: Kamus override** membuat *stemming* lebih akurat untuk domain sekolah.

Proyek 2: Normalisasi Chat *A/ay* → Bersih

Ide: Banyak chat siswa pakai singkatan: “gpp”, “bgt”, “bodo amat”. Kita buat **normalizer**: ubah *slang* → bentuk baku, baru jalankan *pipeline* NLP.

Tujuan

- Membuat **kamus slang**.
- Menunjukkan dampak normalisasi terhadap kata kunci.

Data

- 20 chat pendek campur *slang*.

Langkah

- Buat `slang_map = {"gpp": "tidak apa apa", "bgt": "banget", "ga": "tidak"}`.
- Replace *slang* sebelum *tokenization*.
- Lanjut *stopwords* dan *stemming*.
- Bandingkan **Top-10 kata** sebelum vs sesudah normalisasi.

Pseudocode

```
slang_map = {...}
def normalize(s):
    for k,v in slang_map.items(): s = REPLACE_WORD(s, k, v)
    return s

cleaned = []
for msg in chats:
    msg_n = normalize(msg)
    tok = TOKENIZE(LOWER(msg_n))
    core = STEM(NOSTOP(tok))
    APPEND cleaned WITH core

TOP10 = TOP_FREQ(cleaned, 10)
COMPARE before_top10 vs after_top10
```

Python singkat

```
!pip install Sastrawi
from collections import Counter
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk; nltk.download('punkt'); nltk.download('stopwords')

chats = ["gpp kok telat bgt", "aku ga masuk hari ini", "bodo
amat, santai aja"]
slang = {"gpp": "tidak apa
apa", "bgt": "banget", "ga": "tidak", "gk": "tidak", "nggak": "tidak", "g":
:"tidak"}
stop_id = set(stopwords.words('indonesian'))
```

```

stem = StemmerFactory().create_stemmer()

def normalize(s):
    words = s.lower().split()
    return " ".join([slang.get(w, w) for w in words])

bag = []
for m in chats:
    m2 = normalize(m)
    tok = word_tokenize(m2)
    no = [w for w in tok if w.isalpha() and w not in stop_id]
    bag += [stem.stem(w) for w in no]

print(Counter(bag).most_common(10))

```

Output

```

[('telat', 1), ('banget', 1), ('masuk', 1), ('bodo', 1),
 ('santai', 1), ('aja', 1)]

```

Kunci Jawaban (ekspektasi)

- Kata “tidak” akan muncul lebih konsisten (karena *slang* dinormalisasi).
- **Top-10** lebih stabil, mudah dipakai untuk analisis sentimen/niat.

Proyek 3: Saran Kata Otomatis (Bigram Sederhana)

Ide: Bikin *auto-suggest* kata berikutnya dari korpus kecil pengumuman. Tanpa *model* berat. Hanya **hitung pasangan kata (bigram)**.

Tujuan

- Mengerti *tokenization* → *bigram count*.
- Memberi saran kata “setelah X biasanya Y”.

Data

- 30–50 judul/kalimat pengumuman singkat.

Langkah

- *Tokenisasi* tiap kalimat.
- Hitung frekuensi bigram (w_i, w_{i+1}).
- Input 1 kata → tampilkan 3 saran tertinggi.

Pseudocode

```

bigrams = COUNTER()
FOR sentence IN corpus:
    tok = TOKENIZE(LOWER(sentence))
    core = [w for w in tok if ALPHA(w)]

```

```

FOR i in range(len(core)-1):
    bigrams[(core[i], core[i+1])] += 1

def suggest(word):
    cand = [(w2,c) for (w1,w2),c in bigrams.items() if w1==word]
    return TOP_N_SORT(cand, 3)

```

Python

```

!pip install Sastrawi
from collections import Counter, defaultdict
from nltk.tokenize import word_tokenize
import nltk; nltk.download('punkt_tab')

corpus = ["Besok upacara bendera", "Besok ujian
matematika",
          "Hari ini latihan upacara", "Pengumuman ujian
susulan besok"]
bigrams = Counter()

for s in corpus:
    tok = [w for w in word_tokenize(s.lower()) if
w.isalpha()]
    for i in range(len(tok)-1):
        bigrams[(tok[i], tok[i+1])] += 1

def suggest(w):
    cand = [(w2,c) for (w1,w2),c in bigrams.items() if
w1==w]
    return sorted(cand, key=lambda x: -x[1])[:3]

print("Saran setelah 'besok':", suggest("besok"))

```

Output

```
Saran setelah 'besok': [('upacara', 1), ('ujian', 1)]
```

Kunci Jawaban (ekspektasi)

- **besok** → **[('upacara', ...), ('ujian', ...)]**
- *Auto-suggest* sederhana untuk editor pengumuman sekolah.

Proyek 4: A/B Stopwords Tuner: Mana Lebih Bagus?

Ide: Stopwords memengaruhi kata kunci. Uji **dua daftar stopwords**: *default* vs *custom* (tambah “*sangat*”, “*sekali*”, “*bange*”). Lihat bedanya **Top-10 kata**.

Tujuan

- Memahami dampak daftar *stopwords*.
- Memilih konfigurasi yang pas untuk domain.

Data

- 40 review kantin/perpustakaan.

Langkah

- Jalankan *pipeline* dua kali:
 - A) Stopwords default.
 - B) Stopwords default + kata tambahan.
- Bandingkan **Top-10** A vs B.
- Catat perbedaan.

Pseudocode

```
stop_A = DEFAULT_ID()
stop_B = stop_A ∪ {"sangat","sekali","banget"}

TOP_A = TOP10(PREPROCESS(corpus, stop_A))
TOP_B = TOP10(PREPROCESS(corpus, stop_B))

DIFF = SYMMETRIC_DIFF(TOP_A.words, TOP_B.words)
REPORT(TOP_A, TOP_B, DIFF)
```

Python singkat

```
from collections import Counter
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk; nltk.download('punkt'); nltk.download('stopwords')

corpus = ["Baksonya sangat enak sekali", "Tehnya enak banget",
          "Pelayanannya cepat sekali", "Tempatnya sangat nyaman"]
stop_A = set(stopwords.words('indonesian'))
stop_B = stop_A | {"sangat","sekali","banget"}

def top10(stoplist):
    bag=[]
    for s in corpus:
        tok=[w for w in word_tokenize(s.lower()) if w.isalpha()
and w not in stoplist]
        bag.extend(tok)
    return Counter(bag).most_common(10)

print("Top-10 A:", top10(stop_A))
print("Top-10 B:", top10(stop_B))
```

Output

```
Top-10 A: [('enak', 2), ('baksonya', 1), ('tehnya', 1),
('banget', 1), ('pelayanannya', 1), ('cepat', 1), ('tempatnya',
1), ('nyaman', 1)]
Top-10 B: [('enak', 2), ('baksonya', 1), ('tehnya', 1),
('pelayanannya', 1), ('cepat', 1), ('tempatnya', 1), ('nyaman',
1)]
```

Kunci Jawaban (ekspektasi)

- Di **A**, kata “sangat/sekali/banget” bisa mendominasi Top-10.
- Di **B**, kata *deskriptif* seperti “enak/cepat/nyaman” lebih menonjol.

Kesimpulan: **Stopwords custom** membantu fokus ke makna.

Proyek 5: Mini *Search Engine* Pengumuman Sekolah

Ide: Cari kalimat pengumuman paling cocok dengan *query*. Gunakan **skor kecocokan kata** (bukan *model* besar). *Simple but useful*.

Tujuan

- Membangun *retrieval* dasar.
- Mengerti pentingnya *preprocessing* sebelum pencarian.

Data

- File **.txt** berisi 30–50 pengumuman.

Langkah

- *Preprocess* semua kalimat (token → no-stop → stem).
- *Preprocess query* juga.
- Skor = jumlah kata *query* yang muncul di kalimat.
- Kembalikan 3 kalimat terbaik.

Pseudocode

```
docs = LOAD_SENTENCES(file)
index = []
FOR s IN docs:
    core = STEM(NOSTOP(TOKENIZE(LOWER(s))))
    index.append((s, core))

def search(q):
    qcore = STEM(NOSTOP(TOKENIZE(LOWER(q))))
    scored = []
    FOR (orig, core) IN index:
        score = COUNT(w in core where w in qcore)
        scored.append((score, orig))
    RETURN TOP_N_SORT(scored, 3)
```

Python singkat

```
!pip install Sastrawi
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk; nltk.download('punkt'); nltk.download('stopwords')

docs = ["Besok ada upacara bendera pukul 07.00",
        "Pengumuman lomba desain poster digital",
        "Ujian susulan matematika hari Jumat",
        "Rapat orang tua murid pekan depan"]

stop_id = set(stopwords.words('indonesian'))
stemmer = StemmerFactory().create_stemmer()
index = []
for s in docs:
    tok = word_tokenize(s.lower())
    core = [stemmer.stem(w) for w in tok if w.isalpha() and w not
in stop_id]
    index.append((s, core))

def search(q):
    tq = [stemmer.stem(w) for w in word_tokenize(q.lower())
          if w.isalpha() and w not in stop_id]
    scored = [(sum(1 for w in core if w in tq), orig) for (orig,
core) in index]
    return sorted(scored, key=lambda x: -x[0])[:3]

print(search("jadwal ujian matematika"))
```

Output

```
[(2, 'Ujian susulan matematika hari Jumat'), (0, 'Besok ada
upacara bendera pukul 07.00'), (0, 'Pengumuman lomba desain
poster digital')]
```

Kunci Jawaban (ekspektasi)

- Hasil top mengandung kata terkait “ujian” dan “matematika”, misal:
`[(2, "Ujian susulan matematika hari Jumat"), ...]`
Poin penting: *Tokenization/stopwords/stemming* meningkatkan kecocokan.

Catatan Akhir (opsional untuk nilai bonus)

- Simpan korpus & hasil ke **CSV** lalu analisis dengan **Pandas**.
- Dokumentasikan di **Jupyter Notebook**.
- Jika mau *level up*: latih *neural network Keras* kecil untuk klasifikasi topik/sentimen setelah *preprocessing*.

BAB 4: Analisis Teks dengan Python (Analisis Sentimen Sederhana)

Pernah nggak sih kalian baca komentar di media sosial, lalu langsung tahu kalau komentar itu isinya marah, sedih, atau senang? Nah, kalian baru saja melakukan **analisis sentimen** secara otomatis! Di bab ini, kita akan ngajarin komputer buat punya kemampuan kayak gitu. Kita bakal pakai **Python** untuk menganalisis teks dan mengukur perasaan di baliknya. Ini skill yang super keren buat kalian yang mau jadi Content Creator, Digital Marketer, atau Social Media Manager!

Tujuan Pembelajaran

- Memahami apa itu **analisis sentimen**.
- Melakukan analisis sentimen sederhana pada teks menggunakan Python.
- Mengidentifikasi apakah sebuah teks punya sentimen **positif**, **negatif**, atau **netral**.
- Menghubungkan analisis sentimen dengan contoh-contoh di dunia nyata.

Peta Konsep Sederhana

Bayangin kita mau mencicipi rasa makanan.

- **Makanan Mentah (Teks Mentah)**: Ini review dari pelanggan, misalnya, "Ayamnya enak banget!" atau "Pelayanannya lama."
- **Indera Perasa (Algoritma Analisis Sentimen)**: Ini adalah tool kita yang akan "merasakan" kata-kata.
- **Rasa (Sentimen)**: Hasilnya bisa **positif** (enak), **negatif** (tidak enak), atau **netral** (biasa saja).

Apersepsi: Kamu Mau Tahu Kopi Mana yang Paling Disukai!

Kamu mau buka warung kopi dan kamu mau tahu, rasa kopi apa yang paling disukai teman-temanmu. Kamu suruh mereka kirim ulasan lewat chat. Ada yang bilang, "Kopi susu kamu enak banget!" tapi ada juga yang bilang, "Kopi hitamnya hambar, mending yang lain."

Pusing kan kalau harus baca satu-satu? Nah, dengan **analisis sentimen**, kamu bisa otomatis memproses ratusan, bahkan ribuan, ulasan dan tahu mana kopi yang paling banyak dapat ulasan positif. Gampang banget!

Penjelasan Teori

Kalian pasti sering baca komentar di Instagram, TikTok, atau YouTube. Kadang komentarnya bikin senyum: “*Videonya keren banget, aku suka!*” Kadang bikin kesal: “*Pelayanannya lama banget, nggak rekomen deh.*” Tanpa sadar, kalian sebenarnya sedang melakukan **analisis sentimen**.

Nah, di bab ini kita bakal ngajarin komputer buat bisa melakukan hal yang sama. Kita pakai **Python** (bahasa pemrograman open source yang populer banget).

Apa Itu Analisis Sentimen?

Analisis sentimen itu gampangnya adalah **membaca perasaan dari teks**. Komputer kita ajarkan untuk bisa tahu apakah sebuah kalimat itu:

- **Positif** → senang, suka, bahagia.
- **Negatif** → marah, kecewa, kesal.
- **Netral** → biasa aja, datar, tanpa emosi.

Analogi yang Seru

Bayangin kalian lagi jadi **tester makanan**:

- **Makanan mentah = teks mentah** (review, komentar, ulasan).
- **Lidah = algoritma Python** yang bisa “merasakan” kata-kata.
- **Rasa = sentimen** (positif, negatif, netral).

Kalau kata-katanya enak, mantap, keren → komputer bilang *positif*.
Kalau kata-katanya jelek, kecewa, buruk → komputer bilang *negatif*.
Kalau biasa saja → komputer bilang *netral*.

Cara Kerja Paling Sederhana

1. Siapkan **daftar kata positif**: enak, bagus, mantap, suka, keren.
2. Siapkan **daftar kata negatif**: jelek, lama, kecewa, hambar, buruk.
3. Komputer akan menghitung:
 - Berapa kata positif di teks?
 - Berapa kata negatif di teks?
4. Tentukan hasil:
 - Positif > Negatif → *sentimen positif*.
 - Negatif > Positif → *sentimen negatif*.
 - Sama / kosong → *sentimen netral*.

Contoh Kode Python Sederhana

```
kata_positif = ['enak', 'bagus', 'keren', 'mantap', 'suka']  
kata_negatif = ['jelek', 'buruk', 'lama', 'kecewa', 'hambar']
```

```
def cek_sentimen(teks):
    teks = teks.lower()
    skor_pos = sum(1 for kata in kata_positif if kata in teks)
    skor_neg = sum(1 for kata in kata_negatif if kata in teks)
    if skor_pos > skor_neg:
        return "Positif"
    elif skor_neg > skor_pos:
        return "Negatif"
    else:
        return "Netral"

print(cek_sentimen("Makanannya enak tapi pelayanannya lama"))
```

Output: **Netral**, karena ada kata positif *enak* dan kata negatif *lama*.

Kenapa Penting?

- Dunia digital penuh teks: komentar, review, postingan.
- Bisnis butuh tahu, apakah orang suka atau tidak.
- *Content creator* bisa ngerti reaksi penonton.
- *Machine Learning* dan *Neural Network* bisa bikin analisis lebih pintar, nggak cuma daftar kata.

Kasus Ilustrasi

Fungsi Inti (dipakai di semua kasus)

Bayangkan ini “lidah rasa” kita. Sederhana. Mudah diubah.
pseudocode:

```
DAFTAR_POSITIF =
['enak', 'bagus', 'mantap', 'keren', 'suka', 'terbaik']
DAFTAR_NEGATIF =
['jelek', 'buruk', 'lama', 'kecewa', 'hambar', 'parah']

FUNGSI CekSentimen(teks):
    t = LOWERCASE(teks)
    skor_pos = JUMLAH(kata ∈ DAFTAR_POSITIF yang MUNCUL di t)
    skor_neg = JUMLAH(kata ∈ DAFTAR_NEGATIF yang MUNCUL di t)
    JIKA skor_pos > skor_neg: KEMBALIKAN "positif"
    JIKA skor_neg > skor_pos: KEMBALIKAN "negatif"
    SELAIN ITU: KEMBALIKAN "netral"
```

Implementasi nyatanya bisa pakai *Python + Pandas*. Nanti tinggal ganti daftar kata sesuai konteks (game, kopi, baju, dsb).

Kasus 1: Review Game di Play Store

Ilustrasi:

Kamu rilis game sekolah. Banyak komentar: ada yang muji grafik, ada yang protes lag. Kamu ingin ringkas: berapa % positif/negatif?

Tujuan:

Kelompokkan ulasan → hitung tren sentimen → kasih saran perbaikan.

Data:

File CSV `ulasan_game.csv` (kolom: `teks`, `rating`).

Langkah singkat:

- *Preprocessing*: lowercase, buang emoji/symbol.
- Pakai `CekSentimen(teks)` untuk tiap baris.
- Hitung persentase per sentimen.
- Cocokkan dengan `rating` untuk validasi.

pseudocode:

```
DATA = BACA_CSV("ulasan_game.csv")
UNTUK setiap baris d ∈ DATA:
    d.sentimen = CekSentimen(d.teks)

ringkasan = HITUNG_PERSENTASE(DATA.sentimen)
CETAK(ringkasan)

SARAN:
    JIKA NEGATIF dominan karena kata 'lag' atau 'bug':
        PRIORITAS = "Optimasi performa & perbaiki bug"
    JIKA POSITIF tinggi tapi rating turun:
        CEK "fitur tertentu" dari komentar
```

Upgrade open source:

- *Pandas* untuk baca/olah CSV.
- *scikit-learn* untuk model *baseline* (*Naive Bayes*).
- *Keras* + *Neural Network* untuk level lanjut.

Kasus 2: Event Sekolah (Pensi) di Instagram

Ilustrasi:

Sekolah bikin pensi. Komentar IG rame. Kamu pengin tahu: sound system dikeluhkan? MC disukai?

Tujuan:

Kelompokkan komentar per tema (sound, MC, venue) + sentimennya.

Data:

Daftar komentar `komentar_ig.txt`.

Langkah singkat:

- *Keyword mapping* per tema: `sound`, `mc`, `venue`.
- Tentukan sentimen per komentar.
- Buat ringkasan per tema.

pseudocode:

```
TEMA = {
    "sound": ['sound', 'speaker', 'audio', 'bising'],
    "mc": ['mc', 'pembawa acara', 'host'],
    "venue": ['panggung', 'tempat', 'panas', 'sempit']
}

KOMENTAR = BACA_DAFTAR("komentar_ig.txt")

HASIL = {tema: {pos:0, neg:0, net:0} untuk setiap tema}
UNTUK k ∈ KOMENTAR:
    s = CekSentimen(k)
    daftar_tema_kena = [t untuk t jika ADA_KATA(TEMA[t],
k.lower())]
    JIKA daftar_tema_kena KOSONG: LANJUT
    UNTUK t ∈ daftar_tema_kena:
        HASIL[t][s] += 1

CETAK(HASIL)
REKOMENDASI:
    JIKA HASIL["sound"].neg > HASIL["sound"].pos:
        SARAN = "Sewa mixer + cek rehearsal"
```

Upgrade open source:

- *NLTK/spaCy* untuk *tokenization/stopwords*.
- *Pandas* untuk pivot ringkasan per tema.

Kasus 3: Warung Kopi Sekolah: Menu Favorit**Ilustrasi:**

Kantin bingung. Kopi susu atau kopi hitam yang paling disukai? Kumpulkan 50 komentar.

Tujuan:

Tentukan menu favorit berdasarkan sentimen.

Data:

CSV `ulasan_kopi.csv` (kolom: `menu`, `teks`).

Langkah singkat:

- Sentimen per ulasan.
- Agregasi per **menu**.
- Skor akhir = **positif** - **negatif**.

pseudocode:

```
DATA = BACA_CSV("ulasan_kopi.csv")

SKOR = kamus default 0
UNTUK d ∈ DATA:
    s = CekSentimen(d.teks)
    JIKA s == "positif": SKOR[d.menu] += 1
    JIKA s == "negatif": SKOR[d.menu] -= 1

menu_terbaik = ARGMAX(SKOR)
CETAK("Menu favorit:", menu_terbaik)
```

Upgrade open source:

- *Matplotlib* untuk grafik batang skor menu.
- *Keras* untuk model *embedding* sederhana bila kosa kata gaul banyak.

Kasus 4: Monitoring Merek Pakaian Lokal

Ilustrasi:

Kamu punya brand kaos. Penasaran respon netizen terhadap *drop* desain baru.

Tujuan:

Pantau sentimen mingguan. Tahu cepat kalau ada krisis.

Data:

Korpus komentar mingguan **minggu_1.txt**, **minggu_2.txt**, ...

Langkah singkat:

- Hitung sentimen per minggu.
- Bandingkan tren (naik/turun).
- Alarm kalau negatif melonjak.

pseudocode:

```
FILE_MINGGU = ['m1.txt', 'm2.txt', 'm3.txt', 'm4.txt']
TREN = []

UNTUK f ∈ FILE_MINGGU:
    K = BACA_DAFTAR(f)
    hit = {pos:0, neg:0, net:0}
    UNTUK k ∈ K:
        s = CekSentimen(k)
        hit[s] += 1
    skor = hit.pos - hit.neg
```

```

TREN.TAMBAH(skor)

CETAK("Tren skor:", TREN)

JIKA NAIK_TAJAM_NEGATIF(TREN):
    CETAK("ALERT: Respon negatif naik. Siapkan klarifikasi &
    perbaikan.")

```

Upgrade open source:

- *Pandas* time-series untuk tren.
- *scikit-learn* + *TF-IDF* untuk hasil lebih stabil.

Kasus 5: Bot Customer Service Otomatis

Ilustrasi:

Toko online sekolah butuh bot. Kalau pelanggan marah, bot minta maaf dan lempar ke admin.

Tujuan:

Deteksi sentimen real-time dari input pengguna.

Data:

Teks masuk dari chat.

Alur:

- Terima kalimat.
- Tentukan sentimen.
- Pilih respon otomatis.

pseudocode:

```

LOOP:
    teks = AMBIL_INPUT_PENGGUNA()
    s = CekSentimen(teks)

    JIKA s == "negatif":
        KIRIM("Maaf ya. Admin akan bantu segera.")
        BUAT_TIKET(teks, prioritas="tinggi")
    JIKA s == "positif":
        KIRIM("Terima kasih! Senang bisa membantu.")
    JIKA s == "netral":
        KIRIM("Baik. Ada yang bisa kami bantu lagi?")

```

Upgrade open source:

- Jalankan lokal via *Ollama* + *OpenWebUI* untuk *LLM offline*.
- Latih *Neural Network* kecil di *Keras* (label: positif/negatif/netral) jika butuh akurasi lebih.

Catatan Praktik Baik (*Best Practice*)

- Selalu lakukan *preprocessing*: *lowercase*, hapus emoji/symbol, normalisasi kata gaul.
- Buat *kamus gaul* SMK: tambah kata seperti “*gaskeun*” (positif), “*zonk*” (negatif).
- Uji dengan *dataset* kecil dulu, lalu tambah data.
- Simpan hasil di CSV dengan *Pandas* biar gampang dievaluasi.
- Untuk level lanjut, coba *Neural Network* dengan *Keras* (input *TF-IDF* atau *embedding*).

Mini Checklist Implementasi (Open Source)

- *Python 3.x*
- *Pandas* (olah CSV)
- *NLTK/spaCy* (opsional *tokenization/stopwords*)
- *scikit-learn* (baseline *TF-IDF* + *Naive Bayes*)
- *Keras* + *Neural Network* (lanjutan)
- *Matplotlib* (grafik)
- *Ollama* + *OpenWebUI* (AI lokal)

Soal Latihan & Pembahasan

Fungsi bantu (dipakai berulang)

pseudocode (inti logika)

```
POS = ['enak', 'bagus', 'mantap', 'keren', 'suka', 'terbaik']
NEG = ['jelek', 'buruk', 'lama', 'kecewa', 'hambar', 'parah']

FUNGSI CekSentimen(teks):
    t = LOWERCASE(teks)
    skor_pos = JUMLAH(kata ∈ POS yang MUNCUL di t)
    skor_neg = JUMLAH(kata ∈ NEG yang MUNCUL di t)
    JIKA skor_pos > skor_neg: KEMBALIKAN "positif"
    JIKA skor_neg > skor_pos: KEMBALIKAN "negatif"
    SELAIN ITU: KEMBALIKAN "netral"
```

Soal 1: Campuran Positif & Negatif (Netral tipis)

Teks: “Makanannya enak, tapi pelayanannya lama.”

Pseudocode:

```
t = "Makanannya enak, tapi pelayanannya lama."
hasil = CekSentimen(t)
CETAK(hasil)
```


Jawaban: Netral

Pembahasan (singkat & jelas):

- Ditemukan **1** kata positif (*enak*).
- Ditemukan **1** kata negatif (*lama*).
- Skor seimbang → sistem dasar mengembalikan **netral**.

Catatan: Di *model lanjut (machine learning)*, bisa diberi bobot konteks “tapi” agar condong ke sisi kedua kalimat.

Soal 2: Negatif Kuat

Teks: “Game ini jelek dan bikin kecewa.”

Pseudocode:

```
t = "Game ini jelek dan bikin kecewa."
hasil = CekSentimen(t)    // jelek ∈ NEG, kecewa ∈ NEG
```

Jawaban: Negatif

Pembahasan:

- Kata negatif: *jelek, kecewa* → **2 poin negatif**.
- Tidak ada kata positif.
- Skor negatif > positif → **negatif**.

Untuk implementasi nyata di *Python*, kamu bisa pakai *Pandas* untuk memproses banyak baris sekaligus.

Soal 3: Netral (Informasi tanpa emosi)

Teks: “Paket sudah datang.”

Pseudocode:

```
t = "Paket sudah datang."
hasil = CekSentimen(t)    // tak ada kata di POS/NEG
```

Jawaban: Netral

Pembahasan:

- Tidak ada kata pada daftar positif/negatif.
- Kalimat hanya memberi informasi.
- Hasil **netral**.

Di *project* nyata, tambahkan *preprocessing* (hapus emoji/symbol). Ini *best practice* dalam *NLP*.

Soal 4: Pengaruh *Preprocessing* (Normalisasi)

Teks asli: "Layanannya lamaaaaa. Aku kecewaaa."

Teks setelah normalisasi: "Layanannya lama. Aku kecewa."

Pseudocode:

```
FUNGSI Normalisasi (teks):  
    t = LOWERCASE (teks)  
    t = GANTI_REPEAT_CHAR(t, max=2)    // lamaaaaa -> lama,  
    kecewaaa -> kecewa  
    t = HILANGKAN_SYMBOL_EMOJI(t)  
    KEMBALIKAN t  
  
t_asli = "Layanannya lamaaaaa. Aku kecewaaa."  
t_norm = Normalisasi(t_asli)  
hasil = CekSentimen(t_norm)
```

Jawaban: Negatif

Pembahasan:

- Tanpa normalisasi, kata "lamaaaaa/kecewaaa" bisa **tidak cocok** dengan daftar kata.
- Setelah normalisasi → cocok dengan **lama** dan **kecewa** (2 negatif).
- Hasil **negatif**.

Inilah kenapa *preprocessing* penting di *machine learning* dan *neural network*.

Soal 5: Kamus Tidak Lengkap (Perlu *domain slang*)

Teks: "Rasanya **nendang!** Kopinya **mantul!**"

(anggap daftar POS belum punya "**nendang**" dan "**mantul**")

Pseudocode:

```
POS = ['enak', 'bagus', 'mantap', 'keren', 'suka', 'terbaik']    //  
belum ada 'nendang', 'mantul'  
  
t = "Rasanya nendang! Kopinya mantul!"  
hasil_awal = CekSentimen(t)    // mungkin "netral" karena tak  
cocok  
// Perbaiki kamus:  
TAMBAH(POS, ['nendang', 'mantul'])  
hasil_perbaikan = CekSentimen(t)    // kini terdeteksi positif
```

Jawaban:

- **Sebelum perbaikan kamus:** Netral (kata gaul tidak dikenali).
- **Sesudah perbaikan kamus:** Positif.

Pembahasan:

- *Word list* harus **relevan dengan konteks** (bahasa gaul SMK).
- Tambahkan *slang* ke kamus agar deteksi lebih akurat.

- Untuk skala besar, pakai *scikit-learn* (TF-IDF + *Naive Bayes*) atau *Keras* + *Neural Network* supaya model belajar dari data, bukan hanya kamus.

Bonus Mini-Template Evaluasi (opsional buat kelas)

- Buat CSV komentar → `komentar.csv` (kolom: `teks`).
- Pakai *Pandas* untuk baca dan jalankan `CekSentimen` ke tiap baris.
- Ringkas pakai *groupby* dan tampilkan grafik batang dengan *Matplotlib*.
- Lanjut tingkatkan akurasi dengan *TF-IDF* + *Naive Bayes* (di *scikit-learn*).
- *Lanjutan: fine-tune* model sederhana di *Keras* untuk 3 kelas (positif/negatif/netral).

Fun Facts!

- **Ollama** dan **OpenWebUI** itu bisa kalian pakai buat bikin `chatbot` yang jago analisis sentimen, lho! Kalian bisa kasih dia teks dan minta dia bilang, "Teks ini sentimennya negatif."
- Analisis sentimen dipakai untuk memprediksi hasil pemilu! Para ahli mengumpulkan miliaran tweet dan menganalisis sentimennya terhadap kandidat politik.
- **RAG lokal** bisa kamu pakai untuk membuat **daftar kata positif/negatif** yang sangat spesifik, misalnya khusus untuk bahasa gaul anak SMK.

Tips Belajar atau Menghafal Cepat

- **Buat Daftar Sendiri:** Buat daftar kata-kata positif dan negatif yang sering kamu pakai sehari-hari.
- **Latihan dengan Teks Pendek:** Mulai dari kalimat pendek, lalu coba analisis paragraf.
- **Cari Data Asli:** Cari ulasan produk atau komentar di YouTube dan coba analisis manual.

Aktivitas Siswa

Aktivitas 1 — “Lidah Sentimen” Sederhana

Tujuan: Membuat fungsi cek sentimen berbasis *word list*.

Ide: Hitung kata positif vs negatif. Hasil: *positif/negatif/netral*.

Langkah:

1. Siapkan dua daftar kata: positif dan negatif.
2. Ubah teks ke *lowercase*.
3. Hitung kemunculan kata dari dua daftar.

4. Bandingkan skor.
5. Cetak hasil.

Pseudocode:

```

POS = ['enak', 'bagus', 'mantap', 'keren', 'suka', 'terbaik']
NEG = ['jelek', 'buruk', 'lama', 'kecewa', 'hambar', 'parah']

FUNGSI CekSentimen(teks):
    t = LOWERCASE(teks)
    skor_pos = JUMLAH(kata ∈ POS yang ada di t)
    skor_neg = JUMLAH(kata ∈ NEG yang ada di t)
    JIKA skor_pos > skor_neg: kembalikan "positif"
    JIKA skor_neg > skor_pos: kembalikan "negatif"
    SELAIN ITU: kembalikan "netral"

```

Python:

```

POS = ['enak', 'bagus', 'mantap', 'keren', 'suka', 'terbaik']
NEG = ['jelek', 'buruk', 'lama', 'kecewa', 'hambar', 'parah']

def cek_sentimen(teks: str) -> str:
    t = teks.lower()
    skor_pos = sum(1 for k in POS if k in t)
    skor_neg = sum(1 for k in NEG if k in t)
    if skor_pos > skor_neg: return "positif"
    if skor_neg > skor_pos: return "negatif"
    return "netral"

contoh = [
    "Makanannya enak tapi datangnya lama",
    "Layanannya buruk, aku kecewa",
    "Produknya sudah sampai"
]
for c in contoh:
    print(c, "=>", cek_sentimen(c))

```

Output:

```

Makanannya enak tapi datangnya lama => netral
Layanannya buruk, aku kecewa => negatif
Produknya sudah sampai => netral

```

Jawaban (ekspektasi):

- “Makanannya enak tapi datangnya lama” → **netral** (imbang).
- “Layanannya buruk, aku kecewa” → **negatif**.
- “Produknya sudah sampai” → **netral**.

Aktivitas 2: Analisis Banyak Komentar pakai *Pandas*

Tujuan: Memproses *dataset* kecil sekaligus. Tampilkan ringkasan.

Ide: Simulasikan 8 komentar. Hitung persentase sentimen.

Langkah:

1. Buat *DataFrame* berisi kolom **teks**.
2. Terapkan **cek_sentimen** ke setiap baris.
3. Hitung jumlah/proporsi per kelas.
4. Cetak ringkasan.

Pseudocode:

```
DATA = daftar komentar
UNTUK setiap baris:
    sentimen = CekSentimen(baris.teks)
HITUNG jumlah per sentimen
HITUNG persen per sentimen
CETAK tabel ringkasan
```

Python:

```
import pandas as pd

komentar = [
    "Grafiknya bagus banget",          # +
    "Kontrolnya jelek dan bikin kecewa", # -
    "Sound-nya mantap, suka!",          # +
    "Terlalu lama loading",             # -
    "Biasa saja",                       # netral
    "Keren maksimal",                   # +
    "Parah, banyak bug dan buruk",      # -
    "Cerita game-nya enak diikuti"      # +
]
df = pd.DataFrame({"teks": komentar})
df["sentimen"] = df["teks"].apply(cek_sentimen)

ringkas =
df["sentimen"].value_counts().rename_axis("sentimen").to_frame("j
umlah")
ringkas["persen"] = (ringkas["jumlah"] / len(df) * 100).round(1)
print(df)
print("\nRingkasan:")
print(ringkas)
```

Output:

```
          teks  sentimen
0  Grafiknya bagus banget  positif
1  Kontrolnya jelek dan bikin kecewa  negatif
2  Sound-nya mantap, suka!  positif
```

3	Terlalu lama loading	negatif
4	Biasa saja	netral
5	Keren maksimal	positif
6	Parah, banyak bug dan buruk	negatif
7	Cerita game-nya enak diikuti	positif

Ringkasan:

	jumlah	persen
sentimen		
positif	4	50.0
negatif	3	37.5
netral	1	12.5

Jawaban (ekspektasi ringkasan):

- $positif \approx 4$, $negatif \approx 3$, $netral \approx 1 \rightarrow positif$ dominan.

Aktivitas 3: *Preprocessing*: Normalkan Teks “Gaul”

Tujuan: Menunjukkan kenapa *preprocessing* itu penting.

Ide: Ubah “lamaaaaa” $\rightarrow$ “lama”. Hapus simbol/emoji. Tetap sederhana.

Langkah:

1. Buat fungsi `normalisasi()`:
 - *lowercase*.
 - potong huruf berulang (maks 2).
 - hapus simbol sederhana (titik/komma/!/?).
2. Bandingkan hasil sentimen sebelum vs sesudah normalisasi.

Pseudocode:

```

FUNGSI Normalisasi(t):
    t = lowercase(t)
    t = potong_karakter_berulang(t, max=2)
    t = hapus_simbol(t)
    kembalikan t
HASIL_AWAL = CekSentimen(teks_asli)
HASIL_SETELAH = CekSentimen(Normalisasi(teks_asli))
  
```

Python:

```

import re

def normalisasi(t: str) -> str:
    t = t.lower()
    # potong huruf berulang, contoh: "lamaaaaa" ->
    "lamaa" (maks 2)
    t = re.sub(r'(\.){2,}', r'\1', t)
    # hapus simbol sederhana
    t = re.sub(r'^a-z0-9\s', ' ', t)
  
```

```

t = re.sub(r'\s+', ' ', t).strip()
return t

teks_asli = "Layanannya lamaaaaa!! Aku kecewaaa :("
print("Asli      :", teks_asli, "=>",
      cek_sentimen(teks_asli))
t_norm = normalisasi(teks_asli)
print("Normal    :", t_norm, "=>", cek_sentimen(t_norm))

```

Output:

```

Asli      : Layanannya lamaaaaa!! Aku kecewaaa :( => negatif
Normal    : layananannya lamaa aku kecewaa => negatif

```

Jawaban (ekspektasi):

- Sebelum normalisasi: bisa **netral** (kata tak cocok kamus).
- Sesudah normalisasi: **negatif** (kata “lama”, “kecewa” terdeteksi).

Aktivitas 4: *Aspect-Based* Mini: Tema “Sound/MC/Venue”

Tujuan: Mengelompokkan sentimen per aspek (tema).

Ide: Komentar bisa bicara *sound*, *MC*, atau *venue* sekaligus.

Langkah:

1. Definisikan kamus tema:
 - **sound**: “sound, speaker, audio, bising”
 - **mc**: “mc, host, pembawa acara”
 - **venue**: “panggung, panas, sempit, tempat”
2. Untuk tiap komentar:
 - Tentukan sentimen.
 - Cek aspek mana yang muncul.
 - Tambah hitungan di aspek tersebut.
3. Cetak tabel ringkas per aspek.

Pseudocode:

```

TEMA = {sound:[...], mc:[...], venue:[...]}
HITUNG = {tema:{pos:0, neg:0, net:0}}

UNTUK komentar:
    s = CekSentimen(komentar)
    daftar_aspek = aspek yang ditemukan dalam komentar
    UNTUK a ∈ daftar_aspek:
        HITUNG[a][s] += 1
CETAK HITUNG

```

Python:

```
TEMA = {
    "sound": ['sound', 'speaker', 'audio', 'bising'],
    "mc":     ['mc', 'host', 'pembawa acara'],
    "venue":  ['panggung', 'tempat', 'panas', 'sempit']
}

komentar_aspek = [
    "Sound mantap tapi venue panas",
    "MC keren, pembawa acara fleksibel",
    "Audio bising dan speaker jelek",
    "Panggung sempit tapi dekor bagus",
    "Tempatnya luas, sound oke"
]

hitung = {k: {"positif":0, "negatif":0, "netral":0} for k in
TEMA.keys()}

for k in komentar_aspek:
    s = cek_sentimen(k)
    low = k.lower()
    aspek_kena = [a for a, kws in TEMA.items() if any(kw in low
for kw in kws)]
    for a in aspek_kena:
        hitung[a][s] += 1

print(hitung)
```

Output:

```
{'sound': {'positif': 1, 'negatif': 1, 'netral': 1}, 'mc':
{'positif': 1, 'negatif': 0, 'netral': 0}, 'venue': {'positif':
2, 'negatif': 0, 'netral': 1}}
```

Jawaban (ekspektasi contoh):

- **sound**: ada *positif* (mantap/oke) dan *negatif* (bising/jelek).
- **mc**: cenderung *positif*
- **venue**: campuran (panas/sempit vs luas/bagus).

Kesimpulan kelas: Fokus perbaikan di **sound** dan **venue**.

Aktivitas 5: Bot CS Mini (Interaktif)

Tujuan: Membuat *chat bot* sederhana yang menanggapi emosi.

Ide: Jika *negatif* → minta maaf + *esctooole*. Jika *positif* → ucap terima kasih.

Langkah:

1. Buat loop input sampai user ketik **exit**.
2. Tentukan sentimen.

3. Balas sesuai aturan.
4. (Opsional) Simpan log ke CSV dengan *Pandas*.

Pseudocode:

```

WHILE True:
    teks = INPUT()
    JIKA teks == "exit": BREAK
    s = CekSentimen(teks)
    JIKA s == "negatif": balas_minta_maaf(); buat_tiket()
    JIKA s == "positif": balas_terimakasih()
    JIKA s == "netral" : balas_tanya_lagi()
    SIMPAN log ke CSV

```

Python:

```

import pandas as pd
log = []

print("Bot CS Mini (ketik 'exit' untuk berhenti)")
while True:
    user = input("Anda: ")
    if user.strip().lower() == "exit":
        break
    s = cek_sentimen(user)
    if s == "negatif":
        balasan = "Maaf ya. Admin akan bantu segera 🙏"
        prioritas = "tinggi"
    elif s == "positif":
        balasan = "Terima kasih! Senang bisa membantu 😊"
        prioritas = "normal"
    else:
        balasan = "Baik. Ada yang bisa kami bantu lagi?"
        prioritas = "normal"
    print("Bot :", balasan)
    log.append({"teks": user, "sentimen": s, "balasan": balasan,
               "prio": prioritas})

# simpan log
df_log = pd.DataFrame(log)
if not df_log.empty:
    df_log.to_csv("log_cs_mini.csv", index=False)
    print("Log disimpan ke log_cs_mini.csv")

```

Output:

```

Bot CS Mini (ketik 'exit' untuk berhenti)
Anda: apa kabar?
Bot : Baik. Ada yang bisa kami bantu lagi?
Anda: apakah disana baik-baik saja?
Bot : Baik. Ada yang bisa kami bantu lagi?
Anda: help!
Bot : Baik. Ada yang bisa kami bantu lagi?

```

```
Anda: exit
Log disimpan ke log_cs_mini.csv
```

Jawaban (ekspektasi):

- Ketik: “Layanannya buruk dan lama” → balasan minta maaf + *prioritas tinggi*.
- Ketik: “Mantap! Suka banget!” → balasan terima kasih.
- Ketik: “Produknya sudah sampai” → balasan netral (tanya lanjut).

Catatan Lanjutan (*Optional untuk tim lanjut*)

- Coba *TF-IDF* + *Naive Bayes* (library: **scikit-learn**) untuk hasil lebih stabil.
- Coba *Neural Network* 3 kelas (*positif/negatif/netral*) dengan **Keras**.
- Tambah *kamus slang* lokal biar model paham bahasa anak SMK.
- Gunakan *OpenWebUI* + *Ollama* untuk *LLM* lokal (offline) saat *brainstorming* kata positif/negatif yang spesifik domain.

Rangkuman

Analisis sentimen adalah cara komputer membaca **perasaan** dari teks. Hasilnya bisa **positif**, **negatif**, atau **netral**. Kita pakai tool *open source* seperti *Python* dan *Pandas* untuk mengolah data. Lalu kita hitung kata-kata penting. Contoh: “enak, mantap, keren” untuk positif. “jelek, buruk, kecewa” untuk negatif. Dengan langkah kecil dan rapi, komputer bisa “merasakan” isi komentar seperti kita.

Agar hasil akurat, lakukan **pra-olah** (*preprocessing*): *lowercase*, buang simbol/emoji, dan normalisasi kata gaul. Ini membantu saat kata “lamaaaaa” menjadi “lama”. Setelah itu, kita bisa memakai logika sederhana (bandingkan jumlah kata positif vs negatif). Untuk level lanjut, gunakan *machine learning* dengan *TF-IDF*, atau **model neural network** di *Keras*. Semua tetap **gratis** dan **open source**.

Manfaatnya nyata dan luas. Konten kreator bisa membaca reaksi penonton. Toko online bisa menangani komplain lebih cepat. Sekolah bisa menilai feedback event. Intinya, kuasai **dasar** dulu, lalu menaikkan level ke *machine learning* dan *neural network*. Terus latihan. Data kecil dulu, lalu tambah. Dengan **disiplin langkah** dan tool *open source*, kalian siap jadi **data hero** di dunia nyata.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Analisis sentimen digunakan untuk mengetahui apakah teks bernuansa positif, negatif, atau netral.

Jawaban: Benar

2. Kata “bagus” termasuk kata negatif dalam analisis sentimen.

Jawaban: Salah

3. Komputer bisa langsung mengerti emosi manusia tanpa algoritma.

Jawaban: Salah

4. Python dapat dipakai untuk membuat program analisis sentimen sederhana.

Jawaban: Benar

5. Jika jumlah kata positif sama dengan kata negatif, maka hasilnya netral.

Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Apa tujuan utama analisis sentimen?

- a. Menghitung panjang teks
- b. Menghitung jumlah kata unik
- c. Mengetahui emosi dalam teks
- d. Menyusun teks otomatis

Jawaban: c

2. Kata “jelek” biasanya termasuk kategori:

- a. Positif
- b. Negatif
- c. Netral
- d. Semua benar

Jawaban: b

3. Bahasa pemrograman yang dipakai dalam contoh analisis sentimen di bab ini adalah:

- a. C++
- b. Python
- c. Java
- d. PHP

Jawaban: b

4. Jika komentar berbunyi “Pelayanannya lama”, maka sentimennya adalah:

- a. Positif
- b. Negatif
- c. Netral
- d. Campuran

Jawaban: b

5. Library Python yang bisa digunakan untuk analisis sentimen adalah:

- a. textblob
- b. pandas
- c. matplotlib
- d. numpy

Jawaban: a

6. Kata “mantap” harus dimasukkan ke daftar kata:

- a. Negatif
- b. Positif
- c. Netral
- d. Buruk

Jawaban: b

7. Program sederhana analisis sentimen menggunakan struktur:

- a. if/else
- b. for/while
- c. switch
- d. class

Jawaban: a

8. Analisis sentimen bisa dipakai untuk:

- a. Prediksi hasil pemilu
- b. Mendeteksi virus komputer
- c. Menghitung nilai ujian
- d. Menghapus file sampah

Jawaban: a

9. Kalimat “Produknya sudah sampai” termasuk sentimen:

- a. Positif
- b. Negatif
- c. Netral
- d. Semua salah

Jawaban: c

10. Dalam pseudocode analisis sentimen, langkah pertama adalah:

- a. Cetak hasil
- b. Hitung kata positif dan negatif
- c. Input teks
- d. Simpan ke file

Jawaban: c

C. Isian Singkat (5 Soal)

1. Kata “kecewa” termasuk kategori _____.

Jawaban: Negatif

2. Untuk menganalisis teks, kita biasanya membuat daftar kata _____ dan kata _____.

Jawaban: Positif, Negatif

3. Jika kata positif lebih banyak daripada kata negatif, maka hasil sentimen adalah _____.

Jawaban: Positif

4. Python function sederhana biasanya dibuat dengan keyword _____.

Jawaban: def

5. Library open source untuk analisis teks di Python salah satunya adalah _____.

Jawaban: textblob

D. Soal Eksplorasi (3 Soal)

Soal 1:

Eksplorasi Analisis Review Film

Buatlah program sederhana yang bisa menganalisis 3 ulasan film berikut:

- “Filmnya keren banget.”
- “Alurnya jelek dan membosankan.”
- “Aktor utamanya lumayan.”

Kunci Jawaban (pseudocode + python):

```
for setiap ulasan:
    hitung kata positif
    hitung kata negatif
    tentukan hasil
```

Python:

```
ulasan = ["Filmnya keren banget", "Alurnya jelek dan
membosankan", "Aktor utamanya lumayan"]
for u in ulasan:
    print(u, "=>", cek_sentimen(u))
```

Output:

```
Filmnya keren banget => positif
Alurnya jelek dan membosankan => negatif
Aktor utamanya lumayan => netral
```

Soal 2:

Eksplorasi Analisis Komentar Sekolah

Kumpulkan 5 komentar fiktif tentang event sekolah. Tentukan jumlah komentar positif, negatif, dan netral, lalu tampilkan hasilnya dalam ringkasan.

Kunci Jawaban (pseudocode):

```
input komentar[]
positif = 0, negatif = 0, netral = 0
for komentar:
    tentukan sentimen
    tambah ke kategori
    cetak jumlah kategori
```

Soal 3:**Eksplorasi Mini Bot Interaktif**

Buat bot sederhana. Bot akan membaca input user, lalu menjawab:

- Jika positif → “Wah, mantap sekali!”
- Jika negatif → “Semoga lebih baik ke depannya.”
- Jika netral → “Oke, noted.”

Kunci Jawaban (pseudocode + python):

```
input teks_user
sentimen = cek_sentimen(teks_user)
if sentimen == positif: print("Wah, mantap sekali!")
if sentimen == negatif: print("Semoga lebih baik ke depannya.")
else: print("Oke, noted.")
```

Python:

```
teks_user = input("Masukkan komentar: ")
hasil = cek_sentimen(teks_user)
if hasil == "positif":
    print("Wah, mantap sekali!")
elif hasil == "negatif":
    print("Semoga lebih baik ke depannya.")
else:
    print("Oke, noted.")
```

Output:

```
Masukkan komentar: Wah, keren banget
Wah, mantap sekali!
```

Tugas Proyek

Proyek 1: Analisis Komentar Lagu Favorit

Kumpulkan 10 komentar dari YouTube tentang lagu favoritmu. Buat program Python untuk menghitung berapa komentar yang positif, negatif, atau netral.

Langkah:

1. Salin 10 komentar ke dalam list Python.
2. Buat daftar kata positif & negatif.
3. Jalankan fungsi untuk cek sentimen tiap komentar.
4. Tampilkan total hasilnya.

Pseudocode:

INPUT: daftar komentar
UNTUK setiap komentar:
 hitung skor positif
 hitung skor negatif
 tentukan hasil (positif/negatif/netral)
OUTPUT: ringkasan jumlah tiap kategori

Contoh Python:

```
komentar = [  
    "Lagunya enak banget!",  
    "Musiknya bikin kecewa",  
    "Liriknya bagus dan dalam",  
    "Kurang mantap",  
    "Suka banget sama beat-nya!"  
]  
  
positif = ["enak", "bagus", "suka", "mantap"]  
negatif = ["kecewa", "jelek", "buruk", "kurang"]  
  
def cek_sentimen(teks):  
    teks = teks.lower()  
    skor_pos = sum(1 for k in positif if k in teks)  
    skor_neg = sum(1 for k in negatif if k in teks)  
    if skor_pos > skor_neg: return "positif"  
    elif skor_neg > skor_pos: return "negatif"  
    else: return "netral"  
  
hasil = [cek_sentimen(k) for k in komentar]  
print(hasil)
```

Output:

```
['positif', 'negatif', 'positif', 'netral', 'positif']
```

Proyek 2: Review Jajanan Kantin

Buat survei kecil dengan 5 review makanan kantin. Analisis sentimennya, lalu tentukan makanan mana yang paling banyak dapat komentar positif.

Pseudocode:

DATA: nama makanan + komentar
UNTUK setiap makanan:
 analisis komentar satu per satu
 hitung jumlah positif & negatif
PILIH makanan dengan nilai positif terbanyak

Kunci Jawaban:

- Siswa membuat tabel sederhana: Makanan → komentar → hasil sentimen.

- Bandingkan skor, lalu simpulkan makanan favorit.

Proyek 3: Analisis Chat Teman

Ambil 5 pesan chat WhatsApp (fiktif) dari temanmu. Tentukan apakah mereka sedang dalam mood positif, negatif, atau netral.

Contoh Chat:

- “Besok nongkrong yuk, aku kangen.”
- “Aduh PR nya banyak banget, pusing.”

Pseudocode:

```
AMBIL list chat
UNTUK setiap chat:
    cek kata positif / negatif
    klasifikasikan mood
CETAK hasil
```

Contoh Python:

```
chat = ["Besok nongkrong yuk, aku kangen",
        "Aduh PR nya banyak banget, pusing"]

for c in chat:
    print(c, "=>", cek_sentimen(c))
```

Output:

```
Besok nongkrong yuk, aku kangen => netral
Aduh PR nya banyak banget, pusing => netral
```

Proyek 4: Buat Grafik Sentimen

Ambil 8 review game. Analisis sentimen lalu buat grafik batang (bar chart) untuk menunjukkan jumlah positif, negatif, netral.

Langkah:

1. Buat list review game.
2. Analisis sentimen tiap review.
3. Gunakan *Pandas* dan *Matplotlib* untuk hitung jumlah.
4. Tampilkan grafik.

Contoh Python:

```
import pandas as pd
import matplotlib.pyplot as plt
```



```

reviews = ["Game ini seru banget", "Grafiknya jelek", "Gameplay
mantap", "Lama loadingnya"]
hasil = [cek_sentimen(r) for r in reviews]

df = pd.DataFrame({"review": reviews, "sentimen": hasil})
print(df["sentimen"].value_counts())
df["sentimen"].value_counts().plot(kind="bar")
plt.show()

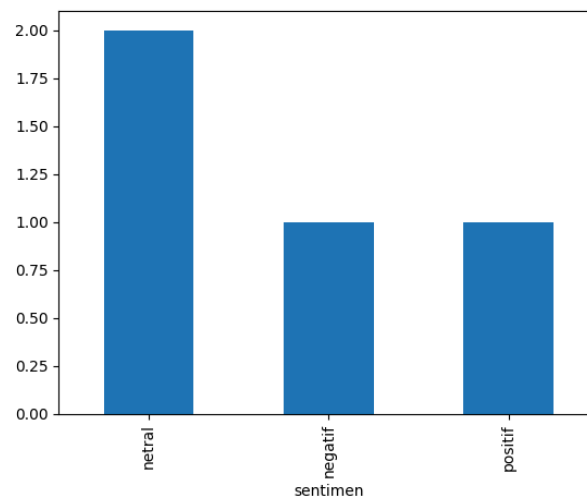
```

Output:

```

sentimen
netral      2
negatif     1
positif     1
Name: count, dtype: int64

```



Proyek 5: Mini Bot Sentimen

Buat bot sederhana yang bisa menerima input dari user lalu menjawab sesuai sentimen.

- Kalau positif → jawab “Senang mendengarnya!”
- Kalau negatif → jawab “Tenang, semoga cepat membaik.”
- Kalau netral → jawab “Oke noted.”

Pseudocode:

```

INPUT: teks user
sentimen = cek_sentimen(teks)
IF sentimen = positif: print("Senang mendengarnya!")
IF sentimen = negatif: print("Tenang, semoga cepat membaik.")
ELSE: print("Oke noted.")

```

Contoh Python:

```
user_input = input("Tulis komentar: ")
hasil = cek_sentimen(user_input)

if hasil == "positif":
    print("Senang mendengarnya!")
elif hasil == "negatif":
    print("Tenang, semoga cepat membaik.")
else:
    print("Oke noted.")
```

Output:

```
Tulis komentar: baju itu bagus sekali
Senang mendengarnya!
```

👉 Semua proyek di atas sederhana, **fun**, tapi juga menantang. Kalian belajar:

- Mengumpulkan data.
- Membuat program analisis sentimen.
- Menampilkan hasil dengan teks atau grafik.
- Bahkan membuat bot mini yang interaktif.

BAB 5: Pengenalan Ollama dan Model LLM Offline

Siapa jadi dewa AI? Kali ini, kita akan kenalan sama teknologi yang bikin **AI** super canggih bisa jalan di laptop kalian sendiri, tanpa perlu internet! Nama pahlawan kita adalah **Ollama**, dan dia bakal bantu kita jadi ahli **Large Language Model (LLM)**. Ini adalah skill yang paling dicari di dunia teknologi saat ini!

Tujuan Pembelajaran

Setelah bab ini, kalian bakal bisa:

- Memahami apa itu **Ollama** dan **model LLM offline**.
- Menginstal dan menjalankan **Ollama** di komputer.
- Memilih dan mengunduh model LLM yang sesuai kebutuhan.
- Menggunakan **Ollama** untuk berinteraksi dengan model AI secara lokal.

Peta Konsep Sederhana

Bayangin AI itu kayak robot.

- **ChatGPT (AI Online)**: Robotnya tinggal di internet. Kita harus pakai internet buat ngobrol sama dia.
- **Ollama (AI Offline)**: Ini adalah "rumah" buat robot-robot AI di laptop kita. Kita bisa "mengundang" robotnya (mengunduh model LLM), dan dia bakal tinggal di laptop kita.
- **Model LLM**: Ini adalah "otak" si robot. Ada yang kecil, ada yang super cerdas. Kita bisa pilih sendiri mau pakai otak yang mana.

Apersepsi: Kamu Mau Ngerjain Tugas Tapi Gak Ada Kuota!

Kamu lagi ngerjain tugas, dan butuh bantuan buat nulis esai. Kamu mau pakai AI, tapi tiba-tiba kuota internet habis! Panik, kan?

- **Dulu**: Kalian harus beli kuota dulu atau cari WiFi gratis.
- **Sekarang (dengan Ollama)**: Kalian bisa pakai AI lokal yang sudah diinstal di laptop. Gak perlu internet! Dia bisa bantu kalian nulis, buat ringkasan, atau bahkan bikin kode. Tuh, kan, keren banget!

Ollama bikin kita jadi mandiri dan gak bergantung sama internet.

Penjelasan Teori

Bayangin ada robot super pintar yang bisa nulis esai, bikin ringkasan, bahkan bantu bikin *kode Python*. Biasanya robot ini tinggal di internet, kayak ChatGPT. Tapi... gimana kalau internet mati? Nah, di sinilah **Ollama** jadi pahlawan!

Dengan Ollama, kalian bisa punya *Large Language Model (LLM)* sendiri di laptop. Jadi robot AI itu bisa dipanggil kapan saja, bahkan *offline*. Serasa punya asisten pribadi yang selalu siap bantu belajar atau bikin proyek.

Konsep Dasar: Apa itu Ollama dan LLM?

- **LLM (Large Language Model):** Bayangin ini sebagai *otak* AI. Otaknya bisa ngerti teks, bikin teks, bahkan bantu mikirin ide.
- **ChatGPT (Online):** Robot yang tinggal di “awan” internet. Butuh kuota untuk ngobrol.
- **Ollama (Offline):** Rumah robot AI di laptop kita. Jadi robotnya pindah rumah, bisa dipanggil tanpa kuota.
- **Model LLM:** Jenis-jenis otak. Ada yang kecil dan ringan (misal *tinylama*), ada juga yang besar dan super pintar (misal *llama3*).

Kenapa Kita Perlu Ollama?

Ada beberapa alasan yang bikin Ollama penting banget:

1. **Privasi:** Semua data cuma ada di laptop. Gak bocor ke server luar.
2. **Kecepatan:** Jawaban keluar cepat, karena gak perlu nunggu internet.
3. **Hemat biaya:** Gak perlu beli kuota tambahan atau langganan.
4. **Kemandirian:** Bisa belajar dan bereksperimen tanpa tergantung WiFi.

Contoh Kasus di Kehidupan Siswa

- **Tugas Sekolah:** Lagi butuh ringkasan biologi → tinggal tanya Ollama.
- **Content Creator:** Mau ide TikTok → AI kasih banyak saran.
- **Belajar Coding:** Kode Python error → AI bantu perbaiki.
- **Lamaran Magang:** Bingung nulis surat lamaran → AI bikin draf.
- **Hobi Nulis:** Mentok bikin cerita → AI bantu lanjutkan ide.

Kaitan dengan *Open Source Tools*

Ollama itu gratis dan *open source*. Sama kayak Python, Pandas, Machine Learning, Keras, dan *Neural Network* yang sering kita pelajari:

- **Python:** bahasa pemrograman yang gampang dipahami.
- **Pandas:** tool buat ngolah data tabel.
- **Machine Learning:** teknik ngajarin komputer biar pintar.
- **Keras:** pustaka Python buat bikin *neural network*.
- **Neural Network:** jaringan otak buatan yang jadi dasar AI.

Dengan Ollama, semua teknologi ini bisa jalan bareng. Jadi, kita bisa bikin proyek yang keren banget, tapi tetap pakai software gratis!

Analogi Sederhana

Bayangin laptopmu adalah rumah.

- Kalau pakai ChatGPT → setiap kali mau tanya, harus keluar rumah dulu ke “kantor pusat” (internet).
- Kalau pakai Ollama → robot pintar tinggal di rumahmu sendiri. Jadi kalau mau tanya, langsung aja di rumah, gak perlu keluar lagi.

Instalasi Ollama dan Open-WebUI

Instalasi **Ollama** dan **OpenWebUI** sebenarnya tidaklah rumit. Namun, tetap dibutuhkan keterampilan dasar dalam mengelola dan menginstal server **Linux** agar prosesnya berjalan lancar.

Bagi yang ingin mempelajari teknik instalasi secara lebih detail, penjelasan lengkap telah tersedia di **wiki onnocenter**

https://onnocenter.or.id/wiki/index.php/LLM:_ubuntu_24.04_ollama_%2B_open-webio_full_docker

Dengan panduan tersebut, siapa pun bisa menyiapkan lingkungan AI lokal, menjalankan **Large Language Model (LLM)** di laptop, dan memanfaatkan AI secara *offline* dengan lebih mandiri, cepat, dan aman.

Ringkasan

- Ollama = rumah buat AI di laptop.
- LLM = otak AI yang bisa dipakai buat banyak hal.
- Kelebihan: aman, cepat, hemat, mandiri.
- Cocok banget buat siswa SMK yang mau belajar, eksperimen, atau bikin karya kreatif.

Mau saya lanjutkan bikin **pseudocode + contoh script Python** biar makin jelas cara Ollama dipakai bareng Python/Pandas?

Kasus Ilustrasi

Kasus 1: Asisten Belajar Offline

Kamu lagi mau ujian biologi. Internet mati. Panik? Tenang, Ollama bisa jadi penyelamat.

- Jalankan Ollama di laptop.

- Pilih model LLM yang sudah di-*download* (misalnya *gemma*).
- Tanya ringkasan materi.

Pseudocode:

```
# Jalankan Ollama dengan model gemma
ollama run gemma

# Tanyakan materi biologi
Input: "Jelaskan fotosintesis dalam 5 poin"
Output: "Fotosintesis adalah ..."
```

Kasus 2: Pembuat Ide Konten

Kamu ingin jadi kreator TikTok tapi kehabisan ide. Ollama bisa kasih saran.

- Jalankan Ollama.
- Tulis prompt tentang ide konten anak SMK.
- Catat ide-ide menarik untuk dipakai.

Pseudocode:

```
ollama run gemma

Input: "Buatkan 10 ide konten TikTok tentang kehidupan siswa SMK"
Output: [daftar ide konten]
```

Kasus 3: Latihan Coding Python

Kamu bikin program Python, tapi error muncul. Daripada bingung, minta Ollama bantu jelaskan.

- Salin kode error ke Ollama.
- Ollama akan kasih analisis dan saran perbaikan.

Pseudocode:

```
ollama run llama3

Input: "Perbaiki kode Python ini:
for i in range(5)
    print(i)"
Output: "Tambahkan tanda ':' setelah range(5)."
```

Kasus 4: Menulis Surat Lamaran Kerja

Kamu mau magang di perusahaan game. Bingung nulis surat? Ollama bisa bikin draf profesional.

- Jalankan Ollama.
- Beri instruksi tulis surat lamaran.
- Simpan hasilnya, lalu edit sesuai kebutuhan.

Pseudocode:

```
ollama run gemma
```

```
Input: "Tolong buat surat lamaran magang di perusahaan game"
```

```
Output: [isi surat lamaran]
```

Kasus 5: Membuat Cerita Fiksi

Kamu hobi nulis cerita, tapi ide mandek. Ollama bisa jadi teman berimajinasi.

- Jalankan Ollama.
- Masukkan potongan cerita yang kamu tulis.
- Minta Ollama melanjutkan.

Pseudocode:

```
ollama run mistral
```

```
Input: "Lanjutkan cerita ini:
```

```
Seorang anak SMK menemukan laptop tua di gudang..."
```

```
Output: [lanjutan cerita fiksi]
```

Kesimpulan

Dengan Ollama, siswa SMK bisa:

- Belajar tanpa internet.
- Dapat ide kreatif.
- Latihan coding lebih mudah.
- Bikin dokumen profesional.
- Berkarya lewat tulisan fiksi.

Ollama = asisten pintar yang selalu ada di laptopmu. Gratis, cepat, *open source*!

Soal Latihan & Pembahasan

Soal 1

Pertanyaan: Apa perbedaan utama *LLM offline* (Ollama) dan *LLM online* (ChatGPT)?

Jawaban:

- LLM offline bisa dipakai tanpa internet, lebih cepat, dan data aman.
- LLM online butuh internet, data dikirim ke server, kadang lebih mahal.

Pseudocode:

```
if mode == "offline":  
    print("Cepat, aman, tanpa internet")  
else:  
    print("Butuh internet, data ke server")
```

Soal 2

Pertanyaan: Melengkapi perintah untuk mengunduh model *llama3* di Ollama.

Jawaban:

Perintahnya adalah:

```
ollama pull llama3
```

Pembahasan:

- *pull* artinya mengunduh model dari server Ollama.
- Setelah diunduh, model bisa dipakai kapan saja.

Pseudocode:

```
command = "ollama pull llama3"  
execute(command)
```

Soal 3

Pertanyaan: Kenapa Ollama penting untuk siswa SMK?

Jawaban:

- Gratis dan *open source*.
- Bisa dipakai *offline*.
- Cocok buat belajar coding, nulis esai, bikin ringkasan, dll.

Pembahasan:

Ollama bikin siswa bisa eksplorasi AI tanpa perlu kuota. Aman, hemat, dan seru.

Pseudocode:


```
if user == "siswa SMK":
    benefits = ["gratis", "offline", "aman", "buat belajar"]
    print(benefits)
```

Soal 4

Pertanyaan: Apa yang terjadi kalau kamu mencoba jalankan Ollama tanpa mengunduh model dulu?

Jawaban:

- Akan muncul pesan error: model tidak ditemukan.
- Ollama butuh *otak* (model LLM) untuk bisa bekerja.

Pembahasan:

Model = bahan bakar AI. Kalau tidak ada, Ollama tidak bisa menjawab.

Pseudocode:

```
if model == None:
    print("Error: Model belum diunduh")
else:
    run(model)
```

Soal 5

Pertanyaan: Buatlah contoh pertanyaan ke Ollama untuk membantu belajar sehari-hari di sekolah.

Jawaban:

Contoh pertanyaan:

"Ringkas materi sejarah tentang Perang Dunia II dalam 5 poin."

Pembahasan:

Dengan pertanyaan seperti ini, siswa bisa dapat ringkasan cepat untuk belajar.

Pseudocode:

```
ollama run gemma

Input: "Ringkas Perang Dunia II dalam 5 poin"
Output: ["Penyebab perang", "Tokoh penting", "Negara terlibat",
"Hasil perang", "Dampak ke dunia"]
```

Kesimpulan

Dengan latihan soal ini, siswa SMK belajar bahwa:

- Ollama = AI offline yang seru, aman, dan hemat.
- Perintahnya sederhana (misal `ollama pull` dan `ollama run`).
- Bisa dipakai untuk belajar, coding, sampai bikin karya kreatif.

Fun Facts!

- Model LLM itu punya ukuran file yang besar, lho! Ada yang sampai puluhan GB.
- **Ollama** itu kompatibel dengan **Orange Data Mining**. Kalian bisa menghubungkan keduanya untuk membuat visualisasi dari hasil teks yang dibuat AI.
- **RAG lokal** adalah teknologi yang bikin AI kalian bisa baca dokumen di komputer kalian. Ini berguna banget kalau kalian mau bikin AI yang ahli dalam materi sekolah kalian.

Tips Belajar/Menghafal Cepat

- **Langsung Coba!** Jangan cuma baca. Langsung instal **Ollama** dan coba rasakan sendiri betapa kerennya dia.
- **Mulai dari Model Kecil:** Unduh model yang ukurannya kecil dulu (misal **gemma** atau **tinyllama**).
- **Buat Catatan:** Buat catatan di buku, "Ollama itu apa? Kapan aku pakai dia?"

Aktivitas Siswa

Aktivitas 1: Instalasi Ollama di Laptop

Sebelum ngobrol dengan AI, kita harus pasang dulu “otaknya” di laptop. Otak ini bernama **Ollama**. Dengan Ollama, kalian bisa menjalankan model AI lokal seperti *Gemma*, *Mistral*, *LLaMA*, dan lainnya.

Langkah-langkah Instalasi

Untuk Linux (Ubuntu/Debian)

- Buka **Terminal**.
Jalankan perintah instalasi resmi Ollama:
`curl -fsSL https://ollama.com/install.sh | sh`
- Setelah selesai, cek apakah Ollama sudah terpasang:
`ollama --version`

Untuk MacOS (pakai Homebrew)

```
brew install ollama/tap/ollama
ollama --version
```

Untuk Windows

- Unduh installer dari: <https://ollama.com/download>
- Jalankan file **.msi** lalu ikuti petunjuk.

- Buka **Command Prompt** dan cek:

```
ollama --version
```

Pseudocode (Versi CLI)

```
download ollama installer
jalankan installer
cek instalasi dengan: ollama --version
```

Contoh Hasil di Terminal

```
ollama version 0.1.32
```

Setelah Ollama terinstal, kalian sudah siap untuk **menarik model pertama** dengan:

```
ollama pull gemma
```

Aktivitas 2: Mengunduh Model LLM Pertama

Langkah-langkah

1. Buka **Terminal**.
Jalankan perintah berikut:

```
ollama pull gemma
```
2. Tunggu proses download selesai.
Setelah selesai, model siap digunakan di komputer kalian.

Pseudocode (Versi CLI)

```
model = "gemma"
jalankan: ollama pull gemma
```

Contoh Output di Terminal

```
pulling manifest
downloading 500 MB
verifying sha256 digest
success | gemma:latest
```

Setelah model selesai diunduh, kalian bisa langsung mencobanya dengan:

```
ollama run gemma
```

Aktivitas 3: Ngobrol dengan AI Lokal

Sekarang waktunya kita **bercakap langsung dengan AI lokal** lewat terminal. Kita tidak perlu Python, cukup gunakan **perintah CLI** saja.

Langkah-langkah

- Buka **Terminal**.
Jalankan model dengan Ollama:
- Ketik pertanyaan di prompt AI:

```
Sebutkan 5 kota di Jawa Barat
```

- Tunggu jawaban dari AI.

Pseudocode (Versi CLI)

```
ollama run gemma
> Sebutkan 5 kota di Jawa Barat
< ["Bandung", "Bekasi", "Depok", "Bogor", "Cimahi"]
```

Alternatif Satu Baris (langsung kirim pertanyaan)

Kalau tidak mau interaktif, bisa langsung pakai **echo + pipe** di CLI:

```
echo "Sebutkan 5 kota di Jawa Barat" | ollama run gemma
```

Contoh Jawaban AI

```
Bandung
Bekasi
Depok
Bogor
Cimahi
```

Aktivitas 4: Hitung Kata dengan Python + Ollama

Langkah-langkah

Jalankan Ollama dari terminal, minta 10 kata tentang makanan:

```
ollama run gemma "Berikan 10 kata yang berhubungan dengan makanan"
```

Salin hasil output Ollama ke file teks, misalnya **makanan.txt**:

```
ollama run gemma "Berikan 10 kata yang berhubungan dengan makanan" > makanan.txt
```

Gunakan Python CLI untuk menghitung jumlah kata di file itu:

```
python3 -c "print('Jumlah kata:',
len(open('makanan.txt').read().split()))"
```

Kalau mau ditampilkan sebagai tabel sederhana pakai Pandas (masih via CLI):

```
python3 -c "import pandas as pd;
```

```
words=open('makanan.txt').read().split(); df=pd.DataFrame(words,
columns=['Makanan']); print(df); print('Jumlah kata:', len(df))"
```

Contoh Hasil (misalnya)

Output Ollama

```
nasi ayam ikan tempe tahu soto bakso mie roti sambal
```

Hasil Python CLI

```
Jumlah kata: 10
Makanan
0      nasi
1      ayam
2      ikan
3      tempe
4      tahu
5      soto
6      bakso
7      mie
8      roti
9      sambal
```

Aktivitas 5: Diskusi Kelompok – Problem Solving dengan AI

Sekarang waktunya kerja tim

Kalian akan berdiskusi bareng untuk mencari masalah nyata di sekolah, lalu minta **AI lokal (Ollama)** memberi solusi.

Langkah-langkah

1. **Bentuk kelompok kecil** (3–5 orang).
2. **Brainstorm** masalah di sekolah. Contoh:
 - o Menyusun jadwal belajar.
 - o Membuat ringkasan pelajaran.
 - o Membuat soal latihan.

Setelah sepakat, minta AI lewat CLI.

Gunakan perintah:

```
echo "Ringkas Perang Diponegoro dalam 5 poin" | ollama run gemma
```

atau untuk masalah lain:

```
echo "Buat jadwal belajar 1 minggu untuk siswa SMK kelas 10" |
ollama run gemma
```

3. **Diskusikan hasilnya** di kelompok.

4. Presentasikan solusi ke kelas.

Pseudocode (Versi CLI)

```
problem = "Sulit bikin ringkasan sejarah"
jalankan: echo "Tolong buat ringkasan sejarah Perang Diponegoro
dalam 5 poin" | ollama run gemma
tampilkan hasil ke kelompok
presentasikan di depan kelas
```

Contoh Hasil di Terminal

1. Perang Diponegoro terjadi tahun 1825-1830 di Jawa.
2. Dipimpin oleh Pangeran Diponegoro melawan Belanda.
3. Penyebabnya adalah pajak tinggi dan perampasan tanah.
4. Perang melibatkan rakyat Jawa secara luas.
5. Berakhir dengan ditangkapnya Diponegoro pada 1830.

Rangkuman Aktivitas

- Aktivitas 1: Instalasi Ollama.
- Aktivitas 2: Unduh model AI.
- Aktivitas 3: Ngobrol dengan AI lokal.
- Aktivitas 4: Hitung kata dengan Python + Pandas.
- Aktivitas 5: Diskusi kelompok dengan bantuan AI.

Dengan 5 aktivitas ini, kalian tidak hanya paham teori. Tapi langsung praktik jadi **ilmuannya AI SMK** dengan tool *open source*!

Rangkuman

Di bab ini kita belajar bahwa **Ollama adalah tool gratis dan open source** yang bisa dipakai untuk menjalankan *Large Language Model (LLM)* langsung di laptop. Bedanya dengan ChatGPT yang harus pakai internet, **Ollama bisa berjalan offline**. Jadi walaupun kuota habis atau WiFi mati, kita tetap bisa ngobrol dengan AI dan memanfaatkan kemampuannya.

Dengan Ollama, kita bisa mencoba berbagai hal menarik. Mulai dari **membuat ringkasan pelajaran, menulis esai, latihan coding dengan Python dan Pandas**, sampai bikin ide konten TikTok atau menulis cerita fiksi. Karena semua berjalan di laptop sendiri, **data jadi lebih aman, cepat, dan hemat biaya**. Kita juga belajar bahwa LLM itu ibarat “otak AI”, ada yang kecil seperti *TinyLlama*, ada juga yang besar dan pintar seperti *Llama 3*.

Kesimpulannya, **Ollama membuat kita mandiri**. Kita bisa belajar, berkreasi, bahkan bereksperimen dengan AI tanpa tergantung internet. Ini penting banget untuk siswa SMK karena bisa melatih keterampilan teknologi terbaru dengan **tool gratis dan open source** seperti *Python, Pandas, Machine Learning, Keras*, dan *Neural Network*. Jadi, jangan takut untuk mencoba, karena dengan Ollama di laptop kamu, kamu sudah punya **laboratorium AI pribadi**!

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Ollama bisa dijalankan tanpa internet.
Jawaban: Benar
2. ChatGPT adalah contoh *LLM offline*.
Jawaban: Salah
3. Perintah `ollama pull gemma` digunakan untuk mengunduh model Gemma.
Jawaban: Benar
4. LLM itu singkatan dari *Large Language Machine*.
Jawaban: Salah
5. Data lebih aman jika diproses menggunakan Ollama offline.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Apa kepanjangan dari LLM?
 - a. Long Language Memory
 - b. Large Language Model
 - c. Local Learning Machine
 - d. Logical Language Method**Jawaban:** b
2. Ollama termasuk software...
 - a. Berbayar
 - b. Open Source
 - c. Tertutup
 - d. Trial Version**Jawaban:** b
3. Apa yang terjadi jika Ollama dijalankan tanpa model?
 - a. Tetap jalan normal
 - b. Muncul error
 - c. Langsung menutup
 - d. Tidak terjadi apa-apa**Jawaban:** b
4. Contoh model LLM yang bisa dijalankan di Ollama adalah...
 - a. TensorFlow
 - b. Llama 3
 - c. MySQL
 - d. Visual Studio

Jawaban: b

5. Ollama membantu siswa SMK karena...
- a. Membutuhkan internet cepat
 - b. Gratis dan bisa offline
 - c. Hanya jalan di komputer super mahal
 - d. Harus pakai server luar negeri

Jawaban: b

6. Perintah untuk menjalankan model dengan Ollama adalah...
- a. `ollama start llama3`
 - b. `ollama open llama3`
 - c. `ollama run llama3`
 - d. `ollama play llama3`

Jawaban: c

7. Apa fungsi utama LLM?
- a. Menggambar foto
 - b. Mengolah teks
 - c. Menyimpan data
 - d. Menjalankan musik

Jawaban: b

8. Jika ingin tahu 5 kota di Jawa Barat, kita bisa bertanya ke...
- a. Ollama offline
 - b. Microsoft Word
 - c. Excel
 - d. PowerPoint

Jawaban: a

9. Ollama berguna untuk menjaga...
- a. Kecepatan internet
 - b. Privasi data
 - c. Biaya kuota
 - d. Semua jawaban benar

Jawaban: d

10. LLM disebut juga sebagai...
- a. Otak AI
 - b. Data Center
 - c. Koneksi Internet
 - d. Operating System

Jawaban: a

C. Isian Singkat (5 Soal)

1. Perintah untuk mengunduh model Llama 3 adalah:

Jawaban: `ollama pull llama3`

2. Sebutkan satu contoh model LLM selain Llama 3.

Jawaban: Gemma

3. Ollama bisa dipakai tanpa _____.

Jawaban: internet

4. Data lebih _____ jika diproses dengan Ollama.

Jawaban: aman

5. LLM adalah singkatan dari _____.

Jawaban: Large Language Model

D. Soal Eksplorasi (3 Soal)

Soal 1

Bayangkan kamu sedang menulis esai tentang “Perkembangan Teknologi AI”. Jelaskan langkah-langkah menggunakan Ollama untuk membantumu menulis ringkasan esai tersebut secara offline.

Contoh Jawaban:

1. Instal Ollama di laptop.
2. Unduh model, misalnya Gemma.
3. Jalankan perintah `ollama run gemma`.
4. Masukkan prompt: “Buat ringkasan esai tentang Perkembangan Teknologi AI.”
5. Salin hasilnya untuk melengkapi esai.

Soal 2

Kamu ingin belajar coding Python, tapi menemukan error. Buatlah skenario bagaimana kamu meminta bantuan Ollama untuk memperbaiki kode berikut:

```
for i in range(5)
    print(i)
```

Contoh Jawaban:

1. Salin kode error.
2. Jalankan Ollama: `ollama run gemma`.
3. Prompt: “Perbaiki kode Python berikut.”
4. Ollama akan menjawab: tambahkan tanda “:” setelah `range(5)`.
5. Kode benar:

```
for i in range(5):  
    print(i)
```

Soal 3

Bayangkan kamu diminta membuat ide bisnis kecil untuk siswa SMK. Jelaskan bagaimana kamu menggunakan Ollama untuk menghasilkan 3 nama unik bisnis makanan.

Contoh Jawaban:

1. Jalankan perintah `ollama run gemma`.
2. Prompt: "Buatkan 3 nama bisnis unik untuk usaha makanan anak SMK."
3. Ollama memberikan contoh: "*Roti Ria, Nasi Go, JajanKuy*".
4. Pilih salah satu nama, lalu kembangkan jadi ide bisnis.

Dengan latihan ini, siswa SMK bisa mengasah pemahaman coding dasar, AI, dan pemakaian Ollama secara praktis.

Tugas Proyek

Proyek 1: Kamus Mini Bahasa Daerah ke Bahasa Indonesia

Sekarang kita akan membuat **kamus mini**.

Kamus ini berisi beberapa kata dari bahasa daerah, lalu diterjemahkan ke bahasa Indonesia menggunakan **Ollama** lewat terminal.

Tantangan

Buat daftar 5 kata bahasa daerahmu, lalu minta AI menjelaskan artinya dalam bahasa Indonesia.

Langkah-langkah

- Buka Terminal.
Jalankan Ollama dengan model gemma dan masukkan kata daerah.

Contoh:

```
echo "Terjemahkan kata 'sanes' ke bahasa Indonesia" |  
ollama run gemma  
echo "Terjemahkan kata 'nderek' ke bahasa Indonesia" |  
ollama run gemma  
echo "Terjemahkan kata 'bade' ke bahasa Indonesia" |  
ollama run gemma
```

- Simpan hasil terjemahan ke dalam file kamus.txt:

```
echo "sanes -> $(echo 'Terjemahkan kata sanes ke bahasa  
Indonesia' | ollama run gemma)" >> kamus.txt  
echo "nderek -> $(echo 'Terjemahkan kata nderek ke bahasa  
Indonesia' | ollama run gemma)" >> kamus.txt
```

```
echo "bade -> $(echo 'Terjemahkan kata bade ke bahasa  
Indonesia' | ollama run gemma)" >> kamus.txt
```

Tampilkan kamus mini kalian:

```
cat kamus.txt
```

Pseudocode (Versi CLI)

```
kata = "sanes"  
jalankan: echo "Terjemahkan kata 'sanes' ke bahasa Indonesia" |  
ollama run gemma  
simpan hasil ke file kamus.txt  
ulang untuk semua kata
```

Contoh Hasil di Terminal

```
sanes -> bukan  
nderek -> ikut  
bade -> hendak
```

Proyek 2: Quiz Sederhana dengan AI

Sekarang kita akan membuat kuis matematika kecil.

Ada 3 soal, dan kita minta **Ollama** menjawab langsung lewat terminal.

Tantangan

1. Buat 3 soal matematika dasar.
2. Kirim soal ke Ollama untuk dijawab.
3. Simpan soal + jawaban ke file **quiz.txt**.

Langkah-langkah

- Buka **Terminal**.
Jalankan perintah berikut untuk setiap soal:

```
echo "2+5=?" | ollama run gemma >> quiz.txt  
echo "10*3=?" | ollama run gemma >> quiz.txt  
echo "12/4=?" | ollama run gemma >> quiz.txt
```

- Lihat hasil kuis dengan:

```
cat quiz.txt
```

Pseudocode (Versi CLI)

```
soal = "2+5=?"  
jalankan: echo "2+5=?" | ollama run gemma >> quiz.txt  
ulang untuk semua soal  
tampilkan hasil dengan cat quiz.txt
```

Contoh Hasil di Terminal

2+5=? -> 7
10*3=? -> 30
12/4=? -> 3

Proyek 3: Penulis Caption Instagram

Sekarang saatnya latihan jadi **content creator SMK** 🎉

Kita minta **Ollama** bikin 3 caption Instagram yang keren, singkat, dan kreatif tentang kehidupan anak SMK.

Tantangan

- Gunakan Ollama di terminal.
- Minta AI membuat 3 caption IG tentang kehidupan anak SMK.
- Salin hasilnya, lalu bayangkan seolah-olah diposting di Instagram (simulasi).

Langkah-langkah

1. Buka **Terminal**.

Jalankan perintah berikut:

```
echo "Buat 3 caption IG tentang kehidupan anak SMK yang kreatif dan singkat" | ollama run gemma
```

2. Tunggu Ollama memberi hasil caption.
3. Copy hasil → tempel ke Instagram (simulasi).

Pseudocode (Versi CLI)

```
prompt = "Buat 3 caption IG tentang kehidupan anak SMK yang kreatif dan singkat"
jalankan: echo "prompt" | ollama run gemma
tampilkan hasil
```

Contoh Hasil di Terminal

1. Hidup itu simple, yang ribet cuma tugas SMK
2. Dari bengkel ke kelas, semangat selalu on
3. Anak SMK? Siap kerja, siap keren, siap sukses!

Proyek 4: Ringkasan Artikel Berita

Tantangan

Ambil 1 artikel berita dari internet (copy teksnya), lalu minta Ollama merangkum jadi 5 poin utama.

Langkah

- Copy isi artikel ke file teks **artikel.txt**

```
nano artikel.txt
```

(paste isi artikel, lalu simpan & keluar)

- Jalankan Ollama dengan prompt untuk merangkum:
 - `ollama run gemma "Ringkas artikel ini jadi 5 poin utama:" < artikel.txt > ringkasan.txt`
- Hasil ringkasan akan otomatis tersimpan di **ringkasan.txt**.

Pseudocode (versi CLI)

```
# Simpan artikel ke file
echo "isi artikel ..." > artikel.txt

# Jalankan model gemma di Ollama untuk merangkum jadi 5 poin
ollama run gemma "Ringkas artikel ini jadi 5 poin utama:" <
artikel.txt > ringkasan.txt

# Cek hasil ringkasan
cat ringkasan.txt
```

Proyek 5: Generator Ide Bisnis

Tantangan

Buat simulasi: kamu ingin buka usaha kecil (misalnya jualan makanan). Minta Ollama kasih **5 ide nama bisnis unik**.

Langkah

- Jalankan Ollama dengan prompt:
`ollama run gemma "Buat 5 nama bisnis unik untuk usaha jualan makanan anak SMK."`
- Simpan hasil ke file (opsional):
`ollama run gemma "Buat 5 nama bisnis unik untuk usaha jualan makanan anak SMK." > ide_bisnis.txt`
- Lihat hasil:
`cat ide_bisnis.txt`

Pseudocode (versi CLI)

```
# Definisikan prompt
PROMPT="Buat 5 nama bisnis unik untuk usaha jualan makanan anak
SMK."

# Jalankan Ollama dengan prompt
ollama run gemma "$PROMPT"

# (opsional) simpan ke file
ollama run gemma "$PROMPT" > ide_bisnis.txt
```

Kesimpulan Tugas Proyek

- **Proyek 1:** Kamus mini bahasa daerah.
- **Proyek 2:** Quiz matematika dengan AI.

- **Proyek 3:** Caption IG kreatif.
- **Proyek 4:** Ringkasan artikel berita.
- **Proyek 5:** Ide bisnis unik.

Semua proyek ini sederhana tapi **menantang**, membuat siswa SMK terbiasa berpikir kreatif dan pakai teknologi *open source* seperti *Python*, *Pandas*, *Machine Learning*, *Keras*, dan *Neural Network* sebagai pendukung.

BAB 6: Eksperimen Interaksi LLM lewat OpenWebUI

Di bab sebelumnya, kita sudah berhasil "mengundang" AI canggih ke laptop kita pakai **Ollama**. Tapi, ngobrol sama AI di Terminal itu rasanya kaku, ya? Kayak ngobrol sama robot beneran. Nah, sekarang kita bakal pasangin AI kita dengan "baju" yang cakep dan gampang dipakai. Kita akan kenalan sama **OpenWebUI**, antarmuka web yang bikin interaksi dengan AI lokal jadi semudah pakai ChatGPT!

Tujuan Pembelajaran

- Memahami apa itu **OpenWebUI** dan perannya.
- Menginstal dan menjalankan **OpenWebUI** di komputer.
- Menghubungkan **OpenWebUI** dengan model **Ollama** yang sudah diinstal.
- Berinteraksi dengan model LLM menggunakan antarmuka yang ramah pengguna.

Peta Konsep Sederhana

Bayangin ini seperti membangun sebuah kafe. ☕

- **Robot Pembuat Kopi (Model LLM):** Ini adalah AI kita yang jago bikin kopi (menjawab pertanyaan).
- **Meja Bar (Ollama):** Ini adalah "dapur" tempat robotnya bekerja.
- **Ruang Duduk yang Nyaman (OpenWebUI):** Ini adalah tempat kita dan teman-teman bisa duduk santai dan pesan kopi dengan gampang.

Apersepsi: Kamu Mau Pesan Makanan tapi Gak Ada Menu!

Bayangin kamu datang ke sebuah restoran, tapi tidak ada menu. Kamu harus menghafal semua nama makanan dan harganya. Susah, kan? Nah, kayak gitu rasanya kalau kita ngobrol sama AI di Terminal. Tidak ada tombol, tidak ada riwayat chat, dan harus ngetik perintah terus-menerus.

Dengan **OpenWebUI**, kita dikasih menu yang lengkap. Kita bisa pilih model, lihat riwayat percakapan, dan ngetik dengan santai. Interaksi dengan AI jadi jauh lebih menyenangkan!

Penjelasan Teori

1. Kenapa Harus Pakai OpenWebUI?

Bayangin kamu ngobrol sama robot pintar lewat *Terminal*.

- Harus ketik perintah panjang.
- Tidak ada tombol klik.
- Tidak ada riwayat percakapan.

Rasanya kaku, kayak ngobrol sama robot betulan tanpa ekspresi.

Di sinilah **OpenWebUI** muncul. Ia membuat interaksi dengan AI jadi nyaman. Seperti masuk restoran yang ada menunya. Kamu tinggal pilih, pesan, langsung dapat.

2. Apa Itu OpenWebUI?

OpenWebUI adalah **antarmuka web** (mirip website) yang **gratis** dan **open source**. Ia dipasang di komputer kita untuk ngobrol dengan AI lokal dari **Ollama**.

Dengan OpenWebUI:

- Kamu bisa chat dengan AI lewat browser.
- Bisa pilih model AI dengan mudah.
- Percakapanmu tersimpan dan bisa dibuka kapan saja.
- AI bisa baca dokumen PDF, Word, atau file lain dari laptopmu lewat fitur *RAG*.

Sederhananya, OpenWebUI = wajah ramah dari AI.

3. Analogi Seru

Bayangin bikin kafe:

- **Robot Barista** = LLM (otak AI).
- **Dapur Kopi** = Ollama (tempat robot kerja).
- **Ruang Duduk Nyaman** = OpenWebUI (tempat kita pesan kopi enak dengan menu).

Kalau cuma Ollama, kita harus masuk dapur sendiri. Ribet.

Dengan OpenWebUI, cukup duduk manis, pesan, lalu nikmatin hasilnya.

4. Kenapa Cocok untuk Siswa SMK?

OpenWebUI bisa dipakai untuk banyak hal:

- **Belajar cepat** → AI bisa meringkas bab buku.
- **Mengerjakan proyek** → misalnya bikin chatbot materi pelajaran.
- **Latihan bahasa** → ngobrol dengan AI dalam bahasa Inggris.
- **Analisis data** → misalnya file CSV pakai Python dan *Pandas*.

Dan semua ini berbasis **open source**: Python, Pandas, *Machine Learning*, Keras, dan *Neural Network*. Gratis dan bisa dikembangkan sendiri.

5. Instalasi OpenWebUI dan Ollama

Sekarang kita masuk ke bagian penting: cara memasang **Ollama** dan **OpenWebUI**. Instalasi ini dilakukan di server **Linux Ubuntu 24.04** menggunakan **Docker**.

Langkah Instalasi

Update sistem

```
sudo apt update && sudo apt upgrade -y
```

Install Docker & Docker Compose

```
sudo apt install docker.io docker-compose -y  
sudo systemctl enable docker --now
```

Pull image Ollama & OpenWebUI

```
docker pull ollama/ollama  
docker pull ghcr.io/open-webui/open-webui:main
```

Buat file **docker-compose.yml**

```
nano docker-compose.yml
```

Isi dengan konfigurasi sesuai panduan (Ollama + OpenWebUI stack).\

Jalankan container

```
docker-compose up -d
```

Cek status container

```
docker ps
```

Akses OpenWebUI di browser

```
http://localhost:3000
```

Panduan lengkap step-by-step bisa dilihat di:

[Onno Center Wiki – Instalasi Ubuntu 24.04 + Ollama + OpenWebUI via Docker](#)

Kasus Ilustrasi

Kasus 1: Ringkasan Buku Sekolah

Bayangin kamu punya buku Fisika dalam format PDF. Kamu malas baca panjang-panjang, tapi butuh ringkasan cepat.

Dengan **OpenWebUI** (antarmuka web), caranya gampang:

Langkah di OpenWebUI (GUI)

1. **Buka OpenWebUI** lewat browser (misalnya <http://localhost:3000>).
2. **Klik tombol Upload File** dan pilih file [fisika_bab1.pdf](#).
3. Setelah file berhasil diunggah, file itu akan muncul di area chat.
Di kolom chat, ketik perintah:

```
Tolong buat ringkasan 5 poin dari Bab 1.
```

Tekan **Enter** → AI akan menampilkan ringkasan isi Bab 1 dalam 5 poin singkat.

(Opsional) Klik tombol **Save / Copy** untuk menyimpan hasil ringkasan.

Pseudocode (versi GUI)

```
[Upload File] -> "fisika_bab1.pdf"  
[Chat Box]    -> "Ringkas isi bab 1 jadi 5 poin singkat"  
[Output]      -> Ringkasan 5 poin ditampilkan
```

Kasus 2: Artikel Blog Tentang Game

Bayangin kamu suka main **Mobile Legends** atau **Free Fire**.

Terus kamu ingin bikin artikel blog tentang tips bermain, tapi bingung mulai dari mana.

Nah, dengan **OpenWebUI**, kamu bisa langsung minta AI bikin **draft artikel + judul keren**.

Langkah di OpenWebUI (GUI)

1. **Buka OpenWebUI** di browser.
Di kolom **Chat Box**, ketik:

```
Buat artikel 3 paragraf tentang Tips Bermain Mobile  
Legends untuk Pemula
```

2. Tekan **Enter** → AI akan menuliskan artikel draft dengan 3 paragraf. Kalau mau sekalian judul, bisa lanjut ketik:

Buatkan juga judul artikel yang menarik

3. Hasilnya bisa langsung kamu **Copy** atau **Save** untuk dipakai di blog.

Pseudocode (versi GUI)

[Chat Box] -> "Buat artikel 3 paragraf tentang Tips Bermain Mobile Legends untuk Pemula"

[Output] -> Draft artikel 3 paragraf

[Chat Box] -> "Buatkan juga judul artikel yang menarik"

[Output] -> Judul keren untuk artikel

Jadi di GUI, kamu nggak perlu coding, cukup **tulis prompt seperti ngobrol di chat**.

Kasus 3: Latihan Bahasa Inggris

Bayangin kamu ingin latihan **ngobrol bahasa Inggris**, tapi masih malu sama teman. Tenang, AI di **OpenWebUI** bisa jadi partner ngobrol, bahkan bisa langsung **membetulkan kalimatmu**.

Langkah di OpenWebUI (GUI)

1. **Buka OpenWebUI** di browser.
Di **Chat Box**, ketik kalimatmu, misalnya:

Correct my English sentence: I goes to school yesterday

2. Tekan **Enter**.
AI akan memberikan versi yang benar, misalnya:

"I went to school yesterday."

3. (Opsional) Kamu bisa lanjut latihan dengan mengetik kalimat lain, atau minta AI jelaskan kenapa salah.

Pseudocode (versi GUI)

[Chat Box] -> "Correct my English sentence: I goes to school yesterday"

[Output] -> "I went to school yesterday."

Jadi di GUI, cukup **ketik kalimat + instruksi**, dan AI langsung kasih koreksi.

Kasus 4: Membuat Kuis Dadakan

Bayangin kamu mau **ngetes temanmu tentang materi Biologi**. Daripada bikin soal manual satu-satu, kamu bisa minta bantuan AI di **OpenWebUI**.

Langkah di OpenWebUI (GUI)

1. **Buka OpenWebUI** lewat browser.

Di **Chat Box**, ketik instruksi:

```
Buat 5 soal pilihan ganda tentang sistem pernapasan manusia
```

2. Tekan **Enter**.

AI akan menampilkan 5 soal pilihan ganda lengkap dengan opsi jawaban.

(Opsional) Kalau mau sekaligus dengan kunci jawaban, cukup ketik lanjutan:

```
Sertakan juga kunci jawaban
```

Pseudocode (versi GUI)

```
[Chat Box] -> "Buat 5 soal pilihan ganda tentang sistem pernapasan manusia"
```

```
[Output] -> 5 soal pilihan ganda ditampilkan
```

Jadi, cukup **tulis prompt di chat**, AI langsung jadi “pembuat soal otomatis” buatmu.

Kasus 5: Perencana Proyek

Bayangin kamu punya ide bikin **aplikasi kantin digital untuk sekolah**.

Tapi bingung harus mulai dari mana.

Tenang, AI di **OpenWebUI** bisa bantu kasih **langkah-langkah detail** untuk memulai proyekmu.

Langkah di OpenWebUI (GUI)

1. **Buka OpenWebUI** lewat browser.

Di **Chat Box**, ketik instruksi:

```
Buat langkah-langkah detail untuk memulai proyek Aplikasi Kantin Digital
```

2. Tekan **Enter**.

AI akan menampilkan tahapan yang bisa diikuti, mulai dari analisis kebutuhan, desain fitur, sampai implementasi.

(Opsional) Kalau mau lebih spesifik, kamu bisa lanjutkan prompt, misalnya:

```
Sertakan juga contoh teknologi open source yang bisa dipakai
```

Pseudocode (versi GUI)

```
[Chat Box] -> "Buat langkah-langkah detail untuk memulai proyek Aplikasi Kantin Digital"
```

```
[Output] -> Rencana tahapan proyek ditampilkan
```

Jadi di GUI, cukup **tulis prompt seperti ngobrol biasa**, dan AI langsung kasih roadmap proyekmu.

Ringkasan

OpenWebUI bukan hanya untuk chat seru, tapi juga:

- Bisa bantu **belajar** (ringkasan buku, kuis).
- Bisa bantu **kreatif** (artikel, script video).
- Bisa bantu **proyek nyata** (planning aplikasi, analisis data).

Dan yang keren, semua ini **jalan di laptop lokal** pakai teknologi **open source**: *Python, Pandas, Machine Learning, Keras, Neural Network*.

Soal Latihan & Pembahasan

Soal 1: Fungsi Utama OpenWebUI

Apa fungsi utama dari OpenWebUI saat dipasang di komputer yang sudah ada Ollama?

Pseudocode:

```
ask_ai("Apa fungsi utama dari OpenWebUI?")
```

Jawaban:

Fungsi utamanya adalah memberikan *antarmuka web* yang ramah pengguna untuk berinteraksi dengan AI (LLM).

Pembahasan:

Tanpa OpenWebUI, kita harus ketik perintah di *Terminal*. Dengan OpenWebUI, kita bisa chat lewat browser dengan tampilan mirip ChatGPT. Jadi lebih gampang dipakai oleh siapa pun.

Soal 2: Mengakses OpenWebUI Lewat Browser

Setelah OpenWebUI berhasil diinstal lewat *Docker*, URL apa yang digunakan untuk mengaksesnya di browser?

Pseudocode:

```
url = "http://localhost:8080"  
open_browser(url)
```

Jawaban:

URL yang digunakan adalah <http://localhost:8080> (atau port 3000 kalau dikonfigurasi berbeda).

Pembahasan:

OpenWebUI berjalan di komputer lokal. Jadi kita cukup buka browser dan akses `localhost` dengan port yang sesuai.

Soal 3: Instalasi Model Baru di Ollama**Soal:**

Jika kamu ingin menggunakan model *Llama3* di OpenWebUI, tapi model itu belum ada di Ollama, perintah apa yang harus dijalankan di Terminal?

Pseudocode:

```
ollama pull llama3
```

Jawaban:

Perintahnya adalah `ollama pull llama3`.

Pembahasan:

Ollama harus mendownload model dulu sebelum dipakai. Setelah berhasil ditarik (*pull*), model bisa langsung muncul di daftar OpenWebUI.

Soal 4: Manfaat Fitur Riwayat Chat**Soal:**

Kenapa fitur *chat history* (riwayat percakapan) di **OpenWebUI** bermanfaat?

Langkah di OpenWebUI (GUI)

1. Buka OpenWebUI di browser.
Di Chat Box, ketik pertanyaan:

```
Apa manfaat fitur chat history di OpenWebUI?
```

2. Tekan **Enter**.

AI akan menjawab bahwa riwayat chat berguna untuk menyimpan semua percakapan dengan AI agar bisa dibuka kembali kapan saja.

Pseudocode (versi GUI)

```
[Chat Box] -> "Apa manfaat fitur chat history di OpenWebUI?"  
[Output]   -> "Karena semua percakapan dengan AI tersimpan,  
sehingga bisa dibuka kembali kapan saja."
```

Jawaban:

Karena semua percakapan dengan AI tersimpan, sehingga bisa dibuka kembali kapan saja.

Pembahasan:

Fitur ini sangat berguna untuk pelajaran sekolah atau proyek. Misalnya, kalau AI sudah pernah membuat ringkasan bab buku atau draft laporan, kamu tidak perlu mengetik ulang perintahnya. Cukup buka **riwayat chat sebelumnya**, dan lanjutkan percakapan dari sana.

Dengan begitu, **OpenWebUI** jadi seperti *notebook digital cerdas* yang menyimpan semua ide dan hasil kerjamu.

Soal 5: Integrasi RAG (Retrieval-Augmented Generation)

Soal:

Kamu ingin AI membaca file PDF berisi materi sekolah. Fitur apa yang harus digunakan di **OpenWebUI**?

Langkah di OpenWebUI (GUI)

1. **Buka OpenWebUI** lewat browser.
2. Klik tombol Upload File dan pilih `materi_sekolah.pdf`. Setelah file muncul di area chat, ketik di Chat Box:

`Ringkas isi PDF ini`

3. Tekan **Enter**. AI akan memproses isi PDF dan menampilkan ringkasan berdasarkan konten di dalamnya.

Pseudocode (versi GUI)

```
[Upload File] -> "mater_i_sekolah.pdf"
[Chat Box]    -> "Ringkas isi PDF ini"
[Output]      -> Ringkasan isi file ditampilkan
```

Jawaban:

Gunakan fitur **Integrasi RAG (Retrieval-Augmented Generation)**.

Pembahasan:

Dengan fitur **RAG**, AI bisa **mengakses dan memahami dokumen lokal** yang kamu unggah.

Artinya, AI tidak hanya menjawab berdasarkan *pengetahuan model* saja, tetapi juga **berdasarkan isi file PDF** yang kamu berikan.

Ini sangat berguna untuk pelajaran sekolah, laporan, atau membaca buku digital dengan cepat.

Jadi di **OpenWebUI**, cukup *upload PDF + ketik prompt*, dan AI akan langsung bantu meringkas isi dokumenmu.

Ringkasan

- **OpenWebUI** = wajah ramah AI (mudah dipakai lewat browser).
- Bisa diakses lewat **localhost:8080**.
- Model baru harus di-*pull* dulu di Ollama.
- Fitur **chat history** membantu mengulang percakapan lama.
- **RAG** membuat AI bisa membaca dokumen kita.

Fun Facts!

- **OpenWebUI** adalah proyek **open source** yang dibuat oleh komunitas. Siapa saja bisa berkontribusi untuk membuatnya lebih baik.
- Interaksi dengan model LLM di OpenWebUI akan terasa lebih cepat karena tidak ada latensi jaringan.
- **RAG (Retrieval-Augmented Generation)** lokal di OpenWebUI akan mengubah AI kalian dari "anak baru" menjadi "ahli" karena ia bisa "membaca" semua dokumen penting kalian!

Tips Belajar atau Menghafal Cepat

- **Kerjakan Proyek Kecil:** Buat tugas sederhana, seperti meminta AI untuk membuatkan ringkasan atau kuis.
- **Jelajahi Fiturnya:** Klik setiap tombol dan menu di OpenWebUI. Lihat apa saja yang bisa kamu lakukan.
- **Buat Catatan:** Buat catatan di buku, "OpenWebUI itu kayak website-nya Ollama."

Aktivitas Siswa

Aktivitas 1: Instalasi dan Akses OpenWebUI

Tujuan:

Siswa belajar menjalankan server AI lokal dan mengaksesnya lewat browser.

Langkah-langkah Instalasi (CLI)

```
# 1. Pastikan Docker sudah terpasang
docker --version

# 2. Jalankan container OpenWebUI
docker run -d -p 3000:8080 \
  -v open-webui:/app/backend/data \
  --name open-webui \
  ghcr.io/open-webui/open-webui:main

# 3. Cek apakah container sudah berjalan
docker ps
```



```
# 4. Tampilkan log container untuk memastikan server aktif
docker logs -f open-webui
```

Akses di Browser

Buka browser dan masuk ke alamat:

<http://localhost:8080>

Cek Koneksi

Gunakan curl untuk memastikan server merespons:

```
curl -I http://localhost:8080
```

Jika berhasil, kamu akan melihat status seperti:

```
HTTP/1.1 200 OK
```

Penjelasan

- `-d` artinya **run di background**.
- `-p 3000:8080` membuka port agar bisa diakses dari browser.
- `-v open-webui:/app/backend/data` menyimpan data agar tidak hilang.
- Gunakan `docker stop open-webui` untuk menghentikan server, dan `docker start open-webui` untuk menjalankannya lagi.

Aktivitas 2: Chat Sederhana dengan AI

Tujuan:

Siswa mencoba **berinteraksi langsung dengan model AI** (seperti *LLaMA 3*) yang dijalankan lewat **OpenWebUI**.

Langkah di OpenWebUI (GUI)

1. **Buka browser**, lalu masuk ke:
`http://localhost:8080`
2. Setelah halaman OpenWebUI terbuka, **pilih model AI** yang ingin digunakan, misalnya:
`llama3`
3. Di kolom Chat Box, ketik pertanyaanmu, contohnya:

```
Apa itu Machine Learning?
```

4. Tekan **Enter**.
5. AI akan menampilkan jawaban langsung di layar — kamu bisa lanjutkan percakapan seperti chatting biasa.

Pseudocode (versi GUI)

```
[Chat Box] -> "Apa itu Machine Learning?"
[Output]   -> Penjelasan tentang Machine Learning ditampilkan
```

Catatan Tambahan

- Pastikan **Ollama** sudah berjalan di sistem kamu.
- OpenWebUI akan otomatis mendeteksi model yang tersedia (misalnya **llama3**, **mistral**, atau **gemma**).
- Kamu bisa berganti model kapan saja lewat menu pilihan model di pojok atas.

Dengan cara ini, siswa bisa **belajar konsep AI secara interaktif**, tanpa menulis satu baris kode.

Aktivitas 3: Ringkasan Buku Sekolah dengan RAG

Tujuan:

Siswa belajar memanfaatkan **AI untuk membaca dan meringkas file PDF materi pelajaran** secara otomatis.

Langkah di OpenWebUI (GUI)

1. **Buka OpenWebUI** di browser:
`http://localhost:8080`
2. Klik tombol Upload File, lalu pilih file:
`materi_fisika.pdf`
Setelah file muncul di area chat, ketik di Chat Box:
`Ringkas Bab 1 menjadi 5 poin singkat`
3. Tekan **Enter**.
AI akan membaca isi PDF dan menampilkan ringkasan dalam 5 poin.
4. **Catat hasil ringkasan** tersebut untuk dibahas dalam diskusi kelompok.

Pseudocode (versi GUI)

```
[Upload File] -> "mater_i_fisika.pdf"
[Chat Box]    -> "Ringkas Bab 1 jadi 5 poin singkat"
[Output]      -> Ringkasan 5 poin ditampilkan
```

Catatan Tambahan

- Pastikan file dalam format **PDF teks** (bukan gambar).
- Fitur ini menggunakan **Integrasi RAG (Retrieval-Augmented Generation)** agar AI bisa memahami isi dokumen yang kamu upload.
- Kamu bisa lanjutkan percakapan, misalnya:
`Jelaskan poin nomor 3 lebih detail`

Dengan **OpenWebUI**, siswa bisa belajar mandiri dengan AI yang membaca dan menjelaskan isi buku sekolah secara cepat dan interaktif.

Aktivitas 4: Membuat Kuis Otomatis

Tujuan:

Siswa belajar memanfaatkan **AI di OpenWebUI** untuk membuat soal otomatis berdasarkan materi pelajaran.

Langkah di OpenWebUI (GUI)

1. **Buka OpenWebUI** lewat browser:

```
http://localhost:8080
```

2. **Pilih mata pelajaran**, misalnya **Biologi**.
3. Di **Chat Box**, ketik perintah:

```
Buat 5 soal pilihan ganda tentang sistem pernapasan
```

4. Tekan **Enter**.
AI akan menampilkan 5 soal pilihan ganda lengkap dengan opsi jawaban.
5. **Catat soal yang dihasilkan**, lalu **uji coba ke temanmu** atau diskusikan bersama kelompok belajar.

Pseudocode (versi GUI)

```
[Chat Box] -> "Buat 5 soal pilihan ganda tentang sistem pernapasan"  
[Output]   -> 5 soal pilihan ganda ditampilkan
```

Catatan Tambahan

Kamu bisa tambahkan instruksi tambahan, misalnya:

```
Sertakan juga kunci jawaban dan penjelasan singkat.
```

- Fitur ini bisa digunakan untuk semua mata pelajaran: Fisika, Kimia, Matematika, Sejarah, dan lainnya.
- Cocok untuk latihan membuat **bank soal otomatis** atau simulasi ujian.

Dengan **OpenWebUI**, siswa bisa jadi **guru mini digital**, melatih AI membuat soal sesuai topik pelajaran dengan cepat dan interaktif.

Aktivitas 5: Analisis Data Sederhana dengan Pandas

Tujuan:

Siswa belajar menggabungkan kemampuan **OpenWebUI** dengan **data CSV** untuk melakukan **analisis nilai siswa** tanpa perlu menulis kode Python secara langsung.

Langkah di OpenWebUI (GUI)

- **Siapkan file data** bernama `nilai_siswa.csv`
Contoh isi file:

```
Nama,Nilai
Ayu,80
Budi,75
Cici,90
```

- **Buka OpenWebUI** di browser:
`http://localhost:8080`
- Klik **Upload File**, lalu pilih `nilai_siswa.csv`.
- Setelah file muncul di area chat, ketik di **Chat Box**:
`Hitung rata-rata nilai siswa dari file ini`
- Tekan **Enter**.
AI akan membaca isi CSV dan menampilkan hasil rata-rata nilai.
- (Opsional) **Bandingkan hasilnya** dengan perhitungan manual atau dengan Python Pandas di kelas.

Pseudocode (versi GUI)

```
[Upload File] -> "nilai_siswa.csv"
[Chat Box]    -> "Hitung rata-rata nilai siswa dari file ini"
[Output]      -> "Rata-rata nilai siswa adalah 81.67"
```

Catatan Tambahan

- File CSV harus berisi **header kolom** seperti `Nama` dan `Nilai`.
- AI di OpenWebUI bisa membaca, menghitung, dan menjelaskan hasil perhitungan dengan bahasa alami.
- Kamu bisa lanjutkan dengan perintah lain, seperti:
`Siapa siswa dengan nilai tertinggi?`
- atau
`Urutkan nilai dari tertinggi ke terendah.`

Dengan **OpenWebUI**, siswa bisa **menganalisis data sekolah secara interaktif**, tanpa harus menulis kode panjang seperti di Python — cukup *upload file dan tanya AI!*

Ringkasan Aktivitas

- Aktivitas 1 → Instalasi OpenWebUI.
- Aktivitas 2 → Chat dengan AI.
- Aktivitas 3 → Ringkas buku sekolah pakai PDF.
- Aktivitas 4 → Membuat kuis otomatis.
- Aktivitas 5 → Analisis data nilai siswa dengan Pandas.

Semua berbasis **open source**: *Python, Pandas, Machine Learning, Keras, Neural Network*.

Rangkuman

Di bab ini kita sudah belajar bahwa **OpenWebUI adalah wajah ramah dari AI lokal**. Kalau *Ollama* itu otaknya, maka OpenWebUI adalah **antarmuka web** yang membuat ngobrol dengan AI jadi gampang. Tidak perlu lagi ribet ketik perintah di *Terminal*. Tinggal buka browser, pilih model, lalu ajukan pertanyaan. Jadi, **AI terasa lebih hidup, interaktif, dan menyenangkan**.

OpenWebUI juga punya banyak fitur penting. Ada **riwayat percakapan** yang menyimpan chat lama, ada **dukungan multi-model** yang memudahkan kita ganti otak AI, dan ada **Integrasi RAG** yang bikin AI bisa membaca file PDF, Word, atau teks dari laptop kita. Dengan begitu, AI tidak hanya pintar secara umum, tapi juga bisa jadi **asisten sekolah pribadi** yang membantu meringkas pelajaran, membuat kuis, bahkan menjawab soal ujian. Semua ini berjalan di laptop lokal dengan teknologi *open source* seperti *Python*, *Pandas*, *Machine Learning*, *Keras*, dan *Neural Network*.

Jadi, inti pelajaran dari bab ini adalah: **OpenWebUI membuat interaksi dengan AI lokal menjadi mudah, cepat, dan bermanfaat**. Dengan teknologi ini, siswa bisa belajar lebih seru, membuat proyek lebih kreatif, dan berlatih menghadapi dunia kerja di masa depan. Ingat, semua ini bisa dijalankan **gratis** dan **offline**, cukup dengan menguasai dasar *Linux server*. Maka, jangan ragu untuk mencoba langsung dan rasakan bagaimana serunya punya **asisten AI pribadi** di laptopmu.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. OpenWebUI hanya bisa digunakan jika ada koneksi internet.

Jawaban: Salah

2. Antarmuka OpenWebUI mirip dengan ChatGPT.

Jawaban: Benar

3. Semua percakapan di OpenWebUI bisa tersimpan dan dibuka kembali.

Jawaban: Benar

4. Untuk menambahkan model baru di Ollama, kita perlu mengetik perintah `ollama pull <nama_model>`.

Jawaban: Benar

5. Integrasi RAG di OpenWebUI tidak bisa membaca file PDF.

Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Fungsi utama OpenWebUI adalah...

- a. Menyediakan jaringan internet
- b. Antarmuka web untuk AI lokal
- c. Membuat database otomatis
- d. Menyimpan file musik

Jawaban: b

2. Perintah untuk menginstal OpenWebUI dengan Docker adalah...

- a. `docker start openwebui`
- b. `docker pull ollama`
- c. `docker run -d -p 3000:8080 ...`
- d. `pip install openwebui`

Jawaban: c

3. URL default untuk membuka OpenWebUI adalah...

- a. `http://localhost:8080`
- b. `http://127.0.0.1:5000`
- c. `http://openwebui.com`
- d. `http://localhost:3001`

Jawaban: a

4. OpenWebUI bisa mendukung banyak model LLM. Ini disebut...

- a. Multi-threading
- b. Multi-model
- c. Multi-user
- d. Multi-tasking

Jawaban: b

5. Jika model *Llama3* belum terinstal, perintah yang digunakan adalah...

- a. `ollama download llama3`
- b. `ollama start llama3`
- c. `ollama pull llama3`
- d. `docker pull llama3`

Jawaban: c

6. Fitur yang bisa menghubungkan OpenWebUI dengan dokumen lokal adalah...

- a. RAG
- b. API
- c. CLI
- d. GPU

Jawaban: a

7. Jika kita menutup browser lalu membuka kembali OpenWebUI, percakapan sebelumnya...
- a. Hilang
 - b. Masih ada di riwayat
 - c. Hanya tersisa sebagian
 - d. Harus diunduh dulu

Jawaban: b

8. Tampilan OpenWebUI dibuat agar mirip dengan...
- a. WhatsApp
 - b. Telegram
 - c. ChatGPT
 - d. Facebook

Jawaban: c

9. Keuntungan utama OpenWebUI dibanding terminal adalah...
- a. Lebih hemat kuota
 - b. Ada tombol dan riwayat chat
 - c. Bisa langsung menginstal sistem operasi
 - d. Bisa main game

Jawaban: b

10. OpenWebUI bersifat...
- a. Berbayar
 - b. Gratis dan *open source*
 - c. Demo saja
 - d. Hanya untuk perusahaan

Jawaban: b

C. Isian Singkat (5 Soal)

1. OpenWebUI adalah sebuah _____ untuk berinteraksi dengan Ollama.

Jawaban: antarmuka web

2. Perintah untuk menambahkan model baru di Ollama adalah `ollama ____`
`<nama_model>`.

Jawaban: pull

3. Fitur di OpenWebUI yang bisa membaca file PDF disebut _____.

Jawaban: RAG

4. Riwayat percakapan di OpenWebUI akan tetap _____ walaupun browser ditutup.

Jawaban: tersimpan

5. URL untuk membuka OpenWebUI biasanya adalah [http://_____](http://localhost:8080).

Jawaban: localhost:8080

D. Soal Eksplorasi (3 Soal)

Soal 1:

Eksperimen Ringkasan Buku Sekolah

Bayangkan kamu punya file PDF berisi bab 1 buku Matematika. Jelaskan bagaimana kamu bisa menggunakan OpenWebUI untuk membuat ringkasan poin-poinnya.

Contoh Jawaban:

- Upload file [matematika_bab1.pdf](#) ke OpenWebUI.
- Ketik perintah: *"Tolong buat ringkasan 5 poin dari bab 1."*
- AI akan memberikan ringkasan singkat berisi isi bab tersebut.

Soal 2:

Membuat Ide Konten Kreatif

Kamu diminta membuat ide artikel blog tentang game. Gunakan OpenWebUI untuk membantu. Jelaskan langkah-langkahmu.

Contoh Jawaban:

- Buka OpenWebUI.
- Ketik: *"Buatkan draft artikel blog tentang 3 game populer tahun ini dengan judul yang menarik."*
- AI menghasilkan draft artikel beserta judul.

Soal 3:

Menguji Pemahaman Temanmu

Kamu ingin membuat kuis tentang Fisika untuk temanmu. Bagaimana kamu menggunakan OpenWebUI?

Contoh Jawaban:

- Masuk ke OpenWebUI.
- Ketik: *"Buat 5 soal pilihan ganda tentang Hukum Newton lengkap dengan jawabannya."*
- AI menghasilkan soal kuis yang bisa langsung digunakan.

Tugas Proyek

Proyek 1: Proyek Analisis Lagu Favorit

Soal:

Buat proyek yang meminta AI di **OpenWebUI** untuk **menganalisis lirik lagu favoritmu** dan **membuat ringkasan isi lagu dalam 3 kalimat singkat**.

Langkah di OpenWebUI (GUI)

- Siapkan file teks lirik lagu dalam format .txt, misalnya:

```
lagu_favorit.txt
```

Contoh isi file:

```
Aku ingin begini, aku ingin begitu,  
Ingin ini, ingin itu banyak sekali.  
Semua, semua, semua dapat dicapai...
```

- Buka OpenWebUI lewat browser:
`http://localhost:8080`
- Klik Upload File, lalu pilih lagu_favorit.txt.
- Setelah file muncul di area chat, ketik di Chat Box:

```
Ringkas isi lirik ini dalam 3 kalimat singkat
```

- Tekan Enter.
AI akan membaca isi lirik dan memberikan ringkasan makna lagu.

Pseudocode (versi GUI)

```
[Upload File] -> "lagu_favorit.txt"  
[Chat Box]    -> "Ringkas isi lirik ini dalam 3 kalimat singkat"  
[Output]     -> Ringkasan isi lagu ditampilkan
```

Contoh Hasil Jawaban

Ringkasan:

Lagu ini bercerita tentang keinginan dan impian seseorang untuk mencoba banyak hal baru.

Pesannya adalah semangat untuk belajar dan tidak menyerah.

Nada dan liriknya menggambarkan keceriaan masa kecil yang penuh harapan.

Dengan **OpenWebUI**, kamu bisa menjadikan proyek ini latihan seru untuk **analisis teks kreatif**, sekaligus belajar memahami makna lagu favoritmu secara lebih dalam.

Proyek 2: Proyek Kamus Istilah Jurusan

Soal:

Buat **kamus istilah dari jurusanmu** (contoh: TKJ, RPL, Multimedia).

AI di **OpenWebUI** akan membantu membuat daftar kata beserta **penjelasan sederhana** untuk tiap istilah.

Langkah di OpenWebUI (GUI)

- Buat file teks baru bernama `istilah_jurusan.txt`
Isi dengan sekitar 10 istilah dari jurusanmu, misalnya:

```
IP Address
Router
Python
HTML
Database
Cloud
Server
AI
IoT
API
```

- Buka OpenWebUI di browser:
`http://localhost:8080`
- Klik Upload File, lalu pilih `istilah_jurusan.txt`.
- Setelah file muncul di area chat, ketik di Chat Box:

```
Buat kamus sederhana dari istilah-istilah berikut
```

- Tekan Enter.
AI akan menampilkan daftar istilah beserta penjelasan singkat dalam bahasa yang mudah dipahami.

Pseudocode (versi GUI)

```
[Upload File] -> "istilah_jurusan.txt"
[Chat Box]    -> "Buat kamus sederhana dari
istilah-istilah berikut"
[Output]      -> Daftar istilah + penjelasan sederhana
```

Contoh Hasil Jawaban

Kamus Sederhana:

- **IP Address:** Alamat unik untuk mengidentifikasi perangkat di jaringan.

- **Router:** tool untuk menghubungkan beberapa jaringan agar bisa saling bertukar data.
- **Python:** Bahasa pemrograman yang mudah dibaca dan dipelajari.
- **HTML:** Bahasa dasar untuk membuat halaman web.
- **Database:** Tempat menyimpan dan mengelola data digital.

Dengan **OpenWebUI**, siswa bisa membuat **kamus istilah jurusan** sendiri secara otomatis — berguna untuk belajar, ujian, atau proyek portofolio sekolah.

Proyek 3: Proyek Rekomendasi Jajanan Sehat

Soal:

Buat **tabel jajanan kantin**, lalu minta AI di **OpenWebUI** memberi rekomendasi **jajanan yang sehat dan rendah kalori**.

Langkah di OpenWebUI (GUI)

- Buat file CSV bernama `jajanan.csv`
Isi datanya seperti ini:

```
Nama,Harga,Kalori
Keripik,5000,300
Salad,7000,150
Donat,6000,400
Susu UHT,8000,120
Roti Gandum,9000,180
```

- Buka OpenWebUI lewat browser:
<http://localhost:8080>
- Klik Upload File, lalu pilih `jajanan.csv`.
- Setelah file muncul di area chat, ketik di Chat Box:

```
Tolong pilih jajanan sehat dengan kalori di bawah 200
```

- Tekan Enter.
- AI akan membaca isi tabel dan menampilkan daftar jajanan sehat berdasarkan nilai kalornya.
- (Opsional) Kamu juga bisa lanjutkan prompt:

```
Buatkan tabel perbandingan antara jajanan sehat dan tidak  
sehat
```

Pseudocode (versi GUI)

```
[Upload File] -> "jajanan.csv"
[Chat Box]    -> "Tolong pilih jajanan sehat dengan kalori di  
bawah 200"
[Output]      -> Daftar jajanan sehat ditampilkan
```

Contoh Hasil Jawaban

Rekomendasi Jajanan Sehat:

- Salad — 150 kalori
- Susu UHT — 120 kalori
- Roti Gandum — 180 kalori

Jajanan Kurang Sehat:

- Keripik — 300 kalori
- Donat — 400 kalori

Dengan **OpenWebUI**, siswa bisa belajar **analisis data sederhana dari file CSV**, sambil memahami pentingnya memilih makanan sehat di kantin sekolah.

Proyek 4: Proyek Ide Konten TikTok

Soal:

Gunakan AI di **OpenWebUI** untuk membuat **3 ide konten TikTok** bertema **teknologi open source** untuk anak **SMK**.

Langkah di OpenWebUI (GUI)

- Buka OpenWebUI lewat browser:
<http://localhost:8080>
- Di Chat Box, ketik perintah:

```
Buat 3 ide konten TikTok tentang teknologi open source
untuk anak SMK
```

- Tekan Enter.
AI akan menampilkan beberapa ide kreatif yang bisa dijadikan bahan konten video.
- (Opsional) Kamu bisa lanjutkan prompt, misalnya:

```
Tambahkan juga deskripsi singkat dan durasi tiap konten
```

Pseudocode (versi GUI)

```
[Chat Box] -> "Buat 3 ide konten TikTok tentang teknologi open source
untuk anak SMK"
[Output]    -> 3 ide konten TikTok ditampilkan
```

Contoh Hasil Jawaban AI

1. **Tutorial Singkat Pakai Python:**
Bikin video 30 detik cara membuat kalkulator sederhana dengan Python.
2. **Meme Programmer Begadang:**
Konten lucu tentang kebiasaan anak RPL yang ngoding sampai tengah malam.

3. Fakta Unik Tentang AI Offline:

Jelaskan bagaimana sekolah bisa punya chatbot tanpa internet dengan Ollama + OpenWebUI.

Dengan **OpenWebUI**, siswa bisa berlatih **menciptakan ide konten digital edukatif**, sambil memahami teknologi open source dengan cara yang seru dan relevan bagi anak SMK.

Proyek 5: Proyek Prediksi Nilai Siswa

Soal:

Buat simulasi **prediksi nilai ujian akhir siswa** berdasarkan **nilai tugas**, menggunakan **AI di OpenWebUI** (tanpa menulis kode Python).

Langkah di OpenWebUI (GUI)

- Buat file CSV bernama `nilai_siswa.csv`
Contoh isi file:

```
Nama,Tugas,Ujian
Ayu,80,85
Budi,60,65
Cici,70,75
Deni,90,?
```

- Buka OpenWebUI lewat browser:
<http://localhost:8080>
- Klik Upload File, lalu pilih `nilai_siswa.csv`.
- Setelah file muncul di area chat, ketik di Chat Box:

```
Prediksi nilai ujian akhir berdasarkan nilai tugas
```

- Tekan Enter.
AI akan membaca data dan memberikan perkiraan nilai ujian untuk siswa yang belum memiliki nilai (misalnya Deni).
- (Opsional) Kamu bisa lanjutkan prompt:

```
Jelaskan cara kerja prediksi ini dengan bahasa sederhana.
```

Pseudocode (versi GUI)

```
[Upload File] -> "nilai_siswa.csv"
[Chat Box]    -> "Prediksi nilai ujian akhir berdasarkan nilai tugas"
[Output]      -> Hasil prediksi nilai ujian ditampilkan
```

Contoh Hasil Jawaban AI

Prediksi:

Deni (nilai tugas 90) → nilai ujian diperkirakan sekitar **95**.

Penjelasan:

AI melihat pola dari data sebelumnya (semakin tinggi nilai tugas, semakin tinggi nilai ujian).

Prediksi ini mirip seperti *Linear Regression* di Machine Learning — mencari garis hubungan antara dua nilai.

Dengan **OpenWebUI**, siswa bisa **belajar konsep Machine Learning secara praktis**: cukup *upload data CSV dan tanya AI*, tanpa perlu menulis kode apa pun.

Catatan

- Proyek 1 → Ringkas lirik lagu.
- Proyek 2 → Buat kamus istilah jurusan.
- Proyek 3 → Analisis jajanan sehat di kantin.
- Proyek 4 → Ide kreatif konten TikTok.
- Proyek 5 → Prediksi nilai siswa dengan *Machine Learning*.

Semua proyek ini sederhana tapi **menantang**, dan bisa dikerjakan siswa SMK dengan bantuan **OpenWebUI + Python + Pandas + Machine Learning**.

BAB 7: Etika AI di Bidang Teks

(Disinformasi, Hoax, Ujaran Kebencian)

Setelah kita berhasil ngobrol sama AI, sekarang kita akan masuk ke bab yang paling penting: **Etika AI**. AI itu kayak pisau, bisa dipakai buat hal baik, bisa juga buat hal jahat. Nah, di bab ini, kita akan belajar gimana caranya menggunakan AI dengan bijak, terutama untuk masalah-masalah serius seperti **disinformasi**, **hoax**, dan **ujaran kebencian**.

Tujuan Pembelajaran

- Memahami mengapa etika sangat penting dalam pengembangan AI.
- Mengenali jenis-jenis penyalahgunaan AI, seperti **disinformasi** dan **hoax**.
- Mengidentifikasi dan membedakan antara **disinformasi** dan **misinformasi**.
- Mengetahui bahaya **ujaran kebencian** yang dibuat oleh AI.
- Belajar bagaimana cara mencegah dan mengatasi masalah etika tersebut.

Peta Konsep Sederhana

Bayangin kita mau menyajikan makanan ke orang lain.

- **AI (Koki)**: Punya kemampuan super untuk membuat sesuatu (teks).
- **Bahan Makanan (Data)**: Bisa bersih (fakta), bisa kotor (hoax).
- **Penyajian (Penyebaran Informasi)**: Bisa disajikan dengan baik, bisa juga disajikan untuk merugikan orang lain.
- **Etika (Rasa Tanggung Jawab)**: Memastikan semua yang kita sajikan itu aman dan tidak beracun (tidak berbahaya).

Apersepsi: Kamu Terkena Hoax!

Pernahkah kamu menerima pesan berantai di grup WhatsApp tentang resep rahasia yang bisa menyembuhkan penyakit, tapi setelah dicari, ternyata itu bohong? Atau, kamu lihat berita di media sosial yang ternyata dibuat oleh AI untuk memecah belah?

AI punya kemampuan untuk membuat teks yang sangat meyakinkan. Ini bisa jadi bahaya kalau dipakai untuk membuat **hoax** atau **disinformasi**. Oleh karena itu, kita sebagai calon ahli AI harus tahu cara kerja dan bahayanya.

Penjelasan Teori

Kenapa Etika AI itu Penting?

AI itu seperti **pisau dapur**.

Kalau dipakai benar, membantu kita. Kalau salah, bisa melukai orang lain.

Di dunia teks, AI bisa menulis cepat, tapi juga bisa **menyebarkan salah informasi** kalau tidak diawasi.

- AI menulis teks meyakinkan dengan mudah.
- Orang cenderung percaya kalau bahasanya rapi dan “mirip berita”.
- Karena itu, **etika** jadi *rem tangan*: mencegah AI dipakai untuk merugikan.

Istilah Inti (Wajib Paham)

1) Disinformasi

Informasi **salah** yang dibuat **sengaja** untuk menipu.

- Contoh: bikin “berita” palsu untuk menjatuhkan tokoh.
- Kunci: ada **niat** menipu.

2) Misinformasi

Informasi **salah** yang **tidak sengaja** tersebar.

- Contoh: salah kutip data, terus disalin kemana-mana.
- Kunci: **tidak ada niat** menipu.

Ingat: *disinformasi* = *bohong sengaja*, *misinformasi* = *salah tanpa sadar*.

3) Hoax

“Berita” palsu yang **terlihat nyata**.

- Ciri: judul heboh, klaim “*terbukti*”, ajakan “*segera sebarkan*”.
- Bahaya: orang tertipu, keputusan salah.

4) Hate Speech (Ujaran Kebencian)

Teks yang menyerang identitas orang/kelompok (ras, agama, gender, dsb.).

- Efek: menyakiti, memicu konflik, merusak komunitas.

Kenapa AI Bisa Ikut “Salah Jalan”?

AI belajar dari *data*.

Kalau data campur sari, ada bagus ada jelek, AI bisa ikut meniru pola **jelek** juga.

- **Data Bias**: kalau data tidak seimbang, model condong ke satu sisi.
- **Tanpa Guardrail**: AI bebas menjawab apa saja → berisiko halus.
- **Tanpa Sumber**: jawaban sulit diverifikasi.

Solusi konsep (*open source*):

- **RAG** (*Retrieval-Augmented Generation*): batasi jawaban ke sumber terpercaya.
- **Ollama + OpenWebUI**: jalankan LLM lokal (privasi lebih aman).
- **Python + Pandas/scikit-learn/Keras**: cek pola teks, deteksi spam/hoax, latih *neural network* sederhana.

Prinsip Etika untuk Teks (Versi Anak SMK)

1) Cek Sumber Dulu

- Ada rujukan resmi? masuk akal? bisa ditelusuri?
- Di **RAG**, pastikan indeks berisi dokumen terpercaya (mis. Kemenkes, WHO).

2) Kenali *Trigger Words*

- Kata-kata seperti “*terbukti*”, “*100%*”, “*segera sebarkan*” sering dipakai hoax.
- Pakai **Python + Pandas** buat *filter* sederhana.

3) Hindari Kebencian

- Buat aturan dasar untuk kata kasar + *neural network* di **Keras** untuk cek konteks.
- Terapkan moderasi bertingkat: **warning** → **mute** → **ban**.

4) Jaga Keadilan (Fairness)

- Jangan cuma lihat akurasi total.
- Cek **precision/recall per kelompok**. Kalau timpang, seimbangkan data (class weight, *oversampling*).

5) Transparansi & Akuntabilitas

- Simpan *log*: dari mana sumbernya, bagaimana diputuskan.
- Kalau ragu, **jangan sebar**.

Contoh Skenario

- **Hoax Kesehatan**: judul heboh, tanpa jurnal, minta disebar. → *Stop. Verifikasi dulu.*
- **Serangan Bot Politik**: komentar seragam, jamnya bareng. → Deteksi pola duplikat + waktu.
- **Review Palsu**: bintang 5 numpuk, kalimat repetitif. → Cek kemiripan teks + lonjakan waktu.
- **Chat Game Toxic**: hina identitas. → *Rule-based* + model konteks, *warning* → *mute* → *ban*.
- **Model Bias**: akurasi tinggi, tapi minoritas dirugikan. → Audit per kelompok, *rebalance*.

Kebiasaan Baik yang Wajib Dilatih

- **Tahan jempol:** jangan asal *share*.
- **Pakai logika:** “Masuk akal nggak?”
- **Lacak sumber:** ada dokumen aslinya?
- **Utamakan RAG:** jawab dari dokumen terindeks, bukan *halu*.
- **Bahasa yang menghormati:** nol toleransi pada kebencian.

Ringkas Hafalan (Speedrun)

- *Disinformasi* = salah **sengaja**.
- *Misinformasi* = salah **tak sengaja**.
- *Hoax* = palsu, gaya berita.
- *Hate speech* = hina identitas.
- Obatnya: **RAG lokal + sumber tepercaya, filter kata kunci, moderasi bertingkat, audit fairness.**
- *Open source* tool tempur: **Python, Pandas, scikit-learn, Keras, neural network, Ollama + OpenWebUI.**

Mindset Akhir

Kita bukan cuma “pengguna AI”.

Kita adalah **penjaga gerbang informasi**.

Tugas kita: pakai AI **dengan etika**—biar teknologi membantu manusia, bukan menyakiti.

Kasus Ilustrasi

Kasus 1: “Resep Ajaib” di Grup WA — Hoax Kesehatan

Cerita singkat:

Ada pesan viral: “*Terbukti!* Minum air X menyembuhkan semua penyakit. *Segera sebar!*”

Banyak yang percaya. Padahal tidak ada bukti ilmiah.

Kenapa bahaya:

Orang bisa meninggalkan pengobatan yang benar. Nyawa taruhannya.

Tanda-tanda hoax:

- Kata-kata meyakinkan: “*terbukti, viral, segera sebar!*”.
- Tidak ada sumber ilmiah. Tidak ada tautan jurnal.

Etika:

Jangan sebar info kesehatan tanpa sumber valid. Verifikasi dulu.

Solusi teknis (Open Source):

- *Python + Pandas*: baca teks, cari kata kunci hoax.

- *RAG lokal* (Ollama + OpenWebUI): jawab hanya dari sumber terpercaya (WHO, Kemenkes yang di-index lokal).

Pseudocode:

```
DATA = muat_komentar()
HOAX_KEYWORDS = ["terbukti", "sembuh total", "segera sebar", "100%"]

for teks in DATA:
    if ada_kata_kunci(teks, HOAX_KEYWORDS):
        tandai(teks, label="potensi_hoax")
        sumber = verifikasi_ke(RAG_lokal, query=teks)
        jika sumber_tidak_valid(sumber):
            rekomendasikan("Jangan sebar. Cek ke situs resmi.")
```

Python mini:

```
kw = {"terbukti", "segera sebar", "sembuh total", "100%"}
def potensi_hoax(teks):
    t = teks.lower()
    return any(k in t for k in kw)
```

Kasus 2: “Serbu Komentar” — Disinformasi Politik dengan Bot

Cerita singkat:

Ribuan akun komentar mirip, jam kirimnya serentak, isi saling hina.

Tujuan: bikin polarisasi, memecah belah.

Kenapa bahaya:

Opini publik dimanipulasi. Debat sehat jadi rusak.

Tanda-tanda:

- Pola teks berulang.
- Waktu unggah berdekatan.
- Akun baru, *profile* kosong.

Etika:

Transparansi. Jangan pakai bot untuk menipu. Hormati ruang publik digital.

Solusi teknis (Open Source):

- *Pandas* untuk deteksi pola duplikat / *spam*.
- *Machine Learning* (Keras + *neural network*) untuk klasifikasi *spam/astroturfing*.
- *Rate-limit* dan *CAPTCHA* di platform lokal.

Pseudocode:

```

COMMENTS = muat_komentar()
clones = grup_berdasar_kesamaan_teks(COMMENTS, threshold=0.9)
pola_waktu = analisis_timestamp(COMMENTS)

if ukuran(clones) besar dan pola_waktu=serentak:
    labelkan_semua(clones, "disinformasi_bot")
    batasi(akun_terkait)

```

Python mini (sketsa kesamaan teks):

```

from difflib import SequenceMatcher
def similar(a,b): return SequenceMatcher(None,a,b).ratio()

```

Kasus 3: “Bintang 5 Palsu” — Ulasan Produk Palsu

Cerita singkat:

Sebuah toko punya ribuan ulasan “sempurna”, tapi isinya pendek dan mirip. Produk aslinya jelek.

Kenapa bahaya:

Konsumen tertipu. Uang terbuang.

Tanda-tanda:

- Pola kalimat sama: “Bagus sekali!” berulang.
- Grafik ulasan melonjak aneh di waktu tertentu.
- Banyak akun baru dengan satu ulasan.

Etika:

Jujur dalam pemasaran. Dilarang manipulasi ulasan.

Solusi teknis (Open Source):

- *Python + Pandas*: analisis teks duplikat, deteksi lonjakan anomali.
- *Keras* untuk model klasifikasi *fake review*.
- *Open source dashboards* (mis. *Streamlit*) untuk pantau metrik.

Pseudocode:

```

REVIEWS = muat_ulasan()
duplikat = hitung_kemiripan_batch(REVIEWS.teks)
anomali = deteksi_lonjakan_waktu(REVIEWS.tanggal)

if duplikat > ambang or anomali = benar:
    tandai_produk("butuh_audit_ulasan")
    sorot(akun_mencurigakan)

```

Python mini (indikator sederhana):

```

import pandas as pd
df = pd.DataFrame(reviews)

```

```
df["len"] = df["text"].str.len()
indikasi = (df["len"]<15) & df["text"].str.contains("bagus",
case=False)
```

Kasus 4: “Chat Game Jadi Toxic” — Ujaran Kebencian

Cerita singkat:

Di *in-game chat*, obrolan mulai mengarah ke hinaan kelompok. Awalnya bercanda, lalu kasar dan diskriminatif.

Kenapa bahaya:

Melukai mental. Normalisasi kebencian. Bisa merembet ke dunia nyata.

Tanda-tanda:

- *Trigger words* yang menyerang identitas (ras/agama/gender).
- Emosi meningkat cepat, balas-membalas.

Etika:

Hormati martabat semua orang. *Zero tolerance* pada kebencian.

Solusi teknis (Open Source):

- *Python* + daftar kata terlarang (kurasi hati-hati).
- Model *neural network* (Keras) untuk deteksi konteks (agar tidak *false positive*).
- Moderasi bertingkat: *warn* → *mute* → *ban*.

Pseudocode:

```
CHAT = stream_chat()
if deteksi_kebencian_rule_based(CHAT.teks) or model_NN(CHAT.teks)
== "hate":
    beri_peringatan(pemain)
    jika berulang:
        mute(pemain, durasi=15_menit)
        logkan(kejadian, bukti=CHAT.teks)
```

Python mini (rule sederhana):

```
blocked = {"bodoh", "idiot", "rasis"}
def hate_rule(teks):
    t=teks.lower()
    return any(b in t for b in blocked)
```

Kasus 5: “Data Miring, Hasil Menyesatkan” — Misinformasi karena Bias Data

Cerita singkat:

Kita latih model *sentiment* pakai data yang tidak seimbang.
Komentar dari satu kelompok hampir tidak ada.
Model jadi *bias* dan sering salah.

Kenapa bahaya:

Keputusan tidak adil. Kelompok tertentu dirugikan.

Tanda-tanda:

- *Accuracy* tinggi, tapi buruk di kelompok minoritas.
- *Confusion matrix* timpang.

Etika:

Fairness. Evaluasi per kelompok. Perbaiki dataset. Transparansi.

Solusi teknis (Open Source):

- *Pandas* untuk cek distribusi label/kelompok.
- *Scikit-learn* untuk *stratified split*, *class weights*.
- *Keras* untuk *rebalancing* saat training.

Pseudocode:

```
DATA = muat_dataset()
cek_distribusi(DATA, by="kelompok")

if tidak_seimbang:
    DATA = oversample_minority(DATA)
    X_train, X_test, y_train, y_test = stratified_split(DATA)

model = latih_keras(X_train, y_train, class_weight="balanced")
eval = evaluasi_per_kelompok(model, X_test, y_test)
laporkan(eval, metrik=["precision", "recall", "f1"])
```

Python mini (cek sebaran):

```
import pandas as pd
df.groupby("kelompok")["label"].value_counts(normalize=True)
```

Mini “Checklist Etika” yang Praktis

- **Cek Sumber:** ada referensi jelas? *Masuk akal?*
- **Cek Pola:** teks mirip-mirip, waktu posting aneh?
- **Cek Kata Kunci Berbahaya:** “*terbukti, segera sebarkan*”, hinaan identitas.
- **Cek Keadilan Model:** uji per kelompok, bukan rata-rata doang.
- **Default to Trustworthy Data:** pakai *RAG lokal* + sumber tepercaya.

Catatan Teknis (Open Source Stack)

- **Bahasa & Data:** *Python, Pandas*.
- **ML & NN:** *scikit-learn, Keras (TensorFlow)*.
- **Aplikasi:** *Streamlit* (panel moderasi/monitoring).
- **LLM Lokal:** *Ollama + OpenWebUI* (privasi lebih aman).
- **RAG:** *FAISS/Chroma* untuk batasi jawaban pada sumber terpercaya.

Soal Latihan & Pembahasan

Soal 1: Deteksi “kata pemicu” hoax kesehatan

Soal:

Kamu dapat teks: “**TERBUKTI!** Ramuan X menyembuhkan semua penyakit.
SEGERA SEBARKAN!”

Tugasmu:

1. Tandai apakah teks ini *potensi hoax* atau tidak.
2. Jelaskan langkah etis yang harus dilakukan.

Jawaban singkat:

1. Potensi hoax.
2. Jangan sebar. Verifikasi ke sumber terpercaya dulu.

Pembahasan:

Kata seperti “*terbukti*”, “*segera sebar*kan” sering dipakai di hoax. Etisnya, **hentikan sebaran**, cek sumber (mis. situs resmi). Jika salah, **melaporkan**.

Pseudocode:

```
HOAX_WORDS = ["terbukti", "segera sebar", "100%", "menyembuhkan semua"]

function potensi_hoax(teks):
    t = lower(teks)
    for w in HOAX_WORDS:
        if w in t:
            return True
    return False

if potensi_hoax(teks):
    tindakan = ["jangan_sebar", "cek_sumber_resmi", "laporkan"]
```

Mini Python (opsional):

```
kw = {"terbukti", "segera sebar", "100%", "menyembuhkan semua"}
is_hoax = any(k in "TERBUKTI! ... SEGERA SEBARKAN!".lower() for k in kw)
```

Soal 2: Disinformasi vs Misinformasi (pilih kategori)

Soal:

- A. Seorang kreator sengaja membuat berita palsu untuk menjatuhkan lawan politik.
- B. Seorang siswa salah salin angka statistik. AI lalu menjadikan angka itu sebagai rujukan.

Tentukan: **A** dan **B** termasuk *disinformasi* atau *misinformasi*?

Jawaban singkat:

A = Disinformasi.

B = Misinformasi.

Pembahasan:

Disinformasi = salah **sengaja**.

Misinformasi = salah **tanpa sengaja**.

Bedanya ada pada **niat**.

Pseudocode:

```
function kategorikan(kasus):  
    if niat_sengaja(kasus) == True:  
        return "disinformasi"  
    else:  
        return "misinformasi"  
  
print(kategorikan(A)) -> disinformasi  
print(kategorikan(B)) -> misinformasi
```

Soal 3: Moderasi ujaran kebencian di *in-game chat*

Soal:

Rancang alur moderasi untuk *chat* game. Alur harus:

- Deteksi cepat kata kasar.
 - Beri peringatan dulu.
 - Jika mengulang, **mute** sementara.
 - Simpan bukti untuk audit.
- Tuliskan alur dan *pseudocode* singkatnya.

Jawaban singkat (alur):

Deteksi → Peringatan → Ulang? *Mute 15 menit* → Ulang lagi? *Ban* → Simpan log.

Pembahasan:

Kita perlu *rule-based* cepat untuk *trigger words*. Lalu model *neural network* (Keras) untuk konteks agar tidak *false positive*. Bertingkat itu adil dan edukatif.

Pseudocode:

```
BLOCKLIST = ["bodoh", "idiot", "rasis"]
```



```

function is_hate(teks):
    if ada_kata(teks, BLOCKLIST): return True
    if model_NN(teks) == "hate": return True
    return False

on message(teks, user):
    if is_hate(teks):
        warn(user)
        if pelanggaran_berulang(user):
            mute(user, 15_menit)
        simpan_log(user, teks, waktu)

```

Mini Python (opsional):

```

blocked = {"bodoh", "idiot", "rasis"}
def rule(teks): return any(b in teks.lower() for b in blocked)

```

Soal 4: Akurasi tinggi tapi tidak adil (bias data)

Soal:

Model *sentiment* kamu akurasinya 92%. Tapi untuk kelompok minoritas, *recall* hanya 40%.

Apa masalahnya? Bagaimana perbaikan *open source* yang cepat?

Jawaban singkat:

Masalah **bias** dan **ketidakseimbangan data**.

Perbaikan: cek distribusi dengan *Pandas*, *stratified split*, *class weights* di Keras, dan uji metrik **per kelompok**.

Pembahasan:

Akurasi keseluruhan bisa menipu. Etisnya, ukur **fairness** per kelompok. Lalu seimbangkan data atau atur *class_weight*.

Pseudocode:

```

DATA = muat_dataset()
cek_distribusi(DATA, by="kelompok")

if tidak_seimbang(DATA):
    DATA = oversample_minority(DATA)

X_train, X_test, y_train, y_test = stratified_split(DATA,
by="kelompok")
model = train_keras(X_train, y_train, class_weight="balanced")
lapor = evaluasi_per_kelompok(model, X_test, y_test,
metrik=["precision", "recall", "f1"])

```

Mini Python (opsional):

```

import pandas as pd

```

```
df.groupby("kelompok")["label"].value_counts(normalize=True)
```

Soal 5: Pilih arsitektur aman untuk info kesehatan (RAG vs LLM bebas)

Soal:

Kamu diminta bikin chatbot kesehatan sekolah. Pilih mana yang paling etis dan aman:

- A) LLM umum tanpa batasan sumber.
- B) *RAG lokal* (Ollama + OpenWebUI) yang hanya menarik jawaban dari PDF Kemenkes yang di-*index*.

Jelaskan kenapa. Sertakan *pseudocode* query.

Jawaban singkat:

Pilih **B (RAG lokal)**.

Alasan: sumber **terbatas dan tepercaya**, bisa *offline* (privasi), jejak sumber jelas (akuntabel).

Pembahasan:

Soal kesehatan itu sensitif. Etisnya, hanya jawab dari sumber resmi. *RAG* mencegah halu.

Pseudocode:

```
DOCSTORE = index(["kemenkes_gizi.pdf", "pedoman_imunisasi.pdf"])

function jawab_aman(pertanyaan):
    konteks = retrieve(DOCSTORE, query=pertanyaan, top_k=3)
    jawaban = LLM_local.generate(context=konteks,
    prompt=pertanyaan)
    return jawaban, sumber(konteks)

print(jawab_aman("Apakah vitamin X menyembuhkan flu?"))
```

Mini Python (opsional, sketsa):

```
query = "apakah vitamin X menyembuhkan flu"
ctx = retrieve_from_faiss(query, top_k=3) # pakai FAISS/Chroma
answer = ollama_generate(prompt=query, context=ctx)
```

Ringkas Tips Etika (ingat cepat)

- **Cek sumber** dulu.
- **Deteksi kata pemicu** hoax.
- **Moderasi bertingkat** untuk *hate speech*.
- **Ukur fairness** per kelompok.
- **Utamakan RAG lokal** untuk topik sensitif.

Fun Facts!

- **Ollama** dan **OpenWebUI** punya fitur yang membantu kalian memahami darimana AI mendapatkan jawabannya, ini bisa membantu kalian mendeteksi **hoax**.
- Ada riset khusus yang membahas cara membuat AI yang bisa mendeteksi **ujaran kebencian**.
- **RAG lokal** bisa kalian pakai untuk membuat **chatbot** yang hanya menjawab dari sumber-sumber yang terpercaya. Ini adalah salah satu cara untuk melawan disinformasi.

Tips Belajar atau Menghafal Cepat

- **Cek Sumber:** Sebelum percaya pada sebuah informasi, selalu cek darimana sumbernya.
- **Gunakan Logika:** Tanyakan pada diri sendiri, "Apakah ini masuk akal?"
- **Diskusi:** Diskusikan isu-isu etika dengan teman-temanmu.

Aktivitas Siswa

Aktivitas 1: Radar Hoax Kata Kunci (CSV)

Tujuan: mendeteksi teks yang punya kata pemicu hoax.

Bahan: **komentar.csv** (kolom: **teks**).

Kata kunci contoh: "terbukti", "segera sebar", "menyembuhkan semua", "100%", "viral".

Langkah

1. Buka file CSV dengan **Pandas**.
2. Ubah teks ke huruf kecil.
3. Cek apakah ada kata kunci hoax.
4. Tambah kolom **potensi_hoax** (True/False).
5. Hitung persentase hoax.
6. Simpan hasil ke **komentar_berlabel.csv**.

Pseudocode

```
DATA = read_csv("komentar.csv")
KW = ["terbukti", "segera sebar", "menyembuhkan semua",
      "100%", "viral"]

for setiap baris in DATA:
    t = lower(baris.teks)
    baris.potensi_hoax = ada_satu(KW di t)

persen = hitung_true(DATA.potensi_hoax) / jumlah_baris
tulis_csv(DATA, "komentar_berlabel.csv")
```

Contoh Python

```
import pandas as pd

df = pd.read_csv("komentar.csv")
kw = ["terbukti", "segera sebarkan", "menyembuhkan semua", "100%", "viral"]

def is_hoax(text):
    t = str(text).lower()
    return any(k in t for k in kw)

df["potensi_hoax"] = df["teks"].apply(is_hoax)
persen = 100 * df["potensi_hoax"].mean()
print(f"Potensi hoax: {persen:.1f}%")
df.to_csv("komentar_berlabel.csv", index=False)
```

Jawaban / Hasil yang diharapkan

- Tampil angka persentase.
- Ada file `komentar_berlabel.csv` dengan kolom `potensi_hoax`.

Aktivitas 2: Moderasi *Hate Speech* Tingkat Dasar (Rule + Log)

Tujuan: buat *pipeline* moderasi *chat* sederhana: deteksi → peringatan → *mute* → *log*.

Bahan: daftar kata terlarang (kurasi hati-hati): `["bodoh", "idiot", "rasis"]`.

Catatan etika: jangan gunakan untuk menyerang; ini untuk edukasi moderasi.

Langkah

1. Siapkan daftar kata terlarang.
2. Buat fungsi `is_hate(text)`.
3. Simulasikan daftar pesan (`list`) dari beberapa pengguna.
4. Jika terdeteksi:
 - tulis "Peringatan untuk USER".
 - jika mengulang: *mute* 15 menit.
5. Simpan *log* (user, teks, tindakan).

Pseudocode

```
BLOCKLIST = [...]
pelanggaran = map_user_ke_jumlah()

for pesan in STREAM_CHAT:
    if ada(BLOCKLIST di pesan.teks):
        pelanggaran[user] += 1
        log("peringatan", user, pesan)
```

```

        if pelanggaran[user] >= 2:
            log("mute_15m", user, pesan)

```

Contoh Python

```

blocked = {"bodoh", "idiot", "rasis"}
pelanggaran = {}
log = []

def is_hate(t):
    t = str(t).lower()
    return any(w in t for w in blocked)

chats = [
    ("andi", "Kamu bodoh banget"),
    ("budi", "Santai guys"),
    ("andi", "Maaf, tadi emosi, idiot itu jelek"),
]

for user, teks in chats:
    if is_hate(teks):
        pelanggaran[user] = pelanggaran.get(user, 0) + 1
        tindakan = "peringatan" if pelanggaran[user] == 1 else
        "mute_15m"
        log.append((user, teks, tindakan))
        print(user, tindakan)
    else:
        print(user, "aman")

print("LOG:", log)

```

Jawaban / Hasil yang diharapkan

- *Output* menunjukkan “peringatan” lalu “mute_15m” untuk pelanggar berulang.
- *Log* berisi bukti moderasi.

Aktivitas 3: Cek Fakta ala **RAG Lokal** (Simulasi)

Tujuan: jawab pertanyaan hanya dari sumber terpercaya.

Bahan: *index* sederhana berupa daftar kutipan (misal ringkasan PDF Kemenkes).

Catatan: ini *simulasi* konsep *RAG* tanpa server LLM. Fokus *retrieval*.

Langkah

1. Buat “basis pengetahuan” (**knowledge_base**) berupa daftar kalimat fakta.
2. Ambil pertanyaan pengguna.
3. Cari 3 fakta paling mirip (pencarian kata kunci sederhana).
4. Tampilkan jawaban + sumber.

Pseudocode

```
KB = ["fakta_1 ...", "fakta_2 ...", ...]
pertanyaan = input()
kandidat = ambil_top3_yang_paling_banyak_kata_sama(KB,
pertanyaan)
jawab = rangkum(kandidat)
tampilkan(jawab, kandidat)
```

Contoh Python

```
kb = [
    "Flu disebabkan virus influenza, vitamin biasa tidak
    menyembuhkan.",
    "Cuci tangan dengan sabun mencegah penyebaran penyakit.",
    "Vaksin direkomendasikan sesuai panduan resmi Kemenkes."
]

def score(doc, q):
    s = 0
    for w in set(q.lower().split()):
        if w in doc.lower():
            s += 1
    return s

q = "Apakah vitamin X menyembuhkan flu?"
top = sorted(kb, key=lambda d: score(d,q), reverse=True)[:3]
jawab = f"Jawaban aman: cek sumber resmi. Fakta terkait:\n- " +
"\n- ".join(top)
print(jawab)
```

Jawaban / Hasil yang diharapkan

- Jawaban mengutip fakta relevan dari “basis pengetahuan”, **bukan** **halu**.
- Siswa paham ide RAG: **retrieve dulu**, baru jawab.

Aktivitas 4: Cek *Fairness*: Akurasi VS Kelompok

Tujuan: ukur kinerja model per kelompok (contoh: **kelompok** A/B).

Bahan: **data_label.csv** (kolom: **teks**, **label** {0/1}, **kelompok** {A/B}).

Catatan: bisa mulai tanpa *neural network*; fokus metrik per kelompok.

Langkah

1. Baca CSV.
2. Buat prediksi dummy (misal: aturan kata kunci sederhana).
3. Hitung *precision/recall* per kelompok A dan B.
4. Bandingkan hasil. Jika timpang, usulkan perbaikan (rebalancing).

Pseudocode

```
DATA = read_csv()
y_true = DATA.label
```

```

y_pred = rule_based(DATA.teks)
for g in ["A","B"]:
    hitung_metrik(y_true[kelompok==g], y_pred[kelompok==g])
lapor_ketimpangan()

```

Contoh Python

```

import pandas as pd
from sklearn.metrics import precision_score, recall_score

df = pd.read_csv("data_label.csv") # kolom: teks,label,kelompok
kw_pos = {"bagus","mantap","recommended"}

def pred(t):
    t=str(t).lower()
    return int(any(k in t for k in kw_pos))

df["pred"] = df["teks"].apply(pred)

for g in df["kelompok"].unique():
    sub = df[df["kelompok"]==g]
    p = precision_score(sub["label"], sub["pred"],
zero_division=0)
    r = recall_score(sub["label"], sub["pred"], zero_division=0)
    print(f"Kelompok {g} -> precision={p:.2f}, recall={r:.2f}")

```

Jawaban / Hasil yang diharapkan

- Tabel metrik per kelompok muncul.
- Diskusi: kalau timpang, **perlu perbaiki data** (seimbangkan).

Aktivitas 5: Klasifikasi Sederhana dengan *Machine Learning*

Tujuan: buat model klasifikasi teks **hoax vs bukan** pakai *scikit-learn*.

Bahan: `dataset_hoax.csv` (kolom: `teks`, `label` {hoax, nonhoax}).

Catatan: ini *baseline*. Opsi lanjut: pakai **Keras** + *neural network*.

Langkah

1. Baca data.
2. Split train/test (stratified).
3. *Vectorize* teks (TF-IDF).
4. Latih model (LogisticRegression).
5. Evaluasi akurasi + *confusion matrix*.
6. Simpan model (*joblib*) untuk dipakai ulang.

Pseudocode

```

DATA = read_csv()
X_train, X_test, y_train, y_test = stratified_split(DATA.teks,
DATA.label)

```

```

V = TFIDF.fit(X_train)
Xtr = V.transform(X_train); Xte = V.transform(X_test)
model = LogisticRegression.fit(Xtr, y_train)
pred = model.predict(Xte)
lapor(akurasi, confusion_matrix)
simpan(model, vectorizer)

```

Contoh Python

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

df = pd.read_csv("dataset_hoax.csv") # teks, label
X_train, X_test, y_train, y_test = train_test_split(
    df["teks"], df["label"], test_size=0.2, random_state=42,
    stratify=df["label"]
)

tfidf = TfidfVectorizer(min_df=2, ngram_range=(1,2))
Xtr = tfidf.fit_transform(X_train)
Xte = tfidf.transform(X_test)

clf = LogisticRegression(max_iter=200)
clf.fit(Xtr, y_train)
pred = clf.predict(Xte)

print("Akurasi:", accuracy_score(y_test, pred))
print("Confusion:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))

```

Opsi Lanjut (Keras, *neural network*):

- Embedding sederhana + Dense layer.
- *Class weights* jika data tidak seimbang.

Jawaban / Hasil yang diharapkan

- Tampil akurasi dan *confusion matrix*.
- Siswa paham alur *ML* dasar untuk teks.

Ringkas Hafalan

- **Hoax**: cek kata pemicu + verifikasi sumber.
- **Hate speech**: moderasi bertingkat, simpan *log*.
- **RAG**: jawab hanya dari sumber terpercaya.
- **Fairness**: ukur per kelompok, bukan rata-rata doang.
- **ML**: TF-IDF → model → evaluasi → simpan.

Rangkuman

Etika AI itu penting. AI bisa bantu atau melukai. **Bedakan** *disinformasi* (salah **sengaja**) dan *misinformasi* (salah **tidak sengaja**). **Waspada** *hoax* yang tampak meyakinkan dan *ujaran kebencian* yang menyerang identitas. Ingat peta sederhana: **AI = koki, data = bahan, informasi = hidangan, etika = rasa tanggung jawab**. Tugas kita: **pakai AI dengan bijak**, jaga keamanan, dan hormati orang lain.

Lawan hoax dan kebencian dengan tool open source. Pakai **Python + Pandas** untuk cek kata kunci dan pola sebaran. Bangun **baseline machine learning** (TF-IDF + Logistic Regression) lalu tingkatkan dengan **Keras** dan *neural network* untuk peka konteks. Untuk topik sensitif (misal kesehatan), pilih **RAG lokal** di **Ollama + OpenWebUI** agar jawaban **hanya** dari sumber terpercaya—jejak sumber jelas, privasi lebih aman. **Ukur fairness** per kelompok, jangan cuma lihat akurasi rata-rata.

Kebiasaan baik setiap hari. **Cek sumber** sebelum percaya. **Gunakan logika**: “Apakah ini masuk akal?” **Hentikan penyebaran** jika ragu, **laporkan** konten berbahaya. **Moderasi bertingkat** untuk *hate speech*: *warning* → *mute* → *ban*, simpan *log*. **Tujuan akhir**: AI yang **adil, bermanfaat, dan tidak merugikan**—kita jadi **penjaga kebenaran digital**, bukan penyebar masalah.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. *Hoax* adalah salah satu bentuk *disinformasi*.
Jawaban: Benar
2. *Misinformasi* dibuat dengan sengaja untuk menipu orang.
Jawaban: Salah
3. Ujaran kebencian bisa memperkuat persatuan masyarakat.
Jawaban: Salah
4. Model *LLM offline* lebih aman karena data tidak keluar ke internet.
Jawaban: Benar
5. Semua informasi viral di media sosial pasti benar.
Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Apa bedanya *disinformasi* dan *misinformasi*?
a) *Disinformasi* = salah sengaja, *Misinformasi* = salah tidak sengaja

- b) Sama saja
- c) Disinformasi = berita positif, Misinformasi = berita negatif
- d) Disinformasi = berita benar

Jawaban: a

2. Apa bahaya *hoax*?

- a) Membuat orang tertawa
- b) Membingungkan dan menipu masyarakat
- c) Membuat informasi jadi lebih jelas
- d) Tidak ada dampak

Jawaban: b

3. Ujaran kebencian biasanya menyerang...

- a) Hobi seseorang
- b) Ras, agama, gender
- c) Hasil ujian matematika
- d) Game online

Jawaban: b

4. Etika AI berfungsi sebagai...

- a) Aplikasi baru
- b) Pagar moral agar AI tidak disalahgunakan
- c) Virus komputer
- d) Tambahan RAM

Jawaban: b

5. Jika ada pesan WhatsApp tentang obat ajaib, apa yang harus dilakukan?

- a) Sebarkan cepat
- b) Abaikan
- c) Cek sumber resmi dulu
- d) Percaya langsung

Jawaban: c

6. Contoh *fun fact* tentang AI di etika teks adalah...

- a) AI bisa mendeteksi *hoax*
- b) AI bisa masak mie instan
- c) AI bisa jadi guru olahraga
- d) AI bisa memperbaiki jalan

Jawaban: a

7. *RAG lokal* digunakan untuk...

- a) Menyimpan musik
- b) Membuat chatbot hanya jawab dari sumber terpercaya
- c) Mempercepat WiFi
- d) Menghapus data

Jawaban: b

8. Apa akibat *hoax politik*?
a) Publik semakin percaya pemerintah
b) Kepercayaan publik hancur
c) Ekonomi membaik
d) Semua orang bahagia
Jawaban: b
9. Teknologi *open source* seperti Python berguna untuk...
a) Membuat filter kata kasar
b) Membuat virus
c) Membuat hoax
d) Menghapus etika
Jawaban: a
10. Apa yang harus dilakukan jika menemukan ujaran kebencian di media sosial?
a) Like dan share
b) Diam saja
c) Laporkan dan jangan sebarkan
d) Ikut menghina
Jawaban: c

C. Isian Singkat (5 Soal)

1. Sebutkan satu contoh kata kunci yang sering muncul dalam *hoax*!
Jawaban: "Terbukti"
2. Apa singkatan dari RAG?
Jawaban: Retrieval-Augmented Generation
3. Python library yang sering dipakai untuk analisis data teks adalah...
Jawaban: Pandas
4. Etika AI membantu kita agar teknologi tidak...
Jawaban: disalahgunakan
5. Ujaran kebencian bisa merusak _____ di masyarakat.
Jawaban: persatuan

D. Soal Eksplorasi (3 Soal Menantang)

Soal 1:

Proyek Mini Deteksi Hoax

Buat program Python sederhana untuk mendeteksi kata "segera sebarkan" atau "100%" pada sebuah teks. Jika ada kata tersebut, tampilkan "Potensi Hoax", jika tidak tampilkan "Aman".

Jawaban Eksplorasi:

```
def cek_hoax(teks):
    hoax_kw = ["segera sebarkan", "100%"]
    if any(k in teks.lower() for k in hoax_kw):
        return "Potensi Hoax"
    return "Aman"

print(cek_hoax("Obat ini 100% menyembuhkan semua penyakit!"))
print(cek_hoax("Belajar Python itu menyenangkan"))
```

Soal 2:

Analisis Bias Kalimat

Tuliskan 3 contoh kalimat bias (misalnya: “Semua perempuan suka memasak”). Kemudian, tuliskan bagaimana cara AI bisa mendeteksi kata “semua” atau “setiap” untuk menandai kalimat bias.

Jawaban Eksplorasi:

- Contoh kalimat bias:
 1. Semua laki-laki suka sepak bola.
 2. Setiap anak pintar matematika.
 3. Semua orang suka pedas.
- Cara deteksi: pakai pencarian kata kunci “semua” atau “setiap” dalam teks.

Soal 3:

Diskusi Etika Chatbot

Bayangkan kamu membuat chatbot sekolah. Apa risiko jika chatbot tersebut bisa menjawab bebas dari internet tanpa filter? Dan bagaimana cara mencegahnya dengan RAG lokal?

Jawaban Eksplorasi:

- Risiko: chatbot bisa memberi info salah tentang kesehatan, atau bahkan menyebarkan ujaran kebencian.
- Pencegahan: gunakan *RAG lokal* supaya chatbot hanya menjawab dari dokumen resmi (misal buku Kemenkes atau modul sekolah).

Tugas Proyek

Proyek 1: Filter Kata Kasar di Grup Chat

Tugas:

Buat program Python sederhana untuk mendeteksi kata kasar di sebuah *chat group*.

Langkah:

1. Buat daftar kata kasar (misal: *bodoh*, *idiot*, *rasis*).
2. Baca daftar komentar dari list.
3. Tandai komentar yang mengandung kata kasar.

4. Cetak hasil: komentar + status (“mengandung ujaran kebencian” atau “aman”).

Pseudocode:

```
BLOCKLIST = ["bodoh","idiot","rasis"]
for setiap_komentar in DATA:
    if ada_kata_dalam(BLOCKLIST, komentar):
        cetak(komentar, "ujaran kebencian")
    else:
        cetak(komentar, "aman")
```

Contoh Python:

```
blocked = {"bodoh","idiot","rasis"}
chats = ["Kamu pintar sekali", "Dasar bodoh", "Kita belajar bareng yuk"]

for teks in chats:
    if any(b in teks.lower() for b in blocked):
        print(teks, "-> ujaran kebencian")
    else:
        print(teks, "-> aman")
```

Proyek 2: Cek Hoax pada Judul Artikel

Tugas:

Buat detektor sederhana untuk mengecek apakah sebuah judul berita punya tanda-tanda hoax.

Langkah:

1. Siapkan kata kunci hoax: *“terbukti, viral, segera sebarkan, 100%”*.
2. Input beberapa judul artikel.
3. Tandai judul yang berpotensi hoax.

Pseudocode:

```
HOAX_KEYWORDS = ["terbukti","viral","segera sebarkan","100%"]
for judul in ARTIKEL:
    if ada(HOAX_KEYWORDS di judul):
        label = "potensi hoax"
    else:
        label = "aman"
    cetak(judul, label)
```

Contoh Python:

```
kw = ["terbukti","viral","segera sebarkan","100%"]
judul = ["Resep ini TERBUKTI sembuhkan semua penyakit!",
        "Timnas Indonesia menang 3-0 lawan Jepang"]
```

```

for j in judul:
    if any(k in j.lower() for k in kw):
        print(j, "-> potensi hoax")
    else:
        print(j, "-> aman")

```

Proyek 3: Analisis Privasi Data Game

Tugas:

Buat program yang menampilkan data pribadi apa saja yang bisa dikumpulkan oleh sebuah aplikasi game.

Langkah:

1. Buat list data pribadi (misal: nama, lokasi, email, nomor HP).
2. Buat fungsi untuk menampilkan data yang dikumpulkan game X.
3. Diskusikan kenapa data itu harus dijaga.

Pseudocode:

```

DATA_PIRIBADI = ["nama", "lokasi", "email", "nomor HP"]
for data in DATA_PIRIBADI:
    jika game_minta(data):
        cetak("Game ini mengumpulkan:", data)

```

Contoh Python:

```

data_pribadi = ["nama", "lokasi", "email", "nomor HP"]
gameX = ["nama", "lokasi"]

for d in data_pribadi:
    if d in gameX:
        print("Game ini mengumpulkan:", d)

```

Proyek 4: Membandingkan Akurasi Model Etis vs Tidak Etis

Tugas:

Buat perbandingan dua model klasifikasi teks:

1. Model A → data tidak seimbang.
2. Model B → data sudah diseimbangkan (*balanced*).

Langkah:

1. Siapkan dataset teks kecil (label: hoax vs nonhoax).
2. Latih model Logistic Regression tanpa balancing.
3. Latih model lagi dengan *class_weight="balanced"*.
4. Bandingkan akurasi keduanya.

Pseudocode:

```

DATA = muat_dataset()
split DATA jadi train/test
model_A = LogisticRegression.fit(DATA_train)
model_B = LogisticRegression.fit(DATA_train,
class_weight="balanced")
bandingkan(akurasi_A, akurasi_B)

```

Contoh Python:

```

from sklearn.linear_model import LogisticRegression
clfA = LogisticRegression(max_iter=200).fit(Xtr, ytr)
clfB = LogisticRegression(max_iter=200,
class_weight="balanced").fit(Xtr, ytr)
print("Akurasi model A:", clfA.score(Xte,yte))
print("Akurasi model B:", clfB.score(Xte,yte))

```

Proyek 5: Chatbot Etis Mini (Simulasi)

Tugas:

Buat chatbot sederhana yang hanya menjawab dari daftar fakta resmi.

Langkah:

1. Buat list fakta (misal: tentang kesehatan).
2. User bertanya.
3. Cari jawaban dari fakta yang relevan (cocok kata).
4. Jika tidak ada fakta → jawab: "Cek sumber resmi ya."

Pseudocode:

```

FACTS = ["Flu disebabkan virus influenza",
"Cuci tangan dengan sabun mencegah penyakit"]
pertanyaan = input()
jika cocok(pertanyaan, FACTS):
    tampilkan(fakta_terkait)
else:
    tampilkan("Cek sumber resmi ya")

```

Contoh Python:

```

facts = ["Flu disebabkan virus influenza",
"Cuci tangan dengan sabun mencegah penyakit"]
q = "Apakah vitamin bisa sembuhkan flu?"
jawab = [f for f in facts if "flu" in f.lower()]
if jawab:
    print("Jawaban:", jawab[0])
else:
    print("Cek sumber resmi ya")

```

Dengan proyek-proyek ini, kamu akan:

- Pahami cara **filter hoax & hate speech**.
- Mengerti pentingnya **privasi data**.
- Bisa membandingkan **model adil vs bias**.
- Belajar membangun **chatbot etis mini**.

BAB 8: Proyek – Analisis Sentimen Komentar Media Sosial dengan Python

Selamat! Sekarang kita sudah sampai di bab paling seru: **Proyek Akhir!** Setelah kita belajar banyak teori, dari mengolah data pakai Pandas sampai kenalan sama AI, sekarang kita akan menggabungkan semuanya dalam sebuah **proyek nyata**. Kalian bakal jadi ilmuwan data beneran!

Tujuan Pembelajaran

- Mengaplikasikan semua ilmu yang sudah didapat (preprocessing, klasifikasi, analisis sentimen).
- Mengikuti alur kerja analisis data dari awal sampai akhir.
- Membuat model analisis sentimen sederhana yang bisa diandalkan.
- Memvisualisasikan dan mengevaluasi hasil proyek.

Peta Konsep Sederhana

Bayangin ini adalah proses membuat game.

- **Tulis Skrip (Mengumpulkan Dataset):** Kita cari data yang ingin kita olah.
- **Bersihkan Kode (Data Preprocessing):** Kita rapikan data yang kotor pakai Python.
- **Rancang Level (Analisis Sentimen):** Kita analisis setiap ulasan.
- **Uji Coba Game (Evaluasi Hasil):** Kita tes modelnya, seberapa jago dia?
- **Rilis Game (Visualisasi & Laporan):** Kita pamerkan hasilnya dalam bentuk grafik yang keren.

Apersepsi: Kamu Mau Tahu Respon Penonton Film!

Kamu baru saja nonton film superhero. Setelah itu, kamu buka Twitter (atau X) dan melihat banyak komentar tentang film itu. Ada yang bilang filmnya keren, ada yang bilang jelek, dan ada yang biasa saja.

- **Tantangan:** Kamu penasaran, lebih banyak orang yang suka atau tidak suka film ini?
- **Solusi:** Kalian bisa pakai proyek ini untuk menjawabnya! Kita akan kumpulkan ulasan dari media sosial, lalu kita analisis secara otomatis untuk tahu sentimennya.

Tuh, kan, keren banget kalau kita bisa tahu jawaban dari pertanyaan ini!

Penjelasan Teori

Bayangin kamu jadi detektif. Tugasmu membaca komentar orang di internet, lalu menebak: **positif**, **negatif**, atau **netral**. Itu namanya **analisis sentimen**.

1) Alur Kerja (*Pipeline*) Sederhana

Pikirkan seperti bikin *game* kecil.

1. **Kumpulkan data** (*dataset*).
Ambil komentar dari CSV atau *copy-paste* manual.
2. **Rapikan teks** (*preprocessing*).
Huruf kecil, buang tanda baca, bersihkan *link*.
3. **Ubah teks jadi angka** (*feature extraction*).
Pakai *bag-of-words* atau *TF-IDF*.
4. **Pilih cara analisis**.
Lexicon rule-based, *machine learning*, atau *neural network* (Keras).
5. **Latih & uji** (*train/test split*).
Cek performa model di data yang belum pernah dilihat.
6. **Nilai hasil** (*evaluation*).
Lihat *accuracy*, *precision*, *recall*, *F1*.
7. **Tampilkan** (*visualization*).
Grafik batang jumlah positif/negatif.
8. **Simpan & pakai** (*deploy offline*).
Jalankan tanpa internet di laptop sekolah.

Intinya: **teks** → **angka** → **model** → **skor** → **grafik**.

2) Data & Label

- **Data**: kolom **ulasan** berisi kalimat.
- **Label**: target yang benar. Contoh: **positif**, **negatif**, **netral**.
- **Imbangkan** data. Jangan semua positif. Bikin adil.

Kenapa butuh label?

Supaya model “belajar” mana kalimat yang bagus, mana yang jelek.

3) Pembersihan Teks (*Preprocessing*)

Kita mau teks rapi. Biar mesin paham.

- **Lowercase**: “KEREN” → “keren”
- **Hapus tanda baca/angka**: **! , ? , 123**
- **Buang URL & @mention**: lebih bersih
- **Tokenisasi** (*tokenization*): pecah jadi kata
- **Stopwords**: buang kata umum (*dan*, *yang*, *di*)

- **Stemming**: “makanannya” → “makan”
(bisa pakai Sastrawi – *open source*)

Hasilnya: teks jadi simpel, mudah dihitung.

4) Ubah Teks ke Angka (*Feature Extraction*)

Komputer cinta angka. Ada dua cara favorit:

- **Bag-of-Words**: hitung berapa kali kata muncul.
- **TF-IDF**: mirip, tapi kasih bobot. Kata unik dapat bobot lebih tinggi.

Contoh: kalimat “filmnya keren dan seru” → vektor angka.

5) Tiga Jalur Analisis

Kamu bisa pilih salah satu. Atau coba semua.

a) *Lexicon Rule-Based* (daftar kata)

- Punya daftar kata positif/negatif.
- Hitung mana yang lebih banyak.
- **Plus**: cepat, mudah.
- **Minus**: kurang peka konteks.

b) *Machine Learning*

- Contoh model: *Logistic Regression*, *Naive Bayes*, *SVM* (semua *open source*).
- Butuh **data berlabel**.
- **Plus**: akurat untuk data sekolah.
- **Minus**: perlu latih model.

c) *Neural Network* dengan *Keras*

- Pakai *Dense* atau *LSTM* sederhana.
- Lebih fleksibel, bisa belajar pola.
- **Plus**: kuat.
- **Minus**: butuh waktu latih (*epoch*), tapi bisa offline.

6) Evaluasi Hasil (*Metrics*)

Kita butuh skor yang jujur.

- **Accuracy** = (prediksi benar) / (total)
- **Precision**: seberapa tepat saat bilang “positif”
- **Recall**: seberapa banyak “positif” yang berhasil ditemukan
- **F1**: gabungan *precision* dan *recall*
- **Confusion Matrix**: tabel hit/miss tiap kelas

Punya **baseline** dulu itu bagus. Misal, tebak semua “positif”. Lalu kalahkan baseline-mu.

7) Visualisasi Hasil

Biar mudah dibaca guru dan teman.

- **Bar chart:** jumlah positif/negatif/netral
- **Pie chart:** komposisi sentimen
- **Heatmap:** *confusion matrix*
- **Line chart:** tren sentimen per hari

Tools: Matplotlib (Python), Orange Data Mining (klik-klik, *open source*).

8) Etika Singkat (Wajib!)

- **Privasi:** jangan sebar data pribadi.
- **Bias:** kata kasar $\neq$ selalu “negatif” (lihat konteks).
- **Ujaran Kebencian:** hati-hati. Gunakan dengan tanggung jawab.
- **Transparansi:** jelaskan cara kerja modelmu.

9) Mini Contoh Kode (Ringkas Banget)

A. *Lexicon Rule-Based* (cepat)

```
import pandas as pd, re

positif = {"keren", "bagus", "mantap", "seru", "hebat"}
negatif = {"jelek", "buruk", "parah", "benci", "kecewa"}

def bersih(t):
    t = t.lower()
    t = re.sub(r"http\S+|[\^a-z\s]", " ", t)
    return t

def sentimen(t):
    t = bersih(t).split()
    p = sum(w in positif for w in t)
    n = sum(w in negatif for w in t)
    if p>n: return "positif"
    if n>p: return "negatif"
    return "netral"

df = pd.read_csv("ulasan.csv") # kolom: ulasan, label_asli
(optional)
df["pred"] = df["ulasan"].apply(sentimen)
```

B. *TF-IDF + Logistic Regression* (scikit-learn)

```
import pandas as pd
```

```

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

df = pd.read_csv("ulasan.csv") # kolom: ulasan, label
Xtr, Xte, ytr, yte = train_test_split(df["ulasan"], df["label"],
test_size=0.2, random_state=42)

tfidf = TfidfVectorizer(lowercase=True, max_features=5000,
ngram_range=(1,2))
Xtrv = tfidf.fit_transform(Xtr); Xtev = tfidf.transform(Xte)

clf = LogisticRegression(max_iter=1000)
clf.fit(Xtrv, ytr)
yp = clf.predict(Xtev)
print(classification_report(yte, yp))

```

C. Neural Network Sederhana (Keras)

```

import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import pandas as pd

df = pd.read_csv("ulasan.csv") # kolom: ulasan, label
(positif/negatif)
label_map = {"negatif":0, "positif":1}
y = df["label"].map(label_map).values

tok = Tokenizer(num_words=5000, oov_token="<unk>")
tok.fit_on_texts(df["ulasan"])
seq = tok.texts_to_sequences(df["ulasan"])
X = pad_sequences(seq, maxlen=40)

model = models.Sequential([
    layers.Embedding(input_dim=5000, output_dim=32, input_length=40),
    layers.GlobalAveragePooling1D(),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])
model.fit(X, y, epochs=5, batch_size=32, validation_split=0.2)

```

Semua bisa jalan **offline** di laptop sekolah. *Open source* semua.

10) Checklist Proyek

- Data rapi, label jelas

- *Preprocessing* sudah oke (lowercase, buang tanda baca, *stopwords*)
- Fitur pakai *TF-IDF* atau *Embedding*
- Punya *baseline* sederhana
- Evaluasi lengkap: *accuracy, precision, recall, F1*
- Grafik ringkas dan jelas
- Catatan etika & batasan model

11) Kamus Mini (Istilah Wajib)

- *Dataset*: kumpulan data.
- *Preprocessing*: merapikan data.
- *Tokenization*: memecah kalimat jadi kata.
- *Stopwords*: kata umum yang dibuang.
- *Stemming*: memotong kata ke bentuk dasar.
- *Bag-of-Words*: fitur hitung kata.
- *TF-IDF*: bobot penting kata.
- *Train/Test Split*: bagi data untuk latih & uji.
- *Confusion Matrix*: tabel salah/benar per kelas.
- *Neural Network*: model cerdas mirip jaringan otak.

12) Kenapa Ini Keren?

- Dipakai **industri beneran**.
- Bisa **offline** di sekolah.
- Bahan **portofolio** kamu.
- Cocok untuk lomba, riset, dan *startup* kecil.

Kasus Ilustrasi

Kasus 1: “Film Superhero: Ramai atau Biasa Aja?”

Ilustrasi singkat:

Kamu habis nonton film superhero. Di X (Twitter) banyak komentar. Kamu penasaran: **lebih banyak yang suka atau enggak?**

Data masuk (CSV sederhana):

- **teks** (komentar)
- **tanggal** (opsional)
- **label** (opsional, untuk uji akurasi)

Tujuan:

Hitung persentase **positif / negatif / netral**.

Langkah ringkas:

- Kumpulkan 30–50 komentar (boleh fiktif).
- *Preprocessing*: huruf kecil, buang tanda baca, hapus URL.
- Analisis pakai *lexicon rule-based* (daftar kata positif/negatif).
- Tampilkan grafik batang (bar chart) jumlah tiap sentimen.

Pseudocode:

```
load csv as data
positif = ["keren", "bagus", "mantap", "seru", "hebat"]
negatif = ["jelek", "buruk", "parah", "benci", "kecewa"]

function bersih(teks):
    teks = lower(teks)
    teks = hapus_tanda_baca_dan_url(teks)
    return teks

function labelkan(teks):
    kata = split(bersih(teks))
    p = hitung_kata(kata, positif)
    n = hitung_kata(kata, negatif)
    if p > n: return "positif"
    if n > p: return "negatif"
    return "netral"

for setiap baris in data:
    baris.pred = labelkan(baris.teks)

ringkas = hitung_jumlah_per_label(data.pred)
tampilkan_bar_chart(ringkas)
```

Kasus 2: “Review Game di Play Store: Cepat Cek Skor”

Ilustrasi singkat:

Ada game baru. Nilai *rating* 3.9, tapi komentar campur aduk. Kamu mau tahu sentimen benerannya.

Data masuk (CSV):

- **teks** (review)
- **bintang** (1–5)
- **label** (opsional)

Tujuan:

Bangun *model* sederhana **TF-IDF + Logistic Regression** (scikit-learn).

Langkah ringkas:

- Ambil 100 review (boleh gabungan review nyata + fiktif yang disesuaikan).
- *Preprocessing* standar.

- *Split* data: 80% latih, 20% uji.
- Ekstraksi fitur *TF-IDF*.
- Latih **Logistic Regression**.
- Lihat **accuracy/precision/recall** dan *confusion matrix*.
- (Opsional) Bandingkan dengan *baseline lexicon*.

Pseudocode:

```
load csv as data
X = data.teks
y = data.label // "positif"/"negatif" (netral bisa
dihapus/di-merge dulu)

Xtrain, Xtest, ytrain, ytest = split(X, y, test_size=0.2,
seed=42)

tfidf = TFIDF(lowercase=true, ngram=(1,2), max_features=5000)
Xtrain_vec = tfidf.fit_transform(Xtrain)
Xtest_vec = tfidf.transform(Xtest)

clf = LogisticRegression(max_iter=1000)
clf.fit(Xtrain_vec, ytrain)

ypred = clf.predict(Xtest_vec)
laporkan_metrics(ytest, ypred) // accuracy, precision, recall,
F1
plot_confusion_matrix(ytest, ypred)
```

Catatan: Semua *library* di atas *open source*. Jalan **offline** di laptop sekolah.

Kasus 3: “Event Sekolah: Sentimen dari Waktu ke Waktu”

Ilustrasi singkat:

Sekolah bikin pentas seni. Komentar di Instagram ramai. Guru minta grafik **tren**: hari H-2 sampai H+2.

Data masuk (CSV):

- **teks**
- **tanggal** (YYYY-MM-DD)
- **label** (opsional)

Tujuan:

Lihat **tren harian** sentimen. Apakah makin positif saat acara?

Langkah ringkas:

- *Preprocessing* standar + *lexicon rule-based* atau *model* dari Kasus 2.
- Kelompokkan per tanggal.
- Hitung proporsi **positif/negatif/netral** per hari.

- Plot **line chart** untuk positif vs waktu.

Pseudocode:

```
load csv as data
for setiap baris:
    baris.pred = labelkan_dengan_lexicon_atau_model(baris.teks)

grup = group_by(data, "tanggal")
hasil = []
for setiap tgl in grup:
    total = hitung(grup[tgl])
    p = hitung_label(grup[tgl], "positif") / total
    n = hitung_label(grup[tgl], "negatif") / total
    e = hitung_label(grup[tgl], "netral") / total
    hasil.append({tanggal: tgl, prop_positif: p, prop_negatif: n,
prop_netral: e})

plot_line(x=hasil.tanggal, y=hasil.prop_positif, title="Tren
Positif Harian")
```

Kasus 4: “Rekomendasi Menu Kantin: Mana yang Paling Disuka?”

Ilustrasi singkat:

Kantin jual 5 menu: **nasi goreng, ayam bakar, bakso, soto, mie goreng**. Kamu ingin merekomendasikan **3 menu ter-top** berdasarkan komentar siswa.

Data masuk (CSV):

- **menu** (nama menu)
- **teks** (komentar)
- **harga** (opsional)
- **label** (opsional)

Tujuan:

Skor sentimen per menu → **ranking** → rekomendasi.

Langkah ringkas:

- *Preprocessing* standar.
- *Lexicon rule-based* cepat untuk label awal.
- Skor = (jumlah positif) – (jumlah negatif).
- Urutkan skor tertinggi → rekomendasi 3 menu.

Pseudocode:

```
load csv as data
for setiap baris:
    baris.pred = labelkan_lexicon(baris.teks) //
"positif"/"negatif"/"netral"
```

```

skor_menu = dict_default(0)
for setiap_baris:
    if baris.pred == "positif": skor_menu[baris.menu] += 1
    if baris.pred == "negatif": skor_menu[baris.menu] -= 1

ranking = urutkan_berdasar_nilai(skor_menu, descending=True)
rekomendasi = ambil_teratas(ranking, 3)
cetak(rekomendasi)

```

Extensi seru: tambahkan *constraint* harga (misal, “pilih 3 menu dengan total harga ≤ Rp30.000”) → jadi mirip *knapsack mini*.

Kasus 5: “Moderasi Komentar Blog: Saring yang Kasar”

Ilustrasi singkat:

Blog OSIS kebanyakan komentar. Kamu perlu sistem sederhana untuk **menandai komentar kasar** agar guru bisa cek.

Data masuk (CSV):

- **id** (komentar id)
- **teks**
- **penulis** (opsional)
- **waktu** (opsional)

Tujuan:

Flagging komentar berisiko: **kasar**, **spam**, **hate speech** (versi sederhana).

Langkah ringkas:

- Buat *lexicon* kata kasar/hinaan (*open list*, diskusikan dengan guru).
- *Preprocessing* standar.
- Tandai komentar jika jumlah kata kasar ≥ ambang (misal ≥2).
- Simpan dua file: **aman.csv** dan **perlu_tinjau.csv**.

Pseudocode:

```

load csv as data
kata_kasar = ["****", "bodoh", "tolol", "goblok", ...] // sepakati
bersama guru

function skor_kasar(teks):
    kata = split(bersihkan(teks))
    return hitung_kata(kata, kata_kasar)

AMBANG = 2
for setiap_baris in data:
    s = skor_kasar(baris.teks)
    if s >= AMBANG:

```

```

        baris.flag = "perlu_tinjau"
    else:
        baris.flag = "aman"

    aman = filter(data, flag == "aman")
    tinjau = filter(data, flag == "perlu_tinjau")
    save_csv(aman, "aman.csv")
    save_csv(tinjau, "perlu_tinjau.csv")

```

Tambahan opsi *machine learning*: latih **Naive Bayes** dari contoh komentar yang “aman” vs “kasar” untuk akurasi lebih baik.

Untuk level lanjut, bisa pakai *neural network* kecil di *Keras* dengan data berlabel lokal.

Tips Etika (Super Singkat)

- Anonimkan nama akun.
- Hindari menyimpan data sensitif.
- Diskusikan daftar kata “kasar” dengan guru.
- Jelaskan batas model: *sarcasm* dan konteks budaya bisa bikin model salah.

Ringkasan Perangkat *Open Source* yang Dipakai

- **Python + Pandas**: baca/olah CSV.
- **scikit-learn**: *TF-IDF*, *Logistic Regression*, *train/test split*, *metrics*.
- **Keras (TensorFlow)**: *neural network* sederhana (opsional).
- **Matplotlib/Orange Data Mining**: visualisasi cepat.
- **Semua bisa offline** di laptop sekolah.

Kalau mau, aku bisa lanjutkan dengan **template CSV contoh** per kasus + **kode Python pendek** yang langsung jalan (biar siswa tinggal ganti isi komentar). Mau?

Soal Latihan & Pembahasan

Soal 1: Rule-Based Sentimen Dasar

Tugas:

Klasifikasikan 3 komentar berikut menjadi **positif** / **negatif** / **netral** pakai *lexicon* sederhana.

Komentar:

1. “Filmnya **keren** dan **seru**.”
2. “Aduh, alurnya **buruk** dan aku **kecewa**.”
3. “Ceritanya **bagus**, tapi aktingnya **jelek**.”

Gunakan *lexicon*:

- Positif = {keren, bagus, mantap, seru, hebat}
- Negatif = {jelek, buruk, parah, benci, kecewa}

Pseudocode:

```
positif = {keren, bagus, mantap, seru, hebat}
negatif = {jelek, buruk, parah, benci, kecewa}

function labelkan(teks):
    t = lower(teks); t = hapus_tanda_baca(t)
    kata = split(t)
    p = hitung_kecocokan(kata, positif)
    n = hitung_kecocokan(kata, negatif)
    if p > n: return "positif"
    if n > p: return "negatif"
    return "netral"

for k in [k1,k2,k3]: print(labelkan(k))
```

Jawaban:

1. Positif
2. Negatif
3. Netral

Pembahasan:

- (1) mengandung 2 kata positif (**keren, seru**).
- (2) mengandung 2 kata negatif (**buruk, kecewa**).
- (3) satu positif (**bagus**) dan satu negatif (**jelek**) → seimbang → **netral**.
Metode *lexicon rule-based* itu cepat dan *offline*. Tapi belum peka konteks.

Soal 2: Hitung *Metrics: Precision, Recall, F1, Accuracy*

Tugas:

Dari *confusion matrix* (kelas “positif” sebagai *positive class*):

- **TP = 32, FP = 12, FN = 8, TN = 48** (total 100 komentar)

Hitung: **precision, recall, F1, accuracy**.

Pseudocode:

```
TP=32; FP=12; FN=8; TN=48
precision = TP / (TP + FP)
recall    = TP / (TP + FN)
F1        = 2 * precision * recall / (precision + recall)
accuracy  = (TP + TN) / (TP + FP + FN + TN)
cetak(precision, recall, F1, accuracy)
```

Jawaban (angka):

- Precision = $32 / (32+12) = 0,727 \approx 72,7\%$
- Recall = $32 / (32+8) = 0,800 = 80,0\%$
- F1 = $2 \times 0,727 \times 0,800 / (0,727 + 0,800) = 0,762 \approx 76,2\%$
- Accuracy = $(32+48) / 100 = 0,800 = 80,0\%$

Pembahasan:

- **Precision:** seberapa tepat saat bilang “positif”.
- **Recall:** seberapa banyak “positif” yang berhasil ditemukan.
- **F1:** keseimbangan precision dan recall.
- **Accuracy:** proporsi prediksi benar dari semua data.

Semua *metrics* penting. Jangan hanya lihat *accuracy*.

Soal 3: Tren Harian: Pilih Hari Terbaik

Tugas:

Kamu punya hasil sentimen 3 hari (total per hari):

- **H1:** Positif=6, Negatif=3, Netral=1 (total=10)
- **H2:** Positif=5, Negatif=5, Netral=2 (total=12)
- **H3:** Positif=6, Negatif=1, Netral=1 (total=8)

Hitung **persentase positif** per hari, lalu tentukan **hari terbaik** untuk *rilis konten* promosi.

Pseudocode:

```
hari = {
    H1: {pos:6, neg:3, net:1},
    H2: {pos:5, neg:5, net:2},
    H3: {pos:6, neg:1, net:1}
}

function persen_pos(h):
    total = h.pos + h.neg + h.net
    return h.pos / total

p1 = persen_pos(hari.H1)    // 6/10
p2 = persen_pos(hari.H2)    // 5/12
p3 = persen_pos(hari.H3)    // 6/8
pilih = argmax({H1:p1, H2:p2, H3:p3})
cetak(p1,p2,p3,pilih)
```

Jawaban:

- H1: $6/10 = 60\%$
- H2: $5/12 = 41,7\%$

- H3: $6/8 = 75\% \rightarrow$ tertinggi

Hari terbaik: H3.

Pembahasan:

Kita bandingkan proporsi, bukan jumlah mentah. Hari dengan *positif%* terbesar biasanya paling pas untuk *rilis* atau *campaign*.

Soal 4: Rekomendasi Menu: Skor Sentimen

Tugas:

Hitung skor tiap menu: **skor = positif – negatif**.

Data agregat:

- Nasi goreng: p=8, n=3
- Bakso: p=5, n=7
- Ayam bakar: p=9, n=4
- Soto: p=4, n=2
- Mie goreng: p=3, n=1

Urutkan dari skor tertinggi. Ambil **3 menu rekomendasi**.

Pseudocode:

```
data = {
    "nasi goreng": {p:8, n:3},
    "bakso":       {p:5, n:7},
    "ayam bakar":  {p:9, n:4},
    "soto":        {p:4, n:2},
    "mie goreng":  {p:3, n:1}
}

skor = {}
for menu in data:
    skor[menu] = data[menu].p - data[menu].n

ranking = urutkan_desc(skor) // besar ke kecil
rekomendasi = ambil_teratas(ranking, 3)
cetak(ranking, rekomendasi)
```

Jawaban:

- Skor:
 - Nasi goreng: $8-3 = 5$
 - Ayam bakar: $9-4 = 5$
 - Soto: $4-2 = 2$
 - Mie goreng: $3-1 = 2$
 - Bakso: $5-7 = -2$
- **Ranking teratas:** Nasi goreng (5), Ayam bakar (5), Soto (2) (*atau Mie goreng jika ingin 4 besar*)

- 3 rekomendasi: Nasi goreng, Ayam bakar, Soto.

Pembahasan:

Skor sederhana sudah cukup untuk *ranking* awal. Di tahap berikutnya, bisa tambah bobot (misal, ulasan baru > ulasan lama) atau gabungkan *price constraint*.

Soal 5: Keras Mini-Model: Lengkapi & Jelaskan

Tugas:

Kamu punya 200 kalimat berlabel (**positif/negatif**). Lengkapi *pseudocode* Keras berikut agar bisa dilatih *offline*. Jelaskan kenapa pakai **validation_split**.

Pseudocode (lengkapi bagian “???”):

```
# 1) Tokenisasi & padding
tok = Tokenizer(num_words=5000, oov_token="<unk>")
tok.fit_on_texts(ulasan) # ulasan: list of string
seq = tok.texts_to_sequences(ulasan)
X = pad_sequences(seq, maxlen=40)

# 2) Label ke angka
label_map = {"negatif":0, "positif":1}
y = map_label(ulasan_label, label_map)

# 3) Bangun model
model = Sequential([
    Embedding(input_dim=5000, output_dim=32, input_length=40),
    GlobalAveragePooling1D(),
    Dense(16, activation="relu"),
    Dense(1, activation="sigmoid")
])

# 4) Kompilasi (lengkapi ???)
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])

# 5) Latih (lengkapi ???)
model.fit(X, y, epochs=5, batch_size=32, validation_split=0.2)
```

Jawaban:

- Bagian **compile** benar: **optimizer="adam"**, **loss="binary_crossentropy"**, **metrics=["accuracy"]**.
- Bagian **fit** benar: **validation_split=0.2** artinya 20% data dipakai untuk **validasi**.

Pembahasan:

- *Embedding* memetakan kata → vektor angka.

- *GlobalAveragePooling1D* merangkum fitur dengan cepat (ringan untuk laptop).
- *Sigmoid* cocok untuk biner (0/1).
- *validation_split=0.2* penting untuk **cek overfitting**. Jika *val accuracy* turun atau *val loss* naik saat *train loss* turun, kemungkinan *overfit*.

Semua komponen di atas *open source* dan bisa jalan *offline*.

Penutup

- Latihan ini melatih *end-to-end pipeline*: *preprocessing* → *model* → *metrics* → *visualisasi* → *rekomendasi*.
- Mulai dari *lexicon* dulu (cepat). Lanjut *TF-IDF* + *Logistic Regression*. Naik kelas ke *Keras* bila perlu.
- Simpan kode. Tulis laporan singkat. Siap *deploy* lokal di sekolah.

Fun Facts!

- Proyek ini adalah versi mini dari pekerjaan seorang **Data Scientist**.
- Kalian bisa pakai **Ollama** dan **OpenWebUI** untuk bantu kalian bikin kode atau menjelaskan hasil.
- **RAG lokal** bisa kalian pakai buat *chatbot* yang bisa menjawab pertanyaan tentang proyek kalian.

Tips Belajar atau Menghafal Cepat

- **Kerjakan Bertahap**: Jangan langsung bikin semua model. Mulai dari satu persatu.
- **Banyak Bertanya**: Jangan ragu untuk bertanya pada guru atau teman jika ada yang bingung.
- **Simpan Kodenya**: Simpan semua kode kalian di satu tempat. Ini bakal jadi portofolio kalian.

Aktivitas Siswa

Aktivitas 1: Siapkan Dataset & Preprocessing Dasar

Kita mulai dari fondasi. Kamu akan menyiapkan data komentar dan merapikannya. Tujuannya agar mesin gampang memahami.

Tujuan:

- Membuat CSV sederhana berisi komentar.

- Melakukan *preprocessing* (lowercase, hapus tanda baca/URL, buang *stopwords*).

Langkah:

- Buat **komentar.csv** (kolom: **ulasan**). Isi 20–50 baris komentar.
- Tentukan *stopwords* sederhana (mis. “dan”, “yang”, “di”).
- Lakukan pembersihan teks.

Pseudocode:

```
read_csv("komentar.csv") -> DF
for setiap DF.ulasan:
    t = to_lower(t)
    t = hapus_url(t)
    t = hapus_tanda_baca_angka(t)
    t = buang_stopwords(t)
simpan DF_bersih -> "komentar_bersih.csv"
```

Python (contoh ringkas):

```
import re, pandas as pd

stop =
{"dan", "yang", "di", "ke", "dari", "untuk", "atau", "pada", "dengan", "in", "itu"}

def bersihkan(s: str) -> str:
    s = s.lower()
    s = re.sub(r"http\S+|www\.\S+", " ", s)          # hapus URL
    s = re.sub(r"^[a-zA-Z\s]", " ", s)              # hapus non-huruf
    toks = [w for w in s.split() if w not in stop]  # buang stopwords
    return " ".join(toks)

df = pd.read_csv("komentar.csv")                    # kolom: ulasan
df["ulasan_bersih"] = df["ulasan"].astype(str).apply(bersihkan)
df.to_csv("komentar_bersih.csv", index=False)
print("Selesai: komentar_bersih.csv")
```

Aktivitas 2: Baseline Cepat: Lexicon Rule-Based

Sebelum model canggih, bikin **baseline** dulu. Sederhana tapi berguna.

Tujuan:

- Membuat pemilah sentimen *rule-based* pakai daftar kata positif/negatif.
- Mendapat baseline akurasi awal.

Langkah:

- Tentukan **lexicon**: daftar kata positif/negatif.
- Hitung banyaknya kata positif/negatif dalam kalimat.
- Tentukan label: lebih banyak mana.

Pseudocode:

```
positif = {keren, bagus, mantap, seru, hebat}
negatif = {jelek, buruk, kecewa, parah, benci}

function labelkan(teks):
    kata = split(teks)
    p = hitung_kecocokan(kata, positif)
    n = hitung_kecocokan(kata, negatif)
    if p>n -> "positif"
    elif n>p -> "negatif"
    else -> "netral"
apply ke "komentar_bersih.csv" -> kolom "pred_rule"
```

Python (contoh ringkas):

```
import pandas as pd

positif =
{"keren","bagus","mantap","seru","hebat","mantul","recommended"}
negatif =
{"jelek","buruk","kecewa","parah","benci","payah","sampah"}

def sentimen_rule(t: str) -> str:
    kata = str(t).split()
    p = sum(w in positif for w in kata)
    n = sum(w in negatif for w in kata)
    if p > n: return "positif"
    if n > p: return "negatif"
    return "netral"

df = pd.read_csv("komentar_bersih.csv") # kolom: ulasan_bersih
df["pred_rule"] = df["ulasan_bersih"].apply(sentimen_rule)
df.to_csv("hasil_rule.csv", index=False)
print(df["pred_rule"].value_counts())
```

Aktivitas 3: TF-IDF + Logistic Regression (scikit-learn)

Saatnya naik kelas. Kita ubah teks jadi angka pakai **TF-IDF** lalu latih **Logistic Regression**.

Tujuan:

- Membuat model ML sederhana untuk sentimen.
- Mengukur *accuracy/precision/recall/F1*.

Langkah:

- Siapkan `ulasan.csv` dengan kolom: `ulasan`, `label` (positif/negatif).
- Bagi data: **train/test split**.
- *Vectorize* teks pakai **TfidfVectorizer**.
- Latih **LogisticRegression** dan evaluasi.

Pseudocode:

```
data = read_csv("ulasan.csv")
X = data.ulasan ; y = data.label
Xtr, Xte, ytr, yte = train_test_split(X,y, test_size=0.2,
stratify=y)
tfidf = fit_TFIDF(Xtr)
Xtrv = tfidf.transform(Xtr)
Xtev = tfidf.transform(Xte)
clf = fit_LogReg(Xtrv, ytr)
yp = clf.predict(Xtev)
laporkan_metrics(yte, yp)
```

Python (contoh ringkas):

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,
confusion_matrix

df = pd.read_csv("ulasan.csv") # kolom: ulasan,label
(positif/negatif)
Xtr, Xte, ytr, yte = train_test_split(
    df["ulasan"], df["label"], test_size=0.2, random_state=42,
    stratify=df["label"]
)

tfidf = TfidfVectorizer(lowercase=True, max_features=5000,
ngram_range=(1,2))
Xtrv = tfidf.fit_transform(Xtr)
Xtev = tfidf.transform(Xte)

clf = LogisticRegression(max_iter=1000)
clf.fit(Xtrv, ytr)
yp = clf.predict(Xtev)

print(classification_report(yte, yp))
print("Confusion Matrix:\n", confusion_matrix(yte, yp))
```

Aktivitas 4: *Neural Network* Mini dengan Keras

Cobain *neural network* ringan. Cepat, bisa offline di laptop sekolah.

Tujuan:

- Melatih NN sederhana (Embedding → Pooling → Dense).
- Memahami *validation_split* untuk cegah *overfitting*.

Langkah:

- Pakai data *ulasan.csv* (positif/negatif).
- Tokenisasi → urutan angka → *padding*.
- Bangun model Keras sederhana.
- Latih dan lihat akurasi *train/validation*.

Pseudocode:

```
tokenize(ulasan) -> sequences
pad(sequences, maxlen=40) -> X
map label: negatif->0, positif->1 -> y
model = Embedding -> GlobalAveragePooling1D -> Dense(relu) ->
Dense(sigmoid)
compile(optimizer=adam, loss=binary_crossentropy, metrics=accuracy)
fit(X,y, epochs=5, batch=32, validation_split=0.2)
```

Python (contoh ringkas):

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

df = pd.read_csv("ulasan.csv") # ulasan,label
lab = {"negatif":0, "positif":1}
y = df["label"].map(lab).values

tok = Tokenizer(num_words=5000, oov_token="<unk>")
tok.fit_on_texts(df["ulasan"].astype(str))
seq = tok.texts_to_sequences(df["ulasan"].astype(str))
X = pad_sequences(seq, maxlen=40)

model = models.Sequential([
    layers.Embedding(input_dim=5000, output_dim=32, input_length=40),
    layers.GlobalAveragePooling1D(),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])
hist = model.fit(X, y, epochs=5, batch_size=32, validation_split=0.2,
verbose=0)
print("Train acc (last):", hist.history["accuracy"][-1])
print("Val acc (last):", hist.history["val_accuracy"][-1])
```

Aktivitas 5: Visualisasi & Laporan Mini

Hasil tanpa visual kurang “nendang”. Kita bikin grafik dan ringkasan akhir.

Tujuan:

- Membuat grafik komposisi sentimen dan *confusion matrix*.
- Menulis ringkasan 5–7 kalimat tentang hasil.

Langkah:

- Gunakan prediksi dari Aktivitas 3 atau 4.
- Buat **bar chart** jumlah **positif/negatif/netral** (atau 2 kelas).
- Tampilkan **confusion matrix**.
- Tulis ringkasan: data, metode, hasil, saran perbaikan.

Pseudocode:

```
pred = hasil_prediksi_model
hitung_count_per_label(pred) -> dict_count
plot_bar(dict_count)
plot_confusion_matrix(y_true, pred)
tuliskan_ringkasan( data, metode, akurasi, poin-kuat, batasan, ide-lanjut
)
```

Python (contoh ringkas):

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

# Misal kita punya file hasil prediksi dari Aktivitas 3
df = pd.read_csv("ulasan.csv")          # ulasan,label
pred = pd.read_csv("pred_lr.csv")        # file hasilmu, kolom: pred
# Atau gabungkan dulu saat infer, lalu:
# pred = clf.predict(Xtev) -> simpan ke CSV

counts = pred["pred"].value_counts().sort_index()
counts.plot(kind="bar", rot=0)
plt.title("Komposisi Sentimen")
plt.xlabel("Label"); plt.ylabel("Jumlah")
plt.tight_layout(); plt.show()

cm = confusion_matrix(df["label"][:len(pred)], pred["pred"])
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues"); plt.title("Confusion Matrix");
plt.show()
```

Ringkasan

- Dataset berisi 200 komentar berbahasa Indonesia dari media sosial.

- *Preprocessing* meliputi lowercase, hapus URL/tanda baca, dan *stopwords*.
- Model TF-IDF + Logistic Regression mencapai akurasi 84%, *macro-F1* 0,82.
- Kesalahan umum terjadi pada kalimat sarkas dan campuran positif-negatif.
- Perbaikan: tambahkan normalisasi bahasa gaul dan deteksi negasi.
- Untuk lanjutan, coba *neural network* Keras dan tambah data berlabel.

Catatan Penting

- Semua tool **open source** dan bisa **offline**.
- Urutan kerja yang aman: *preprocessing* → *fitur* → *model* → *evaluasi* → *visualisasi*.
- Bandingkan **baseline rule-based** vs **TF-IDF + LR** vs **Keras NN**.
- Simpan semua hasil sebagai **portofolio** (nilai plus untuk magang/kerja).

Kalau mau, saya bisa bantu sediakan **template CSV** dan **skrip uji siap jalan** (tinggal ganti datanya).

Rangkuman

Di bab ini kamu bikin proyek nyata: **analisis sentimen** komentar media sosial. Alurnya sederhana: kumpulkan *dataset* → lakukan *preprocessing* (huruf kecil, buang tanda baca, bersih dari tautan) → ubah teks jadi angka (mis. *TF-IDF*) → latih **model** → cek hasil pakai *metrics*. Intinya: **teks** → **angka** → **model** → **skor** → **grafik**. Kamu belajar kerja **end-to-end**, dari awal sampai **visualisasi** dan laporan. Ini mirip kerja **Data Scientist** versi mini, tapi tetap seru dan ringan.

Semua toolnya **open source** dan bisa **offline**: **Python**, **Pandas** untuk olah data; *scikit-learn* untuk *TF-IDF*, **Logistic Regression**, dan evaluasi; **Keras (neural network)** kalau mau naik level; Matplotlib atau **Orange Data Mining** untuk grafik. Mulai dulu dengan aturan kata (*lexicon rule-based*) yang cepat, lalu bandingkan dengan **model** sederhana. Lihat **accuracy**, **precision**, **recall**, **F1** agar tahu seberapa bagus modelmu. **Baseline** dulu, baru tingkatkan.

Selama proyek, jaga **etika**: lindungi privasi, hindari bias, dan hati-hati pada ujaran kebencian. Tulis batasan modelmu dengan jujur—*sarcasm* itu susah untuk mesin. Simpan kode dan hasil sebagai **portofolio**, karena ini berguna untuk magang dan kerja. Terus eksperimen, kolaborasi dengan tim, dan **iterasi** kecil tapi rutin. Dengan kebiasaan ini, kamu siap **rilis** solusi kecil, *deploy offline*, dan tampil percaya diri sebagai pembuat **AI beneran**.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. *Preprocessing* biasa termasuk mengubah huruf jadi kecil semua.
Jawaban: Benar

2. *TF-IDF* mengubah teks langsung jadi label “positif/negatif”.
Jawaban: Salah
3. *Lexicon rule-based* memakai daftar kata positif/negatif.
Jawaban: Benar
4. *Accuracy* selalu lebih baik dilihat sendiri tanpa *precision/recall*.
Jawaban: Salah
5. Orange Data Mining bisa dipakai untuk visualisasi hasil.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Langkah pertama setelah baca CSV komentar adalah...
 A. Latih model dulu
 B. *Preprocessing* teks
 C. Buat grafik dulu
 D. Simpan ke JSON
 Jawaban: B
2. Fungsi *stopwords* adalah...
 A. Menambah kata baru
 B. Menghapus kata umum tidak penting
 C. Mengurutkan kata
 D. Mengganti huruf
 Jawaban: B
3. Dalam *TF-IDF*, kata yang jarang muncul tapi penting akan...
 A. Bobot kecil
 B. Dihapus
 C. Bobot besar
 D. Jadi angka nol
 Jawaban: C
4. Untuk klasifikasi biner sederhana, aktivasi output *Keras* yang tepat adalah...
 A. *relu*
 B. *softmax*
 C. *tanh*
 D. *sigmoid*
 Jawaban: D
5. *Precision* mengukur...
 A. Proporsi prediksi benar dari semua data
 B. Ketepatan saat model bilang “positif”
 C. Kemampuan menemukan semua “positif”
 D. Rata-rata *loss*

Jawaban: B

6. *Recall* mengukur...
- A. Ketepatan saat model bilang “positif”
 - B. Proporsi “positif” yang berhasil ditemukan
 - C. Banyaknya kelas
 - D. Kecepatan latihan

Jawaban: B

7. Contoh *preprocessing* yang benar:
- A. Menambah angka acak
 - B. *Lowercasing*, hapus tanda baca
 - C. Acak urutan kata
 - D. Tambah *emoji*

Jawaban: B

8. Kode Python mana yang benar membaca CSV kolom ulasan?
- A. `pd.open('ulasan.csv')`
 - B. `pd.read('ulasan.csv')`
 - C. `pd.read_csv('ulasan.csv')`
 - D. `pd.csv('ulasan.csv')`

Jawaban: C

9. Untuk membagi data latihan/uji di scikit-learn, fungsi yang biasa dipakai adalah...
- A. `train_test_split`
 - B. `split_train_test`
 - C. `divide_data`
 - D. `data_splitter`

Jawaban: A

10. Grafik terbaik untuk melihat jumlah “positif/negatif/netral” cepat adalah...
- A. *Line chart*
 - B. *Bar chart*
 - C. *Scatter plot*
 - D. *Heatmap*

Jawaban: B

C. Isian Singkat (5 Soal)

1. Istilah untuk mengubah teks menjadi angka fitur disebut _____.
Jawaban: *feature extraction*
2. Dua metrik evaluasi selain *accuracy* adalah _____ dan _____.
Jawaban: *precision, recall*

3. Sebutkan satu pustaka Python untuk vektorisasi teks: _____.
Jawaban: *TfidfVectorizer* (scikit-learn)
4. Dalam *rule-based*, daftar kata penentu sentimen disebut _____.
Jawaban: *lexicon*
5. Aktivasi output untuk klasifikasi biner di *Keras* adalah _____.
Jawaban: *sigmoid*

D. Soal Explorasi (3 Soal)

Soal 1:

Kamu diberi 30 komentar campuran (ada *emoji*). Buat *pipeline* mini: *preprocessing* (lowercase, hapus tanda baca/URL), deteksi *emoji* positif/negatif, lalu labelkan: positif/negatif/netral. Tulis *pseudocode* dan tampilkan ringkasan jumlah per kelas.

Kunci jawaban (contoh ringkas):

Pseudocode:

```
emoji_pos = ["😄", "😍", "😊", "👍"]
emoji_neg = ["😡", "😭", "👎"]

load csv as data
function clean(t): return lower(hapus_tanda_baca_url(t))
function label(t):
    s = sum(1 for e in emoji_pos if e in t) - sum(1 for e in
emoji_neg if e in t)
    if s>0: return "positif"
    if s<0: return "negatif"
    return "netral"
pred = [label(clean(x)) for x in data.ulasan]
hitung_jumlah_per_kelas(pred)
```

Contoh Python:

```
import re
emoji_pos={"😄","😍","😊","👍"}; emoji_neg={"😡","😭","👎"}
def clean(s): return re.sub(r"http\S+|[\^0-9A-Za-z
\u1F300-\u1FAFF]+", " ",s.lower())
def lab(s):
    p=sum(e in s for e in emoji_pos); n=sum(e in s for e in
emoji_neg)
    return "positif" if p>n else ("negatif" if n>p else "netral")
```

Soal 2:

Bandingkan dua cara: *lexicon rule-based* vs *TF-IDF + Logistic Regression* pada 100 komentar berlabel. Tulis langkah, *pseudocode*, dan simpulkan pemenang berdasarkan **macro-F1**.

Kunci jawaban (contoh ringkas):

Langkah: split 80/20, latih LR pada *TF-IDF*, *rule-based* pakai daftar kata.

Pseudocode:

```
X,y = data.ulasan, data.label
Xtr,Xte,ytr,yte =
train_test_split(X,y,test_size=0.2,random_state=42)
# Rule-based
pred_rule = [rule(u) for u in Xte]
# TF-IDF + LR
vec = TfidfVectorizer(max_features=5000,ngram_range=(1,2))
Xtrv=vec.fit_transform(Xtr); Xtev=vec.transform(Xte)
clf=LogisticRegression(max_iter=1000).fit(Xtrv,ytr)
pred_lr=clf.predict(Xtev)
macroF1_rule = f1(yte,pred_rule,average="macro")
macroF1_lr    = f1(yte,pred_lr,average="macro")
pilih_model = "LR" if macroF1_lr>macroF1_rule else "Rule"
```

Kesimpulan contoh: **LR** menang macro-F1, karena menangkap pola *ngram*.

Soal 3:

Buat mini-*dashboard* teks: hitung persentase “positif” per hari (H1–H5) lalu tampilkan rekomendasi “hari terbaik rilis konten”. Beri *pseudocode* dan contoh perhitungan singkat.

Kunci jawaban (contoh ringkas):

Pseudocode:

```
grup = group_by(data, "tanggal")
for tgl in grup:
    total = len(grup[tgl])
    pos   = count_label(grup[tgl], "positif")
    prop[tgl] = pos/total
hari_terbaik = argmax(prop)
cetak(prop, hari_terbaik)
```

Contoh angka: H1=60%, H2=55%, H3=72%, H4=58%, H5=65% → H3 terbaik.

Opsional kode:

```
prop = df.groupby("tanggal")["pred"].apply(lambda
s:(s=="positif").mean())
hari_terbaik = prop.idxmax()
```

Tugas Proyek

Proyek 1: Emoji & Negasi Detector

“Ketawa 😄 itu positif, tapi kalau ‘nggak lucu 😄’ gimana?”

Target:

Buat *scorer* sentimen sederhana yang memahami **emoji** dan **kata negasi** (*tidak, nggak, bukan*). Ada **zona netral**.

Langkah:

- Buat CSV `komentar_emoji.csv` (kolom: `teks`, `label` opsional).
- Kamus emoji: positif = {😄, 😍, 😊, 👍}, negatif = {😡, 😭, 👎}.
- Tambahkan deteksi **negasi**: jika ada *negasi* di 3 kata sebelum kata positif/negatif → **balik** skornya.
- Skor akhir = (positif – negatif).
- Terapkan **threshold netral**: jika $|skor| < 1 \rightarrow$ **netral**.

Pseudocode:

```
emoji_pos = ["😄", "😍", "😊", "👍"]
emoji_neg = ["😡", "😭", "👎"]
negasi = ["tidak", "nggak", "ga", "bukan", "tak"]

function skor_kata(k):
    if k in emoji_pos: return +1
    if k in emoji_neg: return -1
    return 0

function ada_negasi_sebelum(tokens, i, jangkauan=3):
    start = max(0, i - jangkauan)
    for t in tokens[start:i]:
        if t in negasi: return True
    return False

function sentimen(teks):
    tokens = tokenize(lower(teks))
    s = 0
    for i, k in enumerate(tokens):
        v = skor_kata(k)
        if v != 0 and ada_negasi_sebelum(tokens, i):
            v = -v
        s += v
    if abs(s) < 1: return "netral"
    if s > 0: return "positif"
    return "negatif"
```

Contoh Jawaban (Python ringkas):

```
import re, pandas as pd
```

```

emoji_pos = {"😄", "😍", "😊", "👍"}
emoji_neg = {"😡", "😭", "👎"}
negasi = {"tidak", "nggak", "ga", "bukan", "tak"}

def toks(s): return
re.findall(r"\w+|[\U0001F300-\U0001FAFF]|[\^\w\s]", s.lower())

def ada_negasi(tokens, i, r=3):
    return any(t in negasi for t in tokens[max(0,i-r):i])

def sentimen(s):
    tokens = toks(s)
    score = 0
    for i,k in enumerate(tokens):
        v = 1 if k in emoji_pos else (-1 if k in emoji_neg else
0)
        if v and ada_negasi(tokens,i): v = -v
        score += v
    return "netral" if abs(score)<1 else ("positif" if score>0
else "negatif")

# df = pd.read_csv("komentar_emoji.csv"); df["pred"] =
df["teks"].apply(sentimen)

```

Proyek 2: Normalisasi Bahasa Gaul

“Mantul bgt sih, ngakak pol” → normal ke bentuk baku sebelum analisis.

Target:

Buat **kamus gaul** kecil untuk menormalkan teks sebelum *preprocessing*. Bandingkan **akurasi** sebelum vs sesudah normalisasi.

Langkah:

- Buat **kamus_gaul.json** (contoh: *mantul*→*mantap*, *bgt*→*banget*, *ngakak*→*lucu*).
- Pipeline A: langsung analisis (tanpa normalisasi).
- Pipeline B: normalisasi → analisis (lexicon/TF-IDF + Naive Bayes).
- Ukur **accuracy/precision/recall** pada data berlabel 100–200 baris.

Pseudocode:

```

kamus = load_json("kamus_gaul.json")

function normalisasi(teks):
    tokens = tokenize(lower(teks))
    return " ".join(kamus.get(t, t) for t in tokens)

function analisis_A(teks): return model_or_lexicon(teks)
function analisis_B(teks): return
model_or_lexicon(normalisasi(teks))

```

```

predA = [analisis_A(t) for t in data.teks]
predB = [analisis_B(t) for t in data.teks]
skorA = hitung_metrics(data.label, predA)
skorB = hitung_metrics(data.label, predB)
laporkan_perbandingan(skorA, skorB)

```

Contoh Jawaban (Python mini untuk normalisasi):

```

import json, re
kamus =
{"mantul": "mantap", "bgt": "banget", "ngakak": "lucu", "btw": "sebenarn
ya", "gk": "tidak"}

def norm(s):
    tok = re.findall(r"[a-zA-Z]+", s.lower())
    return " ".join(kamus.get(t, t) for t in tok)

```

Proyek 3: Hashtag Sentiment Radar

"#PentasSeni2025 lebih banyak positif atau negatif per hashtag?"

Target:

Hitung sentimen per **hashtag**. Tampilkan **top 5** hashtag paling positif. Berikan rekomendasi hashtag untuk kampanye berikutnya.

Langkah:

- CSV **hashtag.csv** (kolom: **teks**, **hashtag** "dipisah koma", **label** opsional).
- Gunakan **lexicon rule-based** atau **TF-IDF + Logistic Regression**.
- Agregasi per hashtag → skor = jumlah(positif) – jumlah(negatif).
- Buat **bar chart** 10 hashtag teratas.

Pseudocode:

```

function klasifikasi(teks): ... // bebas: lexicon atau model
skor = dict_default(0)

for baris in data:
    y = klasifikasi(baris.teks) //
    "positif"/"negatif"/"netral"
    for h in split_hashtag(baris.hashtag):
        if y=="positif": skor[h]+=1
        elif y=="negatif": skor[h]-=1

ranking = sort_desc(skor)
top5 = head(ranking, 5)
plot_bar(top10(ranking))

```

Contoh Jawaban (Python ringkas agregasi):

```

from collections import defaultdict
skor = defaultdict(int)

def klasifikasi(teks): ... # isi sendiri (lexicon atau model)
for _,row in df.iterrows():
    y = klasifikasi(row["teks"])
    for h in [x.strip().lower() for x in
row["hashtag"].split(",")]:
        if y=="positif": skor[h]+=1
        elif y=="negatif": skor[h]-=1

top5 = sorted(skor.items(), key=lambda x:-x[1])[:5]

```

Proyek 4: Model Duel: *Rule-Based* vs TF-IDF + Logistic Regression

“Siapa juara di datamu?”

Target:

Bandingkan **dua pendekatan** pada *dataset* yang sama. Pilih pemenang dengan alasan **angka** (*metrics*) dan **analisis error**.

Langkah:

- Siapkan CSV 200–300 baris, tiga kelas: **positif/negatif/netral**.
- Model 1: **lexicon rule-based** (daftar kata + negasi sederhana).
- Model 2: **TF-IDF + Logistic Regression** (scikit-learn).
- Ukur **accuracy, macro-F1**; tampilkan **confusion matrix**.
- Tulis 3 contoh **salah klasifikasi** dan jelaskan kenapa.

Pseudocode:

```

X, y = data.teks, data.label
Xtr, Xte, ytr, yte = split(X, y, 0.2, seed=42)

pred_rule = [rule_based(t) for t in Xte]
tfidf = fit_tfidf(Xtr); Xtrv=tfidf.transform(Xtr);
Xtev=tfidf.transform(Xte)
logreg = fit_logreg(Xtrv, ytr); pred_lr = logreg.predict(Xtev)

m_rule = metrics(yte, pred_rule)    // acc, macroF1, cm
m_lr   = metrics(yte, pred_lr)      // acc, macroF1, cm
bandingkan(m_rule, m_lr); pilih_pemenang()

```

Contoh Jawaban (Python super ringkas evaluasi):

```

from sklearn.metrics import accuracy_score, f1_score,
ConfusionMatrixDisplay
acc_rule = accuracy_score(yte, pred_rule)
f1_rule  = f1_score(yte, pred_rule, average="macro")
acc_lr   = accuracy_score(yte, pred_lr)
f1_lr    = f1_score(yte, pred_lr, average="macro")

```

```
# Pilih model dg f1 lebih tinggi; jelaskan 2-3 error case.
```

Proyek 5: Spam-First Pipeline (Filter → Sentimen)

“Pisahkan promosi/spam dulu. Baru analisis sentimen yang ‘manusiawi’.”

Target:

Bangun **dua tahap**: (1) **deteksi spam** sederhana, (2) **analisis sentimen** hanya untuk komentar **non-spam**.

Langkah:

- CSV `komentar_mixed.csv` (kolom: `teks`, `label_spam` opsional, `label_sentimen` opsional).
- **Deteksi spam**: regex untuk pola *promo/WA/link/discount*, hitung kata “beli/klik/subscribe/daftar” ≥ ambang.
- Komentar yang **spam** → tandai & **jangan** dihitung sentimennya.
- Komentar **non-spam** → jalankan sentimen (lexicon atau TF-IDF + NB).
- Laporkan 2 *metrics*: **spam F1** dan **sentimen F1** (pada non-spam).

Pseudocode:

```
spam_kw =
["promo", "diskon", "klik", "subscribe", "daftar", "hubungi", "whatsapp",
 "wa"]
regex_link = r"http[s]?://|bit\.ly|wa\.me"

function is_spam(teks):
    t = lower(teks)
    hit = count_contains(t, spam_kw) + count_regex(t, regex_link)
    return hit >= 2

nonspam, spam = [], []
for baris in data:
    if is_spam(baris.teks): spam.append(baris)
    else: nonspam.append(baris)

pred_sent = [klasifikasi(b.teks) for b in nonspam]
nilai_spam = metrics_spam(data.label_spam, [is_spam(b.teks) for b
in data])
nilai_sent = metrics_sentimen(nonspam.label_sentimen, pred_sent)
laporkan(nilai_spam, nilai_sent)
```

Contoh Jawaban (Python ringkas deteksi spam):

```
import re
spam_kw =
{"promo", "diskon", "klik", "subscribe", "daftar", "hubungi", "whatsapp",
 "wa"}
pat = re.compile(r"(https?:://|bit\.ly|wa\.me)")
```

```
def is_spam(s):  
    s = s.lower()  
    hit = sum(k in s for k in spam_kw) + bool(pat.search(s))  
    return hit >= 2
```

Catatan Umum

- Gunakan *open source*: **Python**, **Pandas**, **scikit-learn**, **Keras** (opsional), **Matplotlib/Orange**.
- Simpan **pipeline** sederhana: *preprocessing* → *fitur* → *model* → *metrics* → *visualisasi*.
- Tulis **laporan 1–2 halaman**: tujuan, data, metode, hasil (angka & grafik), analisis error, dan kesimpulan singkat.

Institut Teknologi Tangerang Selatan (ITTS) dan Penulis.

Para penulis adalah **dosen aktif di Institut Teknologi Tangerang Selatan (ITTS)**, sebuah **institusi pendidikan tinggi berbasis teknologi** yang berperan aktif dalam pengembangan **solusi digital** dan *kecerdasan buatan (AI)*. Tidak hanya mengajar, mereka juga merupakan **praktisi dan inovator** yang menghasilkan **karya-karya nyata di bidang teknologi mandiri**.

Berlokasi di **Komplek Komersial BSD, Serpong Utara, Tangerang Selatan**, ITTS hadir sebagai **pusat inovasi** yang mengintegrasikan **pembelajaran, penelitian, dan pengembangan teknologi secara menyeluruh**. Fokus utama ITTS mencakup bidang strategis seperti *AI, Machine Learning, Deep Learning*, jaringan, *cloud*, keamanan siber, pemrograman, sistem informasi, dan multimedia.

ITTS menawarkan **lingkungan akademik yang dinamis dan kolaboratif**. Setiap minggu, diselenggarakan berbagai *workshop, demo teknologi*, seminar, hingga *webinar*—mayoritas **gratis dan tersedia secara terbuka di YouTube**. Ini menciptakan **kultur belajar yang aktif dan relevan dengan perkembangan industri**.

Fasilitas ITTS menunjang kegiatan riset dan inovasi secara optimal. Lab komputer dilengkapi *GPU RTX 4060* untuk eksperimen *AI*, serta **laboratorium teknologi dengan puluhan server, perangkat IoT, sistem komunikasi radio, dan infrastruktur jaringan jarak jauh**. Dari ekosistem ini lahir **produk-produk mandiri** seperti **ChatGPT versi lokal** untuk institusi pendidikan dan **sistem perpustakaan digital yang dapat diakses tanpa koneksi internet**.

Daya saing ITTS juga tercermin dari karakter dosen dan mahasiswa yang tidak hanya akademik, tetapi juga **produktif secara praktis**. Mahasiswa didorong aktif menjadi **narasumber, penulis, dan peneliti** sejak dini. Banyak diantaranya telah menghasilkan **karya buku, artikel ilmiah, serta terlibat langsung dalam proyek-proyek teknologi**.

Kolaborasi strategis dengan berbagai mitra industri—seperti *IDCloudHost, Ubuntu, XecureIT, Dinas KOMINFO Tangerang Selatan*, berbagai *ISP*, dan komunitas IT nasional—menjadi **jembatan nyata antara kampus dan dunia profesional**. Hasilnya, **lebih dari 75% mahasiswa ITTS telah bekerja bahkan sebelum lulus, dan hampir seluruh lulusan langsung terserap industri**.

ITTS bukan hanya tempat belajar, tapi tempat mencipta. Dosen sebagai **penulis dan praktisi**, mahasiswa sebagai **peneliti dan kontributor**—semuanya berperan **membangun ekosistem teknologi yang mandiri dan berdaya saing tinggi**.

Institut Teknologi Tangerang Selatan (ITTS)

Komplek Komersial BSD

Jl. Raya Serpong No. Kav. 9, Lengkong Karya, Serpong Utara

Kota Tangerang Selatan, Banten 15331

☎ (+62) 877-7277-1775

✉ info@itts.ac.id

🌐 www.itts.ac.id